
TSOpt

Training Set Optimisation for Genomic Prediction

Theory, Algorithms and Practice

A complete tutorial and reference manual for the R package TSOpt

version 0.5.0

65 functions · 18 criteria · exact fixed effects, robust multi-trait × multi-environment design, the value of every plot and trait, field layout, economics, whom to genotype, transfer between programmes, BrAPI, major genes, Pareto fronts, panels of tens of thousands

Waseem Hussain

PlantNura Consulting & Solutions, Hyderabad, India

waseemhussain@plantnura.com

PlantNura · KultivateEd · Genomic Selection Masterclass

30 September 2026

Every number, table and figure in this document is produced by the code shown next to it, run against the installed package when the document is built. TSOpt is free of charge for everyone under the TSOpt Free Licence; each user registers with PlantNura (waseemhussain@plantnura.com) for a personal licence key. It is proprietary software of PlantNura Consulting & Solutions and may not be copied or redistributed; this tutorial may be shared freely.

Contents

How to use and cite this tutorial	4
1 Introduction	6
1.1 The problem	6
1.2 What existed before	6
1.3 What TSOpt adds	7
1.4 Where it sits	8
2 Theory	10
2.1 The mixed model and the prediction error variance	10
2.2 Contrasts and the coefficient of determination	10
2.3 The rank-one engine	12
2.4 The fixed intercept, made exact	12
2.5 General fixed effects, made exact	13
2.6 Heritability and λ	15
2.7 Which contrasts: targeted and untargeted designs	15
3 The criteria catalogue	16
3.1 Tier 1, Recommended: robust CDmean	17
3.2 Tier 2, Specialist	17
3.3 Tier 3, Classical optimal design	19
3.4 Tier 4, Model-free	19
3.5 Tier 5, Baselines	19
3.6 All criteria on one panel	19
4 Search algorithms	21
4.1 Greedy (forward selection)	21
4.2 Greedy + exchange (modified Fedorov)	22
4.3 ROTS: the Rank-One Tabu Search	22
4.4 GA + SA: for everything else	23
4.5 Which search runs when	24
5 Data in: any format, one object	25
5.1 <code>tso_licence()</code> : activating TSOpt	25
5.2 <code>tso_read()</code> : one reader for every format	26
5.3 <code>tso_qc()</code> : quality control with a log	26
5.4 <code>tso_kinship()</code> : relationship matrices	27
5.5 <code>tso_kernels()</code> : dominance, epistasis, pedigree and single-step	27
5.6 <code>tso_omics_kernel()</code> : transcriptome, metabolome and spectra	29
5.7 <code>tso_variance()</code> : the variance model from your trials	30
5.8 <code>tso_data()</code> : the object every verb takes	30
5.9 <code>tso_write()</code> : converting between formats	32
6 Diagnose, design, explain	33
6.1 <code>tso_diagnose()</code> : is optimisation worth it here?	33
6.2 <code>tso_design()</code> : the main verb	34
6.3 <code>tso_explain()</code> : what exactly was run	37
7 Breeding systems	38
7.1 Inbred lines and doubled haploids	38
7.2 Hybrids: which crosses to make	38
7.3 Clonal and polyploid crops	40
7.4 Sparse multi-environment testing	40
7.5 Testcrosses and three or more heterotic groups	42

7.6	Outcrossing crops and families	42
7.7	Genebanks and reference panels	42
8	Beyond one trait, one stage	43
8.1	Multi-trait design: which trait on which line	43
8.2	Traits and environments together	44
8.3	Robust multi-trait design: uncertain correlations and heritabilities	45
8.4	What is each trait worth? Proxy and high-throughput traits	46
8.5	Robust design over an uncertain heritability	47
8.6	Two-stage designs: where the next plots go	47
8.7	Across seasons: <code>tso_update()</code>	49
8.8	The long term: <code>tso_simulate()</code>	49
9	Major genes and haplotypes	51
9.1	<code>tso_major_scan()</code> : finding the loci	51
9.2	<code>tso_major_effects()</code> : loci from marker effects you already have	52
9.3	<code>tso_haplotypes()</code> : alleles of a block	54
9.4	Genome-wide haplotypes: blocks, the haplotype kernel and coverage	54
9.5	Three ways to protect a major gene	56
9.6	The introgression demonstration	57
9.7	Allele and haplotype coverage in every plan	59
9.8	Weighted kinship	59
10	More decision tools	61
10.1	<code>tso_sample_size()</code> : how many lines do I need?	61
10.2	<code>tso_topk()</code> : will this set find the best lines?	62
10.3	<code>tso_pareto()</code> : accuracy against diversity, cost or anything else	63
10.4	<code>tso_contribution()</code> : which parents, and how many crosses each	64
10.5	<code>tso_design_large()</code> : panels of tens of thousands	65
10.6	Multi-start search	66
10.7	Safe custom criteria	67
11	Decisions around the design	68
11.1	The value of each plot	68
11.2	<code>tso_uncertainty()</code> : is the advantage real?	70
11.3	<code>tso_economics()</code> : how many plots pay off?	71
11.4	<code>tso_layout()</code> : lay out the field	72
11.5	<code>tso_genotype_set()</code> : choose whom to genotype	74
11.6	<code>tso_transfer()</code> : predict another programme	76
11.7	<code>tso_backtest()</code> : forward validation on your own history	77
11.8	<code>tso_season()</code> : where the plots go across stages	77
11.9	<code>tso_crosses()</code> : which crosses to make	77
11.10	<code>tso_imputation_plan()</code> : genotyping and plots under one budget	77
11.11	<code>tso_omics_plan()</code> : profiles or plots?	78
11.12	To the field, the database and other software	78
11.13	Kernel robustness: when the trait is not additive	78
11.14	BrAPI: design from the breeding database	79
12	Check, show and share	81
12.1	<code>tso_score()</code> : any set, any criterion	81
12.2	<code>tso_compare()</code> : the plan against the alternatives	81
12.3	<code>tso_evaluate()</code> : after the season	82
12.4	The figure gallery	83
12.5	<code>tso_save_figure()</code> : journal-ready files	85
12.6	<code>tso_export()</code> : the list for the field team	85
12.7	<code>tso_report()</code> : one page for the programme manager	85

12.8	tso_cli(): the command line	86
13	The head-to-head benchmark	87
13.1	Design	87
13.2	Results	87
13.3	What the benchmark taught us	91
13.4	Reproducing it	92
14	Recipes, FAQ and crosswalk	93
14.1	Recipes	93
14.2	FAQ	97
14.3	Crosswalk from other packages	99
14.4	Troubleshooting	100
15	Reference	102
15.1	Function index	102
15.2	Glossary	120
15.3	Session information	122
	References	122

How to use and cite this tutorial

How to cite

The package. Hussain, W. (2026). *TSOpt: Training Set Optimisation for Genomic Prediction*. R package version 0.5.0. PlantNura Consulting & Solutions. Free licence on request from waseemhussain@plantnura.com.

This tutorial. Hussain, W. (2026). *TSOpt: Training Set Optimisation for Genomic Prediction — Theory, Algorithms and Practice*. Tutorial and reference manual for TSOpt 0.5.0. PlantNura / KultivateEd Genomic Selection Masterclass.

The benchmark. Hussain, W. (2026). *TSOpt head-to-head benchmark: six public panels, 33 scenarios*. PlantNura Consulting & Solutions (on request). The major-gene benchmark of Part 9 is benchmark/major_genes_benchmark.R.

The methods. When you describe a design made with TSOpt in a paper, cite the criterion (Part 3 gives the original reference for each), the search (Part 4), and this tutorial for the engine, the exact fixed effects, the robust default and the major-gene protection; the decision tools of Parts 10 and 11 cite their own sources. `tso_explain(plan)` prints the exact specification to paste into a methods section.

Who this is for. Breeders, quantitative geneticists and students who have a genotyped population and a phenotyping budget. You need basic R (reading a file, calling a function, looking at a data frame). You do not need to know mixed models to *use* the package; Part 2 explains the theory for those who want it, always by hand first.

How it is organised.

Part	What it covers	Read it when
1	The problem, what exists, what TSOpt adds	always
2	Theory: mixed model, PEV, CD, the rank-one engine, exact intercept and fixed effects	you want to understand or cite the method
3	Every criterion: definition, formula, when to use, worked example	you wonder whether to change the default
4	Every search algorithm, including the Rank-One Tabu Search	you want to know how the set is found
5	Data in: any genotype format, QC, kinship and genetic kernels, the <code>tso_data</code> object	day one
6	<code>tso_diagnose</code> , <code>tso_design</code> , <code>tso_explain</code> : every argument	day one
7	Breeding systems: inbreds, hybrids, testcrosses, clones, sparse multi-environment testing (dense or factorised), outcrossers, genebanks	your crop is not a simple inbred panel
8	Multi-trait (with environments), robust trait parameters, the value of each trait, robust heritability, two-stage, cycles, long-term simulation	you plan beyond one trial
9	Major genes and haplotypes: the scan, protection, allele coverage	your trait has a known or suspected major gene
10	More decision tools: sample size, top-k, Pareto fronts, parents, very large panels	you face a decision beyond “which n lines”

Part	What it covers	Read it when
11	Decisions around the design: the value of each plot, uncertainty, economics, field layout, whom to genotype, transfer, forward validation, season planning, crosses, genotyping budget, field and database exports, kernels, BrAPI	the list exists and the practical questions begin
12	Scoring, comparing, evaluating, plotting, exporting, reporting, the command line	you present or publish the plan
13	The head-to-head benchmark	you want the evidence for the defaults
14	Recipes, FAQ, crosswalk from other packages, troubleshooting	you have a concrete question
15	Function index, glossary, references	you need to look something up

Conventions. Code in grey boxes is run when the document is built; its output follows with #>. Coloured boxes mark key points, theory, advice for breeders, hand verification and warnings. Every exported function starts with `tso_` (there are 65 of them, listed with one line each in Part 15); type `tso_` and press Tab in RStudio to see all of them.

One dataset throughout. Parts 2 to 12 use one simulated panel of 300 doubled-haploid lines in four subpopulations, so that numbers can be compared across parts (Part 9 adds a second panel with an introgressed major gene). Part 13 uses six public panels.

```
panel <- tso_simulate_panel(n = 300, markers = 1000, groups = 4, fst = 0.15, seed = 2026)
trait <- tso_simulate_trait(panel, h2 = 0.4, n_qtl = 100, seed = 5)
y <- setNames(trait$y, trait$id)      # phenotypes, used from Part 8 onwards
g <- setNames(trait$g, trait$id)      # true genetic values (only a simulation has
  ↪ them)
```

1 Introduction

TSOpt tutorial · Part 1

Why training-set optimisation

Genotyping is cheap, phenotyping is not: which candidates deserve a field plot?

The question Of N genotyped candidates, which n should be phenotyped so that the rest are predicted best?	What exists STPGA, TSDFGS, TrainSel: slow searches, one fixed heritability, one criterion at a time.	What TSOpt adds A rank-one engine, an exact intercept, robust CDmean, ROTS, decision-aware goals, any data format.	How to read Theory first, by hand in base R; then the package call that gives the same number.
--	---	---	---

One verb: `tso_design(data, n)` - everything else is an optional argument in breeder language

1.1 The problem

A breeding programme genotypes thousands of candidates for a few dollars each. It can phenotype only a fraction of them: a yield plot costs one to two orders of magnitude more than a genotype, and land, seasons and labour are finite. Genomic prediction (Meuwissen et al. 2001) closes the gap: the phenotyped lines form a *training set*, a model such as GBLUP learns the relationship between markers and performance, and every genotyped but unphenotyped *candidate* receives a predicted value.

The accuracy of those predictions depends on the training set: how large it is, and, for a given size, *which* lines it contains (Rincent et al. 2012; Isidro et al. 2015; Akdemir et al. 2015; Riedelsheimer et al. 2013; Isidro y Sánchez and Akdemir 2021). Two training sets of 150 lines drawn from the same 1,000 candidates can differ by 5 to 15% in prediction accuracy on the remaining 850, and the difference grows as the budget shrinks. Choosing well costs nothing but computation. That choice is *training-set optimisation* (TSO), and it is the one question TSOpt answers:

Key point

Of my N genotyped candidates, which n (or which set within a budget) should be phenotyped so that the genetic values of the rest, or of a chosen target group, are predicted as well as possible?

The same question appears in many forms. Which crosses to make and test in a hybrid programme. Which line goes to which site in a sparse multi-environment trial. Which accessions of a genebank deserve a reference phenotyping panel. Which lines get an expensive quality assay when the lab can run only 300. TSOpt treats all of these as the same problem, because they are.

1.2 What existed before

Three R packages have been the workhorses of the field. STPGA (Akdemir et al. 2015) optimises a large family of criteria (CDmean, PEVmean, D-, A-, E-optimality and more) with a genetic algorithm on a principal-component representation of the markers. TSDFGS (Ou and Liao 2019) introduced the *r-score*, the expected correlation between predicted and observed values, and an

exchange search. TrainSel (Fernández-González et al. 2023) is a general-purpose optimiser (genetic algorithm plus simulated annealing) that accepts any user-written criterion, including multi-environment and mixed discrete-continuous problems. The comparison by Fernández-González et al. (2023) across seven public panels remains the reference benchmark.

Each of these solved part of the problem. Each also shares three limitations that TSOpt was built to remove.

1. **Every evaluation inverts a matrix.** The criteria are functions of a matrix inverse that depends on the training set, so a search that evaluates 100,000 candidate sets performs 100,000 inversions. STPGA and TSDFGS work in a reduced space of 20 to 30 principal components to make this affordable, at the price of accuracy; TrainSel works with the full matrix and is slow. TSOpt never inverts: each candidate is scored by a rank-one update (Part 2), so a design for 1,000 lines takes about a second and for 2,000 lines about fifteen seconds.
2. **One heritability, fixed in advance.** All criteria depend on the ratio of residual to genetic variance, and the tools ask for it once. Nobody knows it before the trial. The benchmark in Part 13 shows that optimising hard at one assumed value can *reduce* realised accuracy. TSOpt's default hedges across three values (Part 3).
3. **Accuracy is not the breeder's objective.** The breeder selects the top 5 or 10% and discards the rest. Expected genetic gain, the identification of the best, and the next season's decision are what matter. TSOpt has criteria for the selection decision and a two-stage design that places the next plots where the decision is still uncertain (Part 8).

1.3 What TSOpt adds

- **One engine for every system.** A line, a hybrid, a line in an environment, a trait measured on a line: all are *units*. Adding a unit to the design is one rank-one update of the prediction error variance, done in place by compiled code. New breeding systems are constructors, not new algorithms (Part 7).
- **Exact fixed effects at engine speed.** CDmean in its original form projects the intercept out of every contrast, which breaks the rank-one structure. A flat-prior kernel restores it: the exact projected criterion at the speed of the unprojected one. Since v0.4 the same argument covers *any* fixed effects (trials, checks, covariates, environment means), exactly (Part 2).
- **Robust CDmean as the one default.** The criterion is averaged over heritabilities 0.2, 0.5 and 0.8. It had the best realised accuracy in the benchmark and runs in a fraction of a second (Parts 3 and 13).
- **A catalogue of 18 criteria in five tiers**, so specialists find what they need and everybody else never has to choose (Part 3).
- **Major genes are not invisible.** A genome-wide kernel cannot see whether the training set holds both alleles of a major gene. `tso_major_scan()`, `tso_haplotypes()` and `tso_data(major = )` find the loci, weight them or fix them, and guarantee carriers of every allele the targets carry; every plan reports allele and haplotype coverage (Part 9).
- **Sparse multi-environment testing done properly.** Genetic covariance between sites, site-specific noise, environment means as exact fixed effects, across-environment targets and sparse allocation (`max_per_line`, `min_per_env`) (Part 7).
- **Multi-trait design, completed (v0.5).** Traits and environments together (a unit is one trait on one line in one environment), an exact mean per trait and environment, a design that hedges over uncertain genetic correlations and trait heritabilities, and `tso_trait_value()`, which says what each trait is worth: how many drone scans replace one yield plot, and how much the index loses without a cheap proxy trait (Part 8).
- **More decision tools.** How many lines (`tso_sample_size()`), will the set find the best (`tso_topk()`), accuracy against diversity or cost (`tso_pareto()`), which parents after the trial (`tso_contribution()`), and panels of tens of thousands of lines without an $N \times N$ matrix (`tso_design_large()`) (Part 10).
- **Decisions around the design (v0.5).** The exact value of every plot, for cutting plots knowingly when seed runs short; a marker bootstrap that says whether the advan-

tage over random is real (`tso_uncertainty()`); the training-set size that pays for itself (`tso_economics()`); a genetically aware field layout with blocks, checks and p-rep plots (`tso_layout()`); whom to genotype from a pedigree (`tso_genotype_set()`); how well one programme predicts another (`tso_transfer()`); designs that hedge over additive, dominance and Gaussian kernels; and a BrAPI v2 connector that reads genotypes and phenotypes straight from the breeding database (Part 11).

- **A new search, ROTS.** The Rank-One Tabu Search ranks every member of the current set by its exact removal loss in one operation, and uses tabu memory to escape local optima. It is the strongest optimiser of the criterion in the package (Part 4).
- **Honest before clever.** `tso_diagnose()` says whether optimisation is worth it on your panel. Every plan is compared with random sets and prints a traffic-light verdict (Parts 6 and 12). User criteria are probed for the minimise-versus-maximise trap before they are optimised.
- **Any data in, publication figures out.** VCF, HapMap, PLINK, DArT, spreadsheets in any coding, compressed or not, with the counted allele kept consistent across files and seasons; ggplot2 figures with journal presets (Parts 5 and 12).
- **By hand first.** Every theoretical claim is shown in a few lines of base R before the package call that reproduces it. This follows the teaching order of the PlantNura course from which the package grew ([Hussain 2026](#)).

1.4 Where it sits

	STPGA	TSDFGS	TrainSel	TSOpt
Criteria	many classical	r-score, CD, PEV	user-defined	18 in tiers, one default
Search	GA	exchange	GA + SA	greedy, exchange, ROTS, GA+SA, multi-start
Evaluation cost	inversion in PC space	inversion in PC space	full inversion	rank-one update
Fixed effects	intercept (PC space)	not projected	projected (full)	any X , exact via flat prior
Heritability	one value (λ)	fixed ridge	one value	robust grid (default)
Hybrids, MET, traits	no	no	via user code	built in, traits $\times$ environments together
Uncertain trait correlations	no	no	no	robust over five scenarios (default)
Kernel (gene action)	one	one	one	A, D, AA, AD, DD, Gaussian, pedigree, single-step H; robust shares; total or breeding value
Major genes	no	no	no	scan, weighting, carrier guarantee
Haplotypes	no	no	no	phased reading, LD / window / PLINK blocks, HAP kernel, coverage criterion and report

	STPGA	TSDFGS	TrainSel	TSOpt
Omics (transcriptome, metabolome, spectra)	no	no	via user kernels	omics kernels with batch correction and spectral pre-treatment; unprofiled lines predicted from genomics; profiling budget plan
Programme planning	no	no	no	forward validation, REML variance model, season across stages, crosses, genotyping budget
Field and database	no	no	no	Field Book, FieldHub, Breedbase, BrAPI write-back, plan manifest, CLI, REST, Docker, Python
Value of each plot, each trait	no	no	no	exact, in every plan; tso_trait_value()
Uncertainty of the advantage	no	no	no	marker bootstrap with a verdict
Economic size	no	no	no	net value by size, optimum
Field layout	no	no	no	blocks, checks, p-rep, relatives apart
Whom to genotype, transfer	no	no	no	pedigree coverage; bridge curve
Multi-objective	NSGA-II	no	NSGA-II	NSGA-II seeded by the engine
Sample size	no	SSDFGS	no	exact operating curve
Parent selection	no	no	continuous weights	optimum contribution, exact QP
Very large panels	PC space	PC space	no	low-rank engine, no $N \times N$
Two-stage design	no	no	no	knowledge gradient
Diagnostics, verdict	no	size curve	no	yes
Data formats	matrix	matrix	matrix	any, and BrAPI v2 servers
Licence	GPL	GPL	proprietary (non- commercial)	proprietary, free for everyone with registration

2 Theory

TSOpt tutorial · Part 2

The theory in one page

From the mixed model to the coefficient of determination, without ever inverting $\mathbf{K}$

Mixed model

$y = 1\mu + \mathbf{Z}\mathbf{u} + \mathbf{e}$, $\mathbf{u} \sim N(\mathbf{0}, \sigma_u^2 \mathbf{K})$. The design decides which rows of $\mathbf{Z}$ exist.

PEV and CD

PEV of a contrast $\mathbf{c}$ is $\mathbf{c}'\mathbf{A}\mathbf{c}$; its reliability is $\text{CD} = 1 - \mathbf{c}'\mathbf{A}\mathbf{c} / \mathbf{c}'\mathbf{K}\mathbf{c}$.

Rank-one update

Adding one plot: $\mathbf{A} \leftarrow \mathbf{A} - \mathbf{a}\mathbf{a}' / (\lambda + \mathbf{a}\mathbf{a})$. One outer product, never a solve().

Exact intercept

$\mathbf{K} + \mathbf{c}\mathbf{c}'$ with a huge $\mathbf{c}$: the flat-prior trick makes the fixed intercept exact inside the fast engine.

$\text{CDmean}(\mathbf{S}) = \text{mean over targets of } 1 - \mathbf{c}_i'\mathbf{A}(\mathbf{S})\mathbf{c}_i / \mathbf{c}_i'\mathbf{K}\mathbf{c}_i$

This part derives everything the package computes, on a panel small enough to print. If you only want to use TSOpt, skip to Part 5; if you want to cite or extend it, read on. Notation: N candidates, a training set S of size n , a relationship matrix $\mathbf{K}$ ($N \times N$), entry-mean heritability h^2 and $\lambda = (1 - h^2)/h^2$.

2.1 The mixed model and the prediction error variance

Genomic prediction fits

$$\mathbf{y} = \mathbf{X}\beta + \mathbf{Z}\mathbf{u} + \mathbf{e}, \quad \mathbf{u} \sim N(\mathbf{0}, \sigma_u^2 \mathbf{K}), \quad \mathbf{e} \sim N(\mathbf{0}, \sigma_e^2 \mathbf{I}),$$

where $\mathbf{u}$ holds the genetic values of *all* N units and $\mathbf{Z}$ ($n \times N$) picks out the phenotyped ones. Henderson's mixed model equations give the BLUP $\hat{\mathbf{u}}$ and its prediction error variance (Henderson 1975). For a fixed intercept only ($\mathbf{X} = \mathbf{1}$),

$$\text{PEV}(\hat{\mathbf{u}}) = \sigma_e^2 \mathbf{C}^{22}, \quad \mathbf{C}^{22} = (\mathbf{Z}'\mathbf{M}\mathbf{Z} + \lambda\mathbf{K}^{-1})^{-1}, \quad \mathbf{M} = \mathbf{I} - \mathbf{1}(\mathbf{1}'\mathbf{1})^{-1}\mathbf{1}'.$$

Without the projection $\mathbf{M}$ (that is, treating the mean as known), $\mathbf{Z}'\mathbf{Z}$ is the diagonal indicator $\mathbf{D}_S$ of the training set and

$$\mathbf{A}(S) = (\mathbf{D}_S + \lambda\mathbf{K}^{-1})^{-1}.$$

$\mathbf{A}(S)$ is the object every criterion is built from: up to σ_e^2 , it is the posterior covariance of the genetic values after phenotyping S . Its diagonal is the prediction error variance of each unit; the off-diagonals say how errors are shared between relatives.

2.2 Contrasts and the coefficient of determination

The breeder rarely cares about a genetic value on its own; they care about a *contrast*: this line minus the mean of the candidates, this family mean minus that one, this hybrid, this index. A contrast is a vector $\mathbf{c}$ of length N . Its prediction error variance is $\mathbf{c}'\mathbf{A}\mathbf{c}$, its prior variance is $\mathbf{c}'\mathbf{K}\mathbf{c}$,

and the *coefficient of determination* (Laloë 1993) is the fraction of the prior variance the data explain,

$$CD(\mathbf{c}) = 1 - \frac{\lambda \mathbf{c}' \mathbf{A}(S) \mathbf{c}}{\mathbf{c}' \mathbf{K} \mathbf{c}} = \frac{\text{Var}(\mathbf{c}' \hat{\mathbf{u}})}{\text{Var}(\mathbf{c}' \mathbf{u})}.$$

CD is the squared correlation between the prediction and the truth, so $\sqrt{CD}$ is the expected accuracy. **CDmean** (Rincen et al. 2012) averages CD over one contrast per target unit, each target minus the mean of the targets:

$$CDmean(S) = \frac{1}{|T|} \sum_{i \in T} CD(\mathbf{c}_i), \quad \mathbf{c}_i = \mathbf{e}_i - \frac{1}{|T|} \mathbf{1}_T.$$

The centring matters: prediction accuracy is about ranking lines against each other, not about their absolute level, and the intercept of the model is absorbed by the mean.

2.2.1 By hand on six lines

The six-line panel of Chapter 14 (Hussain 2026) is small enough to check every number.

```
X <- matrix(c(2,0,2,2,0,0,2,0,
              2,0,2,0,0,2,2,0,
              0,2,0,2,2,0,0,2,
              0,2,2,2,2,0,0,2,
              2,2,0,0,0,2,2,2,
              0,0,2,2,2,2,0,0), nrow = 6, byrow = TRUE,
            dimnames = list(paste0("L", 1:6), paste0("m", 1:8)))
p <- colMeans(X) / 2 # allele frequencies
Z <- sweep(X, 2, 2 * p, "-") # centred genotypes
G <- tcrossprod(Z) / (2 * sum(p * (1 - p))) + diag(1e-4, 6) # VanRaden (2008) + tiny
  ↪ ridge
round(G, 2)
```

```
#>      L1    L2    L3    L4    L5    L6
#> L1  1.77  0.91 -1.14 -0.80 -0.46 -0.29
#> L2  0.91  2.11 -2.00 -1.66  0.74 -0.11
#> L3 -1.14 -2.00  2.11  1.43 -0.29 -0.11
#> L4 -0.80 -1.66  1.43  1.77 -0.97  0.23
#> L5 -0.46  0.74 -0.29 -0.97  2.46 -1.49
#> L6 -0.29 -0.11 -0.11  0.23 -1.49  1.77
```

Phenotype lines 1, 3 and 5 with $h^2 = 0.5$ and compute CDmean from the definition, inverting everything:

```
train <- c(1, 3, 5); h2 <- 0.5; lambda <- (1 - h2) / h2
D <- diag(0, 6); diag(D)[train] <- 1
A <- solve(D + lambda * solve(G)) # the PEV operator A(S)
Cm <- diag(6) - matrix(1/6, 6, 6) # contrasts: each line minus the mean
num <- diag(Cm %*% A %*% t(Cm)) # c_i' A c_i
den <- diag(Cm %*% G %*% t(Cm)) # c_i' K c_i
CD_i <- 1 - lambda * num / den
round(CD_i, 3)
```

```
#> [1] 0.685 0.666 0.711 0.529 0.716 0.453
```

```
CDmean_by_hand <- mean(CD_i); CDmean_by_hand
```

```
#> [1] 0.6266
```

2.3 The rank-one engine

A search evaluates many candidate sets. Inverting a $N \times N$ matrix for each is what makes existing tools slow. The Sherman-Morrison identity (Sherman and Morrison 1950) removes the inversion. Adding unit j to the training set changes the precision by $\mathbf{e}_j \mathbf{e}_j'$, so

$$\mathbf{A}(S \cup j) = \mathbf{A}(S) - \frac{\mathbf{a}_j \mathbf{a}_j'}{\lambda + a_{jj}}, \quad \mathbf{a}_j = \mathbf{A}(S) \mathbf{e}_j \text{ (column } j),$$

when $\mathbf{A}$ is written on the scale $\mathbf{A}(S) = (\lambda^{-1} \mathbf{D}_S + \mathbf{K}^{-1})^{-1}$, with the starting point $\mathbf{A}(\emptyset) = \mathbf{K}$. So $\mathbf{K}$ is never inverted: the search starts from $\mathbf{K}$ itself and subtracts one outer product per unit added.

```
A2 <- G # nothing phenotyped: A = K
for (j in train) {
  a <- A2[, j]
  A2 <- A2 - tcrossprod(a) / (lambda + a[j])
}
num2 <- diag(Cm %*% A2 %*% t(Cm))
CDmean_rank1 <- mean(1 - num2 / den); CDmean_rank1
```

```
#> [1] 0.6266
```

Verify by hand

Two routes, one number: 0.6266373383 by inversion and 0.6266373383 by rank-one updates (the λ sits in different places in the two parametrisations, which is why the second formula has no λ in front of the ratio). The package test suite checks this equality to 10^{-8} on every build.

The same identity scores every candidate at once. For a candidate j , the new PEV of contrast $\mathbf{c}_i$ is $\mathbf{c}_i' \mathbf{A} \mathbf{c}_i - (\mathbf{c}_i' \mathbf{a}_j)^2 / (\lambda + a_{jj})$, so with $\mathbf{W} = \mathbf{C} \mathbf{A}$ (contrasts by candidates) the gain of every candidate is one column-wise sum of $\mathbf{W}^2 / (\lambda + \text{diag } \mathbf{A})$ divided by the fixed denominators. A greedy step over 1,000 candidates is one matrix product. Removal is the same formula with the sign flipped, which is what makes exchange and tabu searches cheap (Part 4).

TSOpt implements this in C++ (Rcpp, BLAS rank-one updates, OpenMP for the candidate scoring). The state $\mathbf{A}$ lives in an environment and is updated in place, so no copies are made. In the package, `tso_score()` computes the criterion directly for any set, and the test suite checks it against the engine.

```
tp6 <- tso_data(kinship = G, verbose = FALSE) # TSOpt rescales K to mean diagonal 1
lambda_scaled <- lambda / mean(diag(G)) # same lambda on TSOpt's scale
tso_score(tp6, train = c("L1", "L3", "L5"), h2 = 1 / (1 + lambda_scaled),
  criteria = c("cdmean", "cdmean_exact"))
```

```
#>      cdmean cdmean_exact
#>      0.6266      0.5529
```

The `cdmean` value equals the hand calculation. The `cdmean_exact` value is slightly different: it is the criterion *with* the intercept projected out, which is the subject of the next section.

2.4 The fixed intercept, made exact

Rincent's `CDmean` projects the intercept out: $\mathbf{Z}' \mathbf{M} \mathbf{Z}$ replaces $\mathbf{Z}' \mathbf{Z}$. This is the correct criterion when the model has a fixed mean (it always does), but $\mathbf{M}$ couples all training units and the precision update is no longer rank one. STPGA and TrainSel keep the projection and pay for it with an inversion per evaluation; TSDFGS and the fast engine of Chapter 14 drop it (the ranking of sets is nearly the same, $r \approx 0.96$, but the values differ).

TSOpt gets the exact projected criterion *and* the rank-one speed by a change of kernel. Put a flat prior on the intercept: treat it as a random effect with a huge variance c . That is the same as replacing $\mathbf{K}$ by

$$\mathbf{K}_c = \mathbf{K} + c\mathbf{J}, \quad \mathbf{J} = \mathbf{1}\mathbf{1}',$$

and running the ordinary rank-one engine on $\mathbf{K}_c$. As $c \rightarrow \infty$ the posterior of every *contrast that sums to zero* converges to the fixed-effect posterior, because such contrasts are orthogonal to $\mathbf{1}$ and the flat prior on the mean is integrated out. Centred CDmean contrasts sum to zero by construction. TSOpt uses $c = 10^3 \times \text{diag } \mathbf{K}$.

```
Kc <- G + 1e3 * mean(diag(G)) # K + cJ (adding a scalar adds it
  ↳ everywhere)
A3 <- Kc
for (j in train) { a <- A3[, j]; A3 <- A3 - tcrossprod(a) / (lambda + a[j]) }
CD_flat <- mean(1 - diag(Cm %*% A3 %*% t(Cm)) / den)
# the exact projected criterion by brute force: Z'MZ replaces D
Zt <- diag(6)[train, ]; Mproj <- diag(3) - matrix(1/3, 3, 3)
A4 <- solve(t(Zt) %*% Mproj %*% Zt + lambda * solve(G))
CD_projected <- mean(1 - lambda * diag(Cm %*% A4 %*% t(Cm)) / den)
c(flat_prior = CD_flat, projected = CD_projected, unprojected = CDmean_by_hand)
```

```
#> flat_prior    projected unprojected
#>      0.5530         0.5529         0.6266
```

Key point

The flat-prior kernel reproduces the projected CDmean to four decimals on this tiny panel (the agreement improves with c , and with panel size, because the approximation error is of order $1/c$) while keeping every update rank one. This is what `fixed = "intercept"` (the default) does inside `tso_design()`. The benchmark in Part 13 shows it is worth 0.013 in realised accuracy over the unprojected criterion. `fixed = "none"` gives the unprojected version for comparison with tools that drop the intercept.

2.5 General fixed effects, made exact

A real trial has more fixed effects than a mean: the trial or block a line is grown in, a check versus entry indicator, a covariate such as days to heading, the environment means of a multi-environment trial, a known major gene (Bernardo 2014). With $\mathbf{X}$ holding all of them (the intercept included), the generalised-least-squares prediction error of a genetic contrast after phenotyping S is

$$\text{PEV}(\mathbf{c}) = \mathbf{c}'\mathbf{K}\mathbf{c} - \mathbf{c}'\mathbf{K}_S\mathbf{P}\mathbf{K}_S\mathbf{c}, \quad \mathbf{P} = \mathbf{V}^{-1} - \mathbf{V}^{-1}\mathbf{X}_S(\mathbf{X}_S'\mathbf{V}^{-1}\mathbf{X}_S)^{-1}\mathbf{X}_S'\mathbf{V}^{-1}, \quad \mathbf{V} = \mathbf{K}_{SS} + \lambda\mathbf{I}.$$

$\mathbf{P}$ depends on the whole training set, so once more the update is not rank one. The flat-prior argument of the previous section extends unchanged. Give β a prior $N(\mathbf{0}, c\mathbf{I})$: the observed signal of unit j is $u_j + \mathbf{x}_j'\beta$, with covariance

$$\mathbf{K}_X = \mathbf{K} + c(\mathbf{1}\mathbf{1}' + \mathbf{X}\mathbf{X}').$$

The engine runs its rank-one updates on $\mathbf{K}_X$, but the covariances between the *contrasts* and the units are taken from $\mathbf{K}$, not $\mathbf{K}_X$: what is predicted is $\mathbf{c}'\mathbf{u}$, the genetic contrast, not $\mathbf{c}'(\mathbf{u} + \mathbf{X}\beta)$. As $c \rightarrow \infty$ the posterior variance of $\mathbf{c}'\mathbf{u}$ converges to the GLS PEV above for every contrast, centred or not. TSOpt scales each column of $\mathbf{X}$ to unit mean square, so one constant $c = 10^3 \times \text{diag } \mathbf{K}$ suits all of them.

Verify by hand

On the 300-line panel, 45 random training lines, three trials (a factor) and a continuous covariate: the engine on $\mathbf{K}_X$, the package's direct GLS routine, and the projection $\mathbf{P}$ written out by hand.

```
tp_fx <- tso_data(panel, verbose = FALSE)
K <- tp_fx$K; N <- nrow(K); set.seed(1)
trial <- factor(rep(c("A", "B", "C"), length.out = N))
Xf <- cbind(covariate = rnorm(N), model.matrix(~ trial)[-1]) # 3 columns besides the
  ↪ mean
Xf <- sweep(Xf, 2, sqrt(colMeans(Xf^2)), "/") # unit mean square
S <- sample(N, 45); s2 <- rep(1, N) # h2 = 0.5 on TSOpt's scale
ctr <- TSOpt:::.contrasts(K, setdiff(seq_len(N), S), "centered") # C K, c'Kc from K
cc <- 1e3 * mean(diag(K))
st <- TSOpt:::.engine_init(K + cc + cc * tcrossprod(Xf), ctr) # the engine runs on K_X
for (j in S) TSOpt:::.engine_add(st, j, s2[j])
cd_engine <- mean(1 - st$q / st$den)
cd_gls <- TSOpt:::.cd_direct_X(K, S, s2, ctr, cbind(1, Xf)) # direct GLS projection
V <- K[S, S] + diag(1, 45); Vi <- solve(V); XS <- cbind(1, Xf)[S, ] # and by hand
P <- Vi - Vi %*% XS %*% solve(t(XS) %*% Vi %*% XS, t(XS) %*% Vi)
cd_hand <- mean(rowSums((ctr$CK[, S] %*% P) * ctr$CK[, S]) / ctr$den)
cd_int <- TSOpt:::.cd_direct(K, S, s2, ctr, project = TRUE) # intercept only
print(c(engine = cd_engine, gls = cd_gls, by_hand = cd_hand, intercept_only = cd_int),
      digits = 10)

#>          engine          gls          by_hand intercept_only
#> 0.1924695815 0.1924687986 0.1924687986 0.2000916103
```

The engine agrees with the exact projection to 8×10^{-7} (relative error 4×10^{-6} , the same order as the intercept trick, and it shrinks as $1/c$), while the three extra fixed effects cost 0.008 of CDmean compared with a model that has only a mean: every fixed effect estimated from the training set uses up information. In `tso_design()`, `fixed = c("trial", "moisture")` names columns of `data$units`, or `fixed = X` gives a numeric matrix with one row per unit; environment means and fixed major loci are added automatically, and `plan$spec$fixed` reports what was projected. Part 6 shows it in a design.

Multi-trait designs. Since v0.5 the same kernel serves multi-trait designs, where every trait has its own mean (grain yield in t/ha and protein in % share no intercept) and, with environments, every trait has its own mean at every site. $\mathbf{X}$ then holds one indicator per trait $\times$ environment cell (less one, which the intercept absorbs), and the engine runs on $\mathbf{K} + c(\mathbf{11}' + \mathbf{XX}')$ with $\mathbf{K} = \Omega_E \otimes \mathbf{G}_T \otimes \mathbf{K}_{\text{lines}}$ exactly as above. Up to v0.4 multi-trait designs had only a common intercept.

The first-unit trap. A fixed mean costs a unit: the first measurement of a trait (or in a trial, block or site) only estimates that level's mean and says nothing about genetics. A greedy search, which looks one unit ahead, therefore sees zero gain from opening a new level and never opens it, although the *second* unit there may be the most valuable in the design. TSOpt removes the trap by opening every level of every factor fixed effect (trials, sites, traits, trait $\times$ site cells) with two units before the search allocates the rest freely (when the budget allows it), and reports it in `plan$spec`.

```
p60 <- tso_simulate_panel(n = 60, markers = 500, seed = 60)
d_mt <- tso_data(p60, traits = c("yield", "ndvi"), trait_h2 = c(0.3, 0.6),
               gcor = matrix(c(1, 0.6, 0.6, 1), 2), trait_cost = c(20, 1),
               envs = c("E1", "E2"), env_cor = 0.7, verbose = FALSE)
p_mt <- tso_design(d_mt, n = 40, index = c(yield = 1, ndvi = 0),
                  trait_uncertainty = "none", n_random = 0, verbose = FALSE)
table(p_mt$selected$trait, p_mt$selected$env) # every trait x site cell opened
```

```
#>
#>      E1 E2
#> ndvi  2  2
#> yield 19 17

u_mt <- d_mt$units
X_mt <- TSOpt:::.fixed_matrix(NULL, d_mt, u_mt, u_mt$id, TRUE) # 3 trait x site columns
K_mt <- d_mt$K; c_mt <- 1e3 * mean(diag(K_mt))
ctr_mt <- TSOpt:::.contrasts_index(K_mt, 60, p_mt$spec$index_blocks, 1:60, TRUE)
s2_mt <- (1 - u_mt$h2) / u_mt$h2
st_mt <- TSOpt:::.engine_init(K_mt + c_mt + c_mt * tcrossprod(X_mt), ctr_mt)
for (j in p_mt$index$selected) TSOpt:::.engine_add(st_mt, j, s2_mt[j])
cd_mt <- c(engine = TSOpt:::.crit_value("cdmean", st_mt),
          gls = TSOpt:::.cd_direct_X(K_mt, p_mt$index$selected, s2_mt, ctr_mt, cbind(1,
          ↪ X_mt)))
print(cd_mt, digits = 10)

#>      engine      gls
#> 0.2580048747 0.2580004395
```

Verify by hand

The engine on the flat-prior kernel and the exact GLS projection with a mean per trait and site agree to 4×10^{-6} , and the smallest of the four trait $\times$ site cells holds 2 units (the opening rule guarantees two).

2.6 Heritability and λ

$\lambda = (1 - h^2)/h^2$ is the only parameter of the model the design depends on. Small λ (high heritability) makes each plot informative and the design spreads out to cover the panel; large λ (low heritability) makes single plots noisy and the design concentrates on the structure. TSOpt always rescales $\mathbf{K}$ to a mean diagonal of one, so that h^2 is the *entry-mean* heritability of the trait on the phenotyped unit (a plot mean over reps, a family mean), whatever the ploidy or inbreeding of the panel.

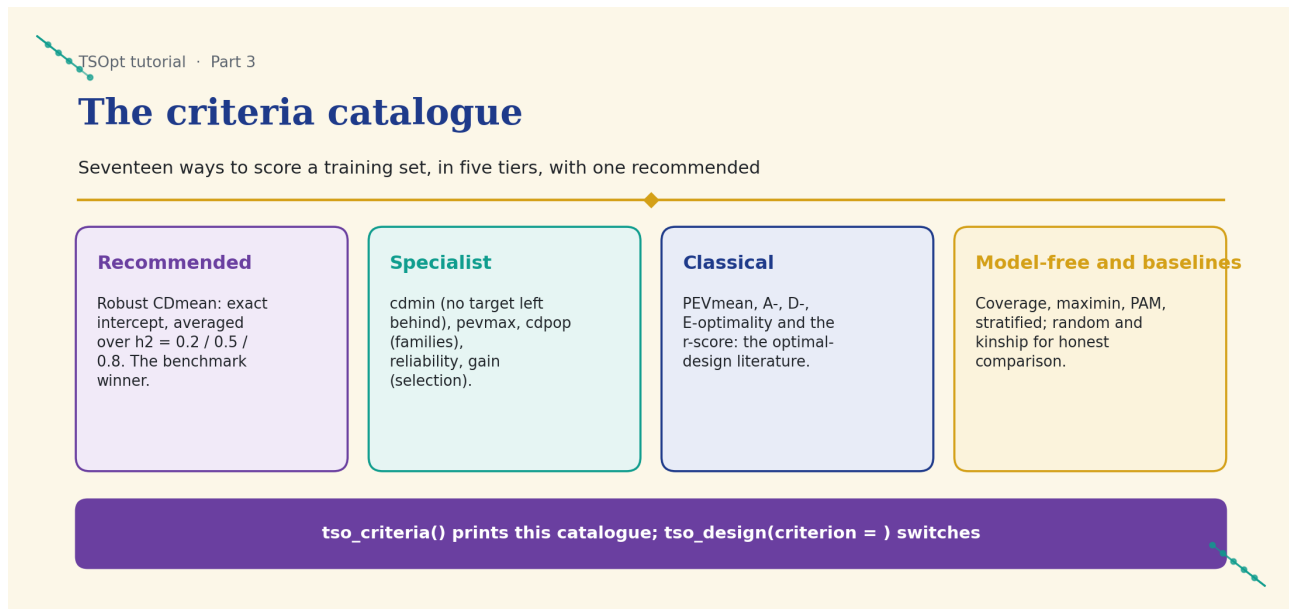
The right h^2 is never known in advance. Rather than bet on one, the default design averages the criterion over $h^2 \in \{0.2, 0.5, 0.8\}$, running three engines in lock-step and adding the candidate that improves the average most. Part 3 defines this *robust CDmean*; Part 13 shows why it is the default.

2.7 Which contrasts: targeted and untargeted designs

Untargeted: the target set is the whole pool, and the design serves a reusable reference panel. *Targeted*: the target is a specific set of unphenotyped lines (this season's candidates, next year's crosses), and the training set is chosen to predict *them*. Targeting buys accuracy on the target and costs accuracy on everything else (Hussain 2026); target = makes the choice explicit and the plan reports coverage of the target.

The contrast type also changes with the goal (Part 3): centred contrasts for ranking, identity contrasts for reliabilities of individual values, family-mean contrasts for CDpop, index contrasts $\mathbf{b} \otimes \mathbf{e}_i$ for multi-trait designs, hybrid contrasts for crosses. The engine is the same; the matrix $\mathbf{C}$ changes.

3 The criteria catalogue



A *criterion* is a number that scores a candidate training set. `tso_criteria()` lists every criterion TSOpt knows, in five tiers, with when to use each. The tiers exist so that the choice is simple: use the default unless you recognise your situation in the Specialist tier.

```
pn_kable(tso_criteria()[, c("criterion", "tier", "use_when")],
         col.names = c("criterion", "tier", "use when"))
```

criterion	tier	use when
cdmean	Recommended	almost always (the default)
pevmean	Specialist	very large pools, ranking identical to CDmean
cdpop	Specialist	targets are crosses or families, not lines
reliability	Specialist	breeding values themselves, not deviations, are what you report
cdmin	Specialist	no target may be left unpredictable; genebanks, pre-breeding
pevmax	Specialist	as cdmin, on the variance scale
gain	Specialist	strong truncation selection (top 1-10%)
dopt	Classical	no target set known; reusable reference panels
aopt	Classical	targets known, variance scale
eopt	Classical	robustness to the worst direction; small sets
rscore	Classical	direct expected correlation
coverage	Model-free	model-free coverage; very large or poorly modelled panels
haplotype	Model-free	genebank cores and reference panels; rare haplotypes must be learnable
maximin	Model-free	model-free diversity
pam	Model-free	model-free representativeness
stratified	Model-free	baseline for structured panels
random	Baseline	baseline
kinship	Baseline	never, except to show why not (harmful)

Throughout this part, the worked examples design $n = 45$ lines from the 300-line panel without stratification, so that the criteria can be compared on equal terms. `tso_data()` builds the object

every function takes (Part 5 explains it):

```
tp <- tso_data(panel, verbose = FALSE)
```

3.1 Tier 1, Recommended: robust CDmean

Definition. CDmean as defined in Part 2, with the intercept treated exactly through the flat-prior kernel, averaged over three heritabilities:

$$\text{CDmean}_{\text{robust}}(S) = \frac{1}{3} \sum_{h^2 \in \{0.2, 0.5, 0.8\}} \text{CDmean}(S; \lambda = (1 - h^2)/h^2).$$

Three engines run in lock-step; at each greedy step the unit that improves the *average* most is added.

Why it is the default. Every criterion in this catalogue depends on λ , and λ is unknown before the trial. In the head-to-head benchmark (Part 13), sets optimised hard at $h^2 = 0.5$ had the best CDmean at $h^2 = 0.5$ and worse realised accuracy than sets that hedged. Averaging over the grid was the single most accurate method across six panels and 33 scenarios, and it costs three times a greedy pass, which is still a fraction of a second.

Use it for the ordinary problem: one trait (or an unspecified one), a set of lines to predict, no unusual constraints. It is what `tso_design(data, n)` does.

```
plan_robust <- tso_design(tp, n = 45, stratify = "none", verbose = FALSE)
plan_robust$spec$search
```

```
#> [1] "greedy (robust CDmean: average over h2)"
```

```
plan_robust$value$cd_by_h2 # CDmean of the chosen set at each h2
```

```
#> h2=0.2 h2=0.5 h2=0.8
#> 0.1470 0.2971 0.4209
```

Give one h2 when you know it. $h^2 = 0.35$ designs for that value alone; the benchmark suggests this only helps when the estimate is good. Give your own grid, $h^2 = c(0.1, 0.3, 0.5)$, for a low-heritability trait.

3.2 Tier 2, Specialist

3.2.1 pevmean: mean prediction error variance

$\text{PEVmean}(S) = \frac{1}{|T|} \sum_{i \in T} \mathbf{c}_i' \mathbf{A}(S) \mathbf{c}_i$, minimised. It is CDmean without the denominator $\mathbf{c}_i' \mathbf{K} \mathbf{c}_i$, so the two agree on which unit to add next whenever the denominators are similar, and the benchmark found no difference in practice. It is slightly cheaper and is the natural criterion when the variance scale, not the correlation scale, is reported (Akdemir et al. 2015).

3.2.2 cdpop: families and groups as the target

Replaces the line contrasts by *group-mean* contrasts: each family or group mean minus the grand mean (Rincent et al. 2017). Use it when the decision is between crosses or families rather than between lines, for example when choosing which crosses to advance from a set of biparental families. In `tso_data()`, `families =` or `groups =` define the contrasts; here the four subpopulations are used.

```
plan_cdpop <- tso_design(tp, n = 45, criterion = "cdpop", stratify = "none", verbose =
  ↪ FALSE)
table(plan_cdpop$selected$group)
```

```
#>
#> G1 G2 G3 G4
#> 12 11 11 11
```

3.2.3 reliability: individual values, not deviations

Identity contrasts $\mathbf{c}_i = \mathbf{e}_i$: the mean CD of the genetic values themselves. Use it when the absolute value is what you report (a variety's breeding value for a registration dossier) rather than a ranking within the candidates. Because the intercept is not removed, the fixed-effect treatment does not apply.

3.2.4 cadmin: no target left behind

CDmean is an average, and an average can hide a sub-population that is predicted badly. cadmin maximises a *soft minimum* of the target CDs,

$$\text{CDmin}_\tau(S) = -\tau \log \sum_{i \in T} \exp(-\text{CD}_i/\tau), \quad \tau = 0.02,$$

which is within $\tau \log |T|$ of the true minimum and is smooth, so the rank-one gain of every candidate is still available. Use it for genebanks and pre-breeding, where every accession must remain predictable, and whenever the coverage line of `print(plan)` shows unreachable targets. It trades mean accuracy for the worst case:

```
plan_cadmin <- tso_design(tp, n = 45, criterion = "cadmin", stratify = "none", verbose =
  FALSE)
rel <- function(p) p$units$reliability[!p$units$id %in% p$selected$id]
c(robust_mean = mean(rel(plan_robust)), robust_min = min(rel(plan_robust)),
  cadmin_mean = mean(rel(plan_cadmin)), cadmin_min = min(rel(plan_cadmin)))
```

```
#> robust_mean robust_min cadmin_mean cadmin_min
#>      0.2522      0.1255      0.2422      0.1791
```

3.2.5 pevmax: G-optimality

The variance-scale twin of cadmin: minimise the largest prediction error variance among the targets (a soft maximum). Classical G-optimality (Atkinson et al. 2007).

3.2.6 gain: designing for the selection decision

The breeder does not use the predictions to admire them; they select the top fraction α (select_top, default 0.1). The expected mean true value of the selected fraction, under the posterior after phenotyping S , is the quantity that the breeders' equation $\Delta G = i r \sigma_A$ approximates when accuracy is the same for every candidate. TSOpt computes it candidate by candidate: each target i has posterior precision CD_i and the expected gain is

$$\text{Gain}_\alpha(S) = \frac{1}{\alpha|T|} \sum_{i \in T} \sqrt{\text{CD}_i v_i} \phi\left(\frac{t}{\sqrt{\text{CD}_i v_i}}\right),$$

where v_i is the prior variance and the truncation point t is found by Newton iterations so that the expected number selected is $\alpha|T|$. It rewards accuracy where it changes the decision: on candidates near the truncation point. It is marked experimental until its validation study is published; the closed form is verified against Monte Carlo in the test suite.

```
plan_gain <- tso_design(tp, n = 45, goal = "gain", select_top = 0.05, stratify = "none",
                        verbose = FALSE)
plan_gain$spec$criteria

#> [1] "gain"
```

3.3 Tier 3, Classical optimal design

These come from the theory of optimal experimental design ([Fedorov 1972](#); [Atkinson et al. 2007](#)) and appear in STPGA under the same names. They are here for comparison and for the rare problem that needs them.

- **dopt**: $\log \det(\mathbf{I} + \mathbf{K}_{SS}/\lambda)$, the information in the training set itself. It needs no target, so it is the criterion of choice when nothing is known about what will be predicted; it tends to pick the most diverse set and, as the benchmark shows, is clearly worse than CDmean when a target exists.
- **aopt**: the trace of $\mathbf{A}_{TT}$, the mean posterior variance of the targets' own values. Identical to pevmean on identity contrasts.
- **eo**pt: the largest eigenvalue of $\mathbf{A}_{TT}$, the worst-predicted *direction* in the target space. It has no rank-one form, so it is optimised by the genetic algorithm (Part 4).
- **rscore**: the expected correlation between GEBV and phenotype of the targets ([Ou and Liao 2019](#)), TSDFGS's criterion. It is a function of the whole training set through a ratio of quadratic forms and is also optimised by the GA.

3.4 Tier 4, Model-free

These need no heritability and no mixed model, only the relationship matrix or the principal components, and they are fast on very large panels.

- **coverage**: facility location. Every target's *closest training relative* should be as close as possible: $\sum_{i \in T} \max_{j \in S} K_{ij}$. This is a submodular function, so the greedy algorithm has a $(1 - 1/e)$ guarantee ([Nemhauser et al. 1978](#)). It did well in the benchmark (third overall) and is a good model-free default for poorly modelled panels.
- **maximin**: farthest-point sampling in PC space, a space-filling design. Diversity, not prediction.
- **pam**: one medoid per cluster (partitioning around medoids, Kaufman and Rousseeuw ([1990](#))). Representativeness.
- **stratified**: random within groups, proportional to group size. The honest baseline for structured panels ([Isidro et al. 2015](#)).

3.5 Tier 5, Baselines

- **random**: what every plan is compared with. The `n_random` random sets of `tso_design()` give the null distribution of the criterion.
- **kinship**: the units most related, on average, to the targets. It sounds sensible and is *harmful* (it picks a tight clique that predicts its own cluster and nothing else); it is in the catalogue so that the comparison can show why not ([Hussain 2026](#)).

3.6 All criteria on one panel

The design criterion is a prediction; realised accuracy is the test. To judge it fairly on one panel, twenty traits are simulated on the same genotypes ($h^2 = 0.4$, 100 QTL each), every design is fitted by GBLUP on each trait, and the accuracy (correlation between predicted and true genetic value of the 255 unphenotyped lines) is averaged over the twenty traits.

```

traits20 <- lapply(1:20, function(s) tso_simulate_trait(panel, h2 = 0.4, n_qlt = 100,
  ↪ seed = 100 + s))
accuracy20 <- function(p) mean(sapply(traits20, function(tr)
  tso_evaluate(p, setNames(tr$y, tr$id), truth = setNames(tr$g,
  ↪ tr$id))$predictive_ability))
res <- list()
for (cr in tso_criteria())$criterion) {
  t0 <- proc.time()[["elapsed"]]
  p <- tso_design(tp, n = 45, criterion = cr, stratify = "none", n_random = 0, verbose =
  ↪ FALSE)
  secs <- proc.time()[["elapsed"]] - t0
  res[[cr]] <- data.frame(criterion = cr, search = sub("\\(.*", "", p$spec$search),
    seconds = round(secs, 2), CDmean = round(p$value$cdmean_exact, 3),
    accuracy = round(accuracy20(p), 3))
}
res <- do.call(rbind, res); res <- res[order(-res$accuracy), ]
pn_kable(res, row.names = FALSE)

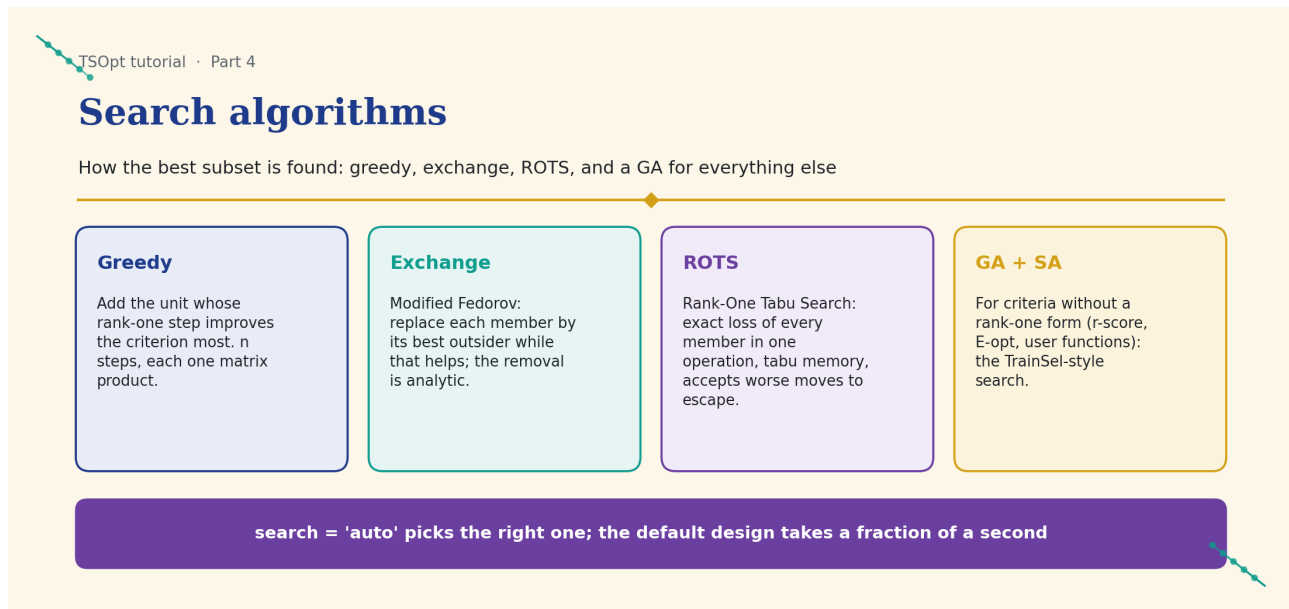
```

criterion	search	seconds	CDmean	accuracy
aopt	greedy	0.05	0.296	0.464
cdmin	tabu	0.38	0.286	0.463
pevmean	greedy	0.04	0.296	0.456
reliability	greedy	0.04	0.296	0.451
coverage	model-free	0.08	0.292	0.451
eopt	ga	8.64	0.296	0.448
rscore	ga	42.83	0.296	0.448
cdmean	greedy	0.04	0.296	0.446
gain	tabu	2.56	0.296	0.443
cdpop	greedy	0.02	0.281	0.432
pevmax	tabu	0.35	0.281	0.432
dopt	tabu	0.24	0.275	0.427
pam	model-free	0.06	0.275	0.426
haplotype	model-free	0.48	0.278	0.402
kinship	model-free	0.05	0.251	0.394
random	model-free	0.05	0.251	0.388
stratified	model-free	0.05	0.253	0.385
maximin	model-free	0.05	0.278	0.384

Watch out

Twenty traits on one simulated panel still leave a standard error of about 0.01 on the accuracy column, so neighbours in the table are tied. Do not rank criteria from this table alone; the 33 scenarios of Part 13 on real data are the evidence. What the table shows reliably is the pattern: the CD family and coverage at the top (about 0.45), the worst-case, group and D-optimal criteria and PAM in the middle (0.43), and random, kinship, stratified and maximin at the bottom (0.39), six accuracy points below the best. Note that eopt and rscore start their genetic algorithm from the greedy CDmean set and, on this panel, did not move away from it.

4 Search algorithms



A criterion scores a set; a *search* finds a good set among the $\binom{N}{n}$ possible ones, which is astronomically many ($\binom{300}{45} \approx 10^{53}$). No search is exhaustive. TSOpt offers four, and `search = "auto"` picks the right one for the criterion.

4.1 Greedy (forward selection)

Start from the empty set (or from `must_include` and `history`). At each step, score every candidate by the improvement it would bring, add the best, update the state, repeat until n units or the budget are reached. With the rank-one engine one step costs one matrix product, so the whole search is $O(n \cdot N \cdot |T|)$.

By hand, on the six-line panel, choosing two lines:

```
A <- G; chosen <- integer(0)
for (step in 1:2) {
  cand <- setdiff(1:6, chosen)
  gain <- sapply(cand, function(j) {                # CDmean after adding j, from one
    ↪ rank-one step
    Aj <- A - tcrossprod(A[, j]) / (lambda + A[j, j])
    mean(1 - diag(Cm %*% Aj %*% t(Cm)) / den)
  })
  j <- cand[which.max(gain)]; chosen <- c(chosen, j)
  A <- A - tcrossprod(A[, j]) / (lambda + A[j, j])
  cat("step", step, ": add L", j, " CDmean =", round(max(gain), 4), "\n")
}
```

```
#> step 1 : add L 2    CDmean = 0.335
#> step 2 : add L 5    CDmean = 0.5284
```

For budgets with unequal costs, the score is *gain per unit cost* (a knapsack greedy), and for `reps > 1` the same unit can be added again. Greedy has a guarantee only for submodular criteria (coverage); for CDmean it is a heuristic, but a strong one: in the benchmark the greedy set was never more than 1% below the best set any search found.

4.2 Greedy + exchange (modified Fedorov)

After the greedy pass, try to improve the set by swaps: for each member, find the best outsider to replace it with and accept the swap if the criterion improves (Fedorov 1972; Cook and Nachtsheim 1980). The effect of removing a member is computed analytically (the rank-one update with the sign flipped), so no state is touched unless a swap is accepted. Passes repeat until no swap helps or `max_pass` (default 3) is reached. This was TSOpt's default in v0.2.

4.3 ROTS: the Rank-One Tabu Search

Exchange stops at the first local optimum. Tabu search (Glover 1989) continues past it: it always makes the best available move, even a worsening one, and forbids reversing recent moves for a while, so that the search walks out of the basin instead of oscillating. ROTS is tabu search built on the rank-one engine, and it is TSOpt's contribution to the search side:

1. **Rank every member by its exact removal loss in one operation.** With $\mathbf{W} = \mathbf{CA}$ and $d_j = \lambda - a_{jj}$ (the removal denominator), the loss of removing member j is the column sum of $\mathbf{W}_{:j}^2/d_j$ over the denominators, so all n losses come from one matrix product.
2. **Try the `n_try = 6` most removable members.** For each, remove it analytically, then find the best outsider to add (again one product).
3. **Accept the best move of the six, even if it is worse than the current set**, unless it is *tabu*: the removed unit may not re-enter, and the added unit may not leave, for a number of iterations. A tabu move is still allowed if it would give the best set ever seen (*aspiration*).
4. **Stop after 30 iterations without improvement or 150 in total; return the best set seen.**

```
searches <- c("greedy", "greedy+exchange", "tabu", "ga")
cmp <- do.call(rbind, lapply(searches, function(s) {
  t0 <- proc.time()[["elapsed"]]
  p <- tso_design(tp, n = 45, h2 = 0.5, search = s, stratify = "none", n_random = 0,
    verbose = FALSE)
  data.frame(search = s, seconds = round(proc.time()[["elapsed"]] - t0, 2),
    CDmean = round(p$value$cdmean_exact, 5),
    swaps = p$spec$n_swaps, tabu_moves = p$spec$tabu_moves,
    tabu_improvements = p$spec$tabu_improved)
}))
pn_kable(cmp, row.names = FALSE)
```

search	seconds	CDmean	swaps	tabu_moves	tabu_improvements
greedy	0.02	0.2960	0	0	0
greedy+exchange	0.04	0.2968	6	0	0
tabu	0.20	0.2974	6	122	9
ga	1.12	0.2960	0	0	0

```
plan_tabu <- tso_design(tp, n = 45, h2 = 0.5, search = "tabu", stratify = "none", verbose
  = FALSE)
tso_plot(plan_tabu, "search")
```

Search path

6 improving swaps in 3 exchange passes

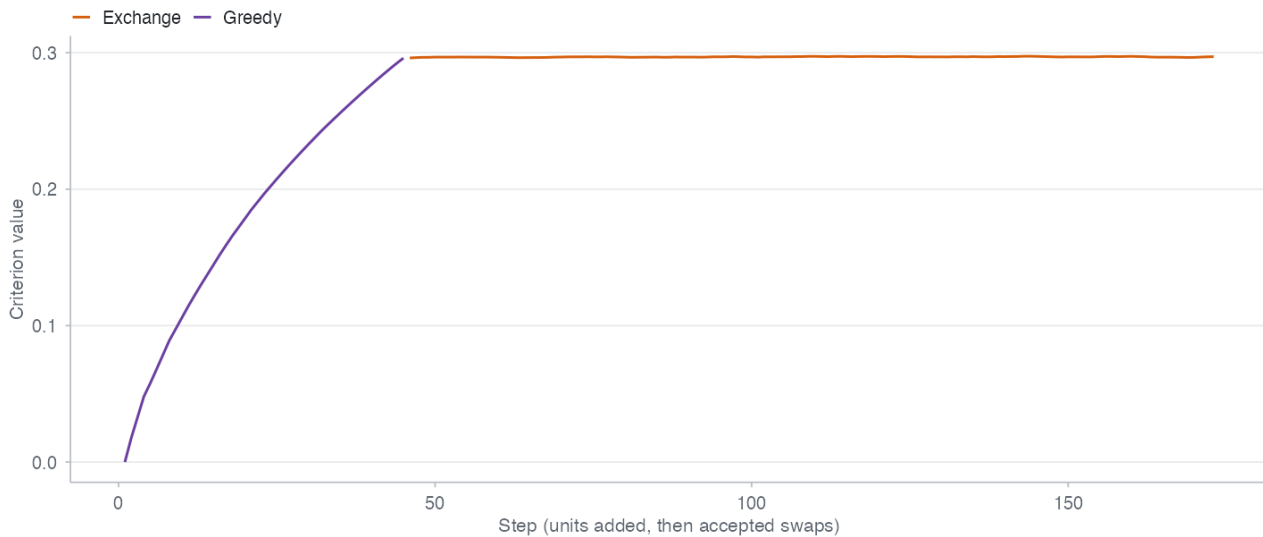


Figure 1: The criterion along the search: greedy steps, then the improvements found by ROTS.

Key point

ROTS is the best optimiser of the criterion in the package: it never ends below greedy + exchange (a unit test checks this) and usually above. It is *not* the default, because the benchmark showed that a higher CDmean at one assumed h^2 does not translate into higher realised accuracy; the extra optimisation over-fits the assumed model. Use `search = "tabu"` when the criterion value itself is the deliverable (a methods comparison, a known h^2), and the robust default otherwise.

Multi-start. Any local search depends on where it starts. `restarts = r` runs the exchange or tabu search r more times from perturbed copies of the best set found so far and keeps the best by the criterion; `cores =` runs the restarts in parallel (forked processes; one core on Windows). `plan$spec$restarts` reports how many runs were made and whether any improved on the first (Part 10 shows an example).

4.4 GA + SA: for everything else

Criteria without a rank-one form (`eopt`, `rscore`, a user-supplied function) are optimised by a genetic algorithm with a simulated-annealing acceptance step, in the style of TrainSel (Fernández-González et al. 2023; Holland 1975; Kirkpatrick et al. 1983): a population of sets (default 60) evolves by crossover and mutation for up to `iterations` generations, the best set is refined by annealed swaps, and the population is seeded with the greedy CDmean set so that the search starts from a good place. It is one to three orders of magnitude slower than the rank-one searches and is chosen automatically only when needed.

A custom criterion is any R function of the selected unit ids and the `tso_data` object that returns one number, maximised by default; `direction = "minimize"` says that smaller is better. Before it optimises, TSOpt probes the function once and warns when it looks like it points the wrong way (Part 10):

```
mean_distance <- function(ids, data) {                                # as unrelated as possible
  KS <- data$K[ids, ids]
  -mean(KS[upper.tri(KS)])
```

```

}
plan_custom <- tso_design(tp, n = 45, criterion = mean_distance, stratify = "none",
                        iterations = 200, n_random = 0, verbose = FALSE)
c(search = plan_custom$spec$search, generations = plan_custom$spec$ga_generations)

#>      search generations
#>      "ga"          "20"

```

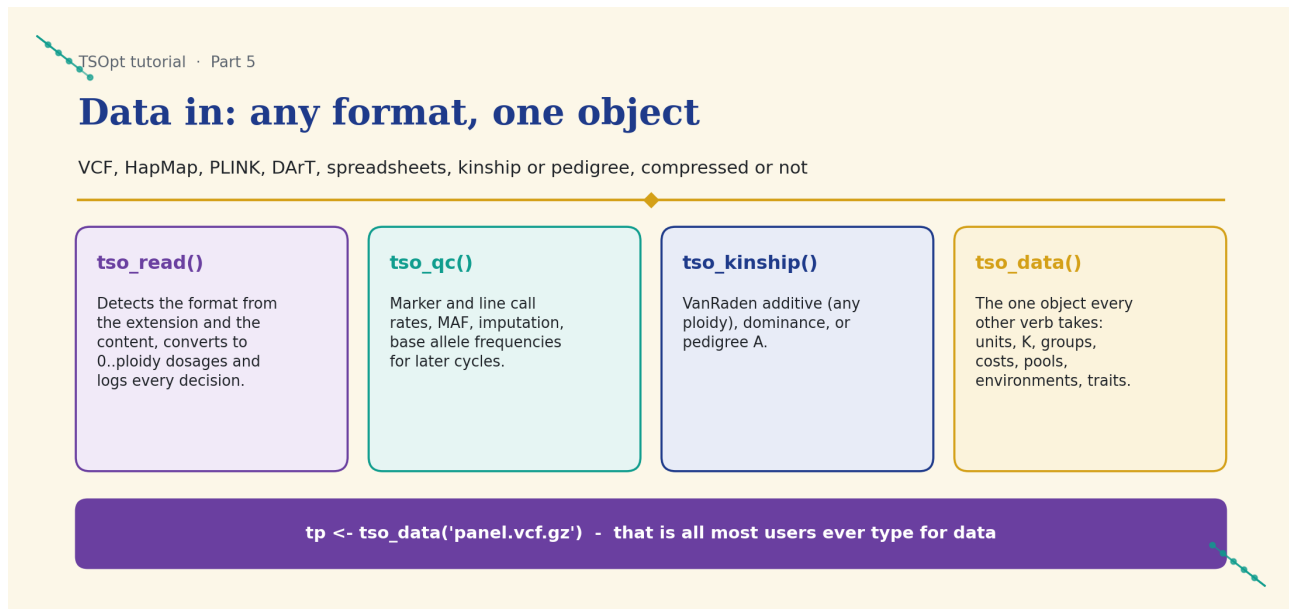
4.5 Which search runs when

criterion	search = "auto" chooses	why
robust CDmean (the default)	greedy over the h^2 grid	exchange and tabu would over-fit one λ
cdmean, pevmean, cdpop, reliability, aopt, dopt with one h2	tabu (ROTS)	rank-one form, full optimisation wanted
cdmin, pevmax, gain	tabu (ROTS)	rank-one gains exist for the soft minimum and the gain integral
eopt, rscore, custom function	GA + SA	no rank-one form
any of the rank-one criteria with restarts > 0	the chosen search, restarted	escapes the first local optimum
coverage, maximin, pam, stratified, random, kinship	their own model-free procedure	no engine needed

For the breeder

You do not need to choose. `tso_design(data, n)` uses robust CDmean with the greedy search, which the benchmark found to be the most accurate combination and which takes a fraction of a second on a panel of a thousand lines. The other searches are there for methods work.

5 Data in: any format, one object



Nobody should have to convert a VCF in TASSEL before designing a trial. `tso_read()` reads whatever the genotyping provider delivered, `tso_qc()` cleans it, `tso_kinship()` turns it into a relationship matrix and `tso_data()` assembles the object every other verb takes. In practice one line does all four: `tso_data("panel.vcf.gz")`. This part goes through each step so that you know what happened to your data, because the package tells you but does not decide for you silently.

The package ships example files in every supported format (300 lines, 1,000 markers, the same panel as this tutorial):

```
ex <- system.file("extdata", package = "TSOpt")
list.files(ex)
```

```
#> [1] "example_costs.csv"          "example_dart.csv"          "example_groups.csv"
#> [4] "example_marker_effects.csv" "example_panel.csv.gz"
#> [7] "example_panel.hmp.txt.gz"   "example_pheno.csv"         "example_plink.bed"
#> [10] "example_plink.bim"          "example_plink.fam"         "example_true_values.csv"
```

5.1 tso_licence(): activating TSOpt

TSOpt is free of charge for everyone, but each user registers with PlantNura for a personal licence key. `tso_licence_request()` writes the request email. `tso_licence("TS01....")` activates the key you receive, once: the key is checked offline and saved on your computer. The key is a record (licence ID, holder, dates) signed with PlantNura's Ed25519 key and verified in compiled code, and the engine does not run without it. The licence ID is recorded in every plan, plan file, manifest and report. `tso_docs()` opens the guides, this tutorial and the manual that come with the package.

```
tso_licence()
```

```
#> Licensed to Waseem Hussain, PlantNura Consulting & Solutions (TS0-2026-0001, until
#> 2028-09-29)
#> Key read from
#> /Users/waseemhussain/Library/Preferences/org.R-project.R/R/TSOpt/licence.key
```

5.2 tso_read(): one reader for every format

Family	Formats	Notes
Variant files	VCF 4.x, .vcf.gz	GT (default) or DS dosage field; polyploid GT such as 0/0/1/1 IUPAC single-letter or two-letter calls
TASSEL / GBS	HapMap .hmp.txt	
PLINK	binary .bed/.bim/.fam, text .ped/.map, .raw	give the .bed file or the prefix
DArT	DArTseq one-row and two-row SNP, SilicoDArT	detected from the header
Tables	CSV, TSV, TXT, Excel, .rds, .RData	any coding: 0/1/2, -1/0/1, 0..ploidy, AA/AB/BB, nucleotide pairs wrapped without copying
R objects	matrix, data frame	transparent
Compression	.gz, .bz2, .xz, .zip	

The format is detected from the extension *and* the content (header lines, magic bytes, column patterns), so a mislabelled file is still read correctly. Every decision is logged:

```
geno <- tso_read(file.path(ex, "example_panel.vcf.gz"))
```

The same panel from HapMap, PLINK and CSV gives the same relationship matrix (the counted allele may differ between formats, the relationships do not):

```
g_hmp <- tso_read(file.path(ex, "example_panel.hmp.txt.gz"), verbose = FALSE)
g_plink <- tso_read(file.path(ex, "example_plink.bed"), verbose = FALSE)
g_csv <- tso_read(file.path(ex, "example_panel.csv.gz"), verbose = FALSE)
K_of <- function(g) tso_kinship(tso_qc(g, verbose = FALSE))
c(hapmap = isTRUE(all.equal(K_of(geno), K_of(g_hmp))), plink =
  ↪ isTRUE(all.equal(K_of(geno), K_of(g_plink))),
  csv = isTRUE(all.equal(K_of(geno), K_of(g_csv))))
```

```
#> hapmap plink csv
#> TRUE TRUE TRUE
```

What the object holds. dosage (lines by markers, count of the counted allele from 0 to ploidy), map (marker, chromosome, position, alleles), ploidy, format, source, log. Arguments: format forces a format; ploidy overrides detection; markers_in = "rows" for tables with markers as rows; id_col names the ID column of a table; vcf_field = "DS" reads imputed dosages; binary_as says whether 0/1 presence-absence data (SilicoDArT) are treated as homozygous calls or as dosages.

Watch out

The counted allele differs between formats (ALT in VCF, A1 in PLINK, minor allele in HapMap). That is harmless for relationship matrices, which are invariant to which allele is counted, and tso_read() reports which one it used so that exported files stay consistent.

5.3 tso_qc(): quality control with a log

```
geno <- tso_qc(geno, maf = 0.05, call_rate = 0.8, line_call_rate = 0.5, impute = "mean")
dim(geno$dosage)
```

```
#> [1] 300 958
```

Markers below the minor-allele frequency or call-rate thresholds are dropped, lines with too many missing calls are dropped, the rest is mean-imputed (adequate for relationship matrices; export to Beagle for anything else), heterozygous calls in inbred lines are counted and reported, and the **base allele frequencies** are stored.

Key point

Base allele frequencies matter across cycles. VanRaden's relationship matrix centres genotypes at the allele frequencies of the panel. If next year's panel is centred at *its own* frequencies, parent-offspring relationships between the years collapse towards zero and the historical training data become useless [ch14]. `tso_qc()` stores the frequencies in the object, `tso_data(base_freq = )` accepts them, and `tso_update()` carries them forward automatically. Since v0.4 the stored frequencies also carry each marker's *counted allele*: a later file in another format (HapMap, allele-pair tables, a PLINK file with a different A1) is re-oriented to it before the relationships are computed, so a flipped allele can no longer silently turn a parent into an unrelated line.

5.4 `tso_kinship()`: relationship matrices

```
K_add <- tso_kinship(geno, method = "additive") # VanRaden (2008), any ploidy
K_dom <- tso_kinship(geno, method = "dominance") # Vitezica et al. (2013),
  ↪ diploids
round(K_add[1:4, 1:4], 3)
```

```
#>      L001 L002 L003 L004
#> L001 0.989 0.039 0.106 0.078
#> L002 0.039 0.978 0.070 0.300
#> L003 0.106 0.070 0.992 0.077
#> L004 0.078 0.300 0.077 0.936
```

"additive" is VanRaden's method 1 generalised to ploidy k : $\mathbf{G} = \mathbf{Z}\mathbf{Z}' / (k \sum p(1 - p))$ with $\mathbf{Z} = \mathbf{X} - k\mathbf{p}$ (VanRaden 2008). "dominance" is the genotypic dominance matrix of Vitezica et al. (2013). "pedigree" builds the numerator relationship matrix $\mathbf{A}$ from a three-column pedigree by the tabular method. `scale = TRUE` (default) rescales to a mean diagonal of one so that h^2 is the entry-mean heritability; `ridge` adds a tiny diagonal for numerical stability. You can also pass any matrix you already have to `tso_data(kinship = )`: an ASReml GIV, an AGHmatrix H, a Gaussian kernel.

`weights =` gives each marker a weight in the additive kernel, $\mathbf{G}_w = \mathbf{Z}\mathbf{W}\mathbf{Z}' / (k \sum_m w_m p_m (1 - p_m))$: the weighted VanRaden matrix used to up-weight markers with evidence of large effects, for example $-\log_{10} p$ from a genome scan (Zhang et al. 2014). Part 9 uses it.

5.5 `tso_kernels()`: dominance, epistasis, pedigree and single-step

The additive kernel models genetic values as additive. `tso_kernels()` builds the other covariance models, each scaled to mean diagonal one, and `tso_data(kernels = )` combines them:

$$\mathbf{K} = \sum_k w_k \mathbf{K}_k, \quad \sum_k w_k = 1,$$

so $\mathbf{K}$ is the covariance of total genetic values in units of the genetic variance and $w_A \mathbf{K}_A$ (the additive kernels) that of breeding values.

Kernel	Model	Source
A, D	additive, dominance (VanRaden 2008; Vitezica et al. 2013)	markers
AA, AD, DD	pairwise epistasis, $\mathbf{A} \circ \mathbf{A}$, $\mathbf{A} \circ \mathbf{D}$, $\mathbf{D} \circ \mathbf{D}$ (Henderson 1985; Jiang and Reif 2015)	markers
gaussian, exponential	$\exp(-d^2/h)$, $\exp(-d/h)$ on standardised markers, h the median distance (Gianola and van Kaam 2008)	markers
A_ped, D_ped, AA_ped	pedigree additive, dominance $d_{ij} = (a_{s_i s_j} a_{d_i d_j} + a_{s_i d_j} a_{d_i s_j})/4$, epistasis	pedigree
H	single-step: pedigree for the ungenotyped, markers for the genotyped (Legarra et al. 2009)	both

```
g_out <- tso_qc(tso_simulate_panel(n = 120, markers = 600, type = "outbred", seed = 5),
  verbose = FALSE)
kk <- tso_kernels(g_out, which = c("A", "D", "AA", "gaussian"))
kk
```

```
#> TSOpt kernels · 4 · each scaled to mean diagonal 1
#> A      additive      120 lines additive genomic relationship (VanRaden 2008)
#> D      dominance     120 lines dominance genomic relationship (Vitezica et
  ↪ al. 2013; diploids)
#> AA     epistatic      120 lines additive x additive epistasis, A # A
  ↪ (Henderson 1985; Jiang & Reif 2015)
#> gaussian non-parametric 120 lines Gaussian kernel exp(-d^2 / h), RKHS
  ↪ (Gianola & van Kaam 2008)
#> Next: tso_data(geno, kernels = kernels, kernel_share = "robust")
```

```
off <- upper.tri(kk$K$A)
round(sapply(kk$K, function(M) mean(abs(M[off]))), 3) # epistatic relationships fall
  ↪ off faster
```

```
#>      A      D      AA gaussian
#> 0.157 0.055 0.036 0.388
```

The element-wise products roughly square the relationships, so epistatic values are shared mainly by close relatives. For H, G is first put on the scale of the pedigree block $\mathbf{A}_{22}$ (same mean diagonal and off-diagonal) and blended with it (5%, Aguilar et al. (2010)); candidates that are only in the pedigree, such as next season's crosses, get relationships conditioned on the genotyped ones, which is what a design for ungenotyped material needs.

Shares of non-additive variance are rarely known and hard to estimate, so `kernel_share` defaults to "robust": the design averages over non-additive shares of 10%, 30% and 50% (split equally over the non-additive kernels), in the same way as over the heritability grid. Inbred lines carry no dominance (TSOpt warns when D is asked for on inbred material); use A and AA. Hybrids already model dominance and between-pool epistasis through SCA.

```
tp_aa <- tso_data(g_out, kernels = c("A", "AA"), verbose = FALSE)
tp_aa$kernels$share
```

```
#> A AA
#> 0.7 0.3

tp_fix <- tso_data(g_out, kernels = kk, kernel_share = c(A = 0.6, D = 0.2, AA = 0.1,
  ↪ gaussian = 0.1),
  verbose = FALSE)
```

5.6 tso_omics_kernel(): transcriptome, metabolome and spectra

A profile measured on a line predicts part of its performance through a kernel of profile similarities, next to the genomic one:

$$\mathbf{y} = \mathbf{1}\mu + \mathbf{g} + \mathbf{o} + \mathbf{e}, \quad \mathbf{g} \sim N(\mathbf{0}, \sigma_g^2 \mathbf{A}), \quad \mathbf{o} \sim N(\mathbf{0}, \sigma_o^2 \mathbf{O}).$$

Metabolites predicted hybrid maize (Riedelsheimer et al. 2012), several omics layers together more so (Schrag et al. 2018), and near-infrared spectra of grain or wood predict wheat and poplar about as well as markers at a fraction of the cost, *phenomic selection* (Rincen et al. 2018). `tso_omics_kernel()` builds $\mathbf{O}$ from a table of profiles:

1. features or samples with too many missing values are dropped;
2. counts are log-transformed, and spectra get a standard normal variate and a Savitzky-Golay first derivative;
3. every feature is adjusted for batches and covariates by least squares;
4. replicate samples of a line are averaged;
5. the features are standardised, and $\mathbf{O} = \mathbf{Z}\mathbf{Z}'/p$ (or a Gaussian kernel) is scaled to mean diagonal 1.

With genotypes it reports how genetic the profiles are: the REML heritability of every feature (one eigen-decomposition for all), and the correlation between the omics and genomic relationships.

Profiles are often measured on some lines only. The omics relationships of the rest are predicted from their genomic relationships, as single-step predicts genomic relationships from the pedigree (Legarra et al. 2009), with $\mathbf{G}$ in the role of the pedigree:

$$\mathbf{O}_{up} = \mathbf{G}_{up} \mathbf{G}_{pp}^{-1} \mathbf{O}_{pp}, \quad \mathbf{O}_{uu} = \mathbf{G}_{uu} + \mathbf{G}_{up} \mathbf{G}_{pp}^{-1} (\mathbf{O}_{pp} - \mathbf{G}_{pp}) \mathbf{G}_{pp}^{-1} \mathbf{G}_{pu},$$

for profiled (p) and unprofiled (u) lines. The profiled block $\mathbf{O}_{pp}$ is first put on the scale of $\mathbf{G}_{pp}$ and blended with it (5%).

This is the conditional covariance of the omics effects given those of the profiled lines, if genomics were all that relates the lines, so it is only as good as the feature heritabilities. The tests check that it returns $\mathbf{G}$ exactly when the profiles are $\mathbf{G}$.

```
om <- tso_simulate_omics(g_out, features = 120, seed = 2)
ok <- tso_omics_kernel(om$profiles[1:80, ], geno = g_out)
ok
```

```
#> TSOpt omics kernel · OMICS (other, linear) · 80 lines x 120 features
#> Type other (detected)
#> Kernel linear on 120 standardised features · 80 lines
#> Genetics on 80 genotyped lines: median feature heritability 0.45 · agreement of
  ↪ omics and genomic relationships r = 0.63
#> Feature heritability: quartiles 0.17 / 0.45 / 0.61
#> The profiles are largely genetic: genomics can stand in for lines without a profile.
#> Next: tso_data(geno, kernels = list("A", OMICS = om)) (lines without a profile are
  ↪ filled from genomics)
```

```
tp_om <- tso_data(g_out, kernels = ok, verbose = FALSE)
tp_om$kernels$omics
```

```
#> kernel profiled filled h2_median
#> 1 OMICS      80      40      0.445
```

A kernel list can mix everything: `kernels = list("A", "AA", NIRS = ok)`. The omics share follows the kernel rules above (robust 10/30/50% by default, fixed with `kernel_share`, or estimated by `tso_variance(kernels = ok)`). For omics, `value = "total"` predicts genetic plus omics value, the performance the profile forecasts, and `value = "breeding"` predicts the additive part only.

In our benchmark (`benchmark/OMICS.md`, simulated profiles), omics raised prediction accuracy by 0.03 to 0.05 when half the signal ran through them and every line was profiled. It did not change which lines were worth phenotyping.

5.7 `tso_variance()`: the variance model from your trials

Every design assumes a variance model: heritabilities, genetic correlations between traits or environments, kernel shares. `tso_variance()` estimates it from historical trials by REML with the average-information algorithm. All three models are $V = \sum_k \theta_k M_k$ (genetic components $\Omega \otimes K$ per pair of traits or environments, residual components per trait, environment or trait pair), so one AI-REML step serves them all: the score $s_k = -\frac{1}{2}\text{tr}(PM_k) + \frac{1}{2}y'PM_kPy$ and the average information $J_{kl} = \frac{1}{2}y'PM_kPM_lPy$, a Newton step with step-halving, projection of covariance matrices onto the positive-definite cone, and an active set for variances that reach zero. The tests check it against a brute-force optimiser of the same likelihood.

```
v1 <- tso_variance(tp, data.frame(id = names(y), y = y))
v1
```

```
#> TSOpt variance components (AI-REML) · single · 300 records on 300 lines · logL -270.49
↪ · converged in 5 iterations
#> heritability: y 0.35 (SE 0.09)
#> 5 scenarios drawn from the sampling distribution. Next: tso_data(geno, variance = v)
```

`tso_data(variance = v)` uses the estimates; robust designs then average over scenarios drawn from their sampling distribution (for the heritability, its 10%, 50% and 90% quantiles replace 0.2, 0.5 and 0.8).

5.8 `tso_data()`: the object every verb takes

```
pheno <- read.csv(file.path(ex, "example_pheno.csv"))
groups <- read.csv(file.path(ex, "example_groups.csv"))
costs <- read.csv(file.path(ex, "example_costs.csv"))
tp_ex <- tso_data(geno, groups = setNames(groups$group, groups$id),
                 cost = setNames(costs$cost, costs$id), verbose = TRUE)
```

`tso_data()` accepts a `tso_geno` object, a file path, a dosage matrix, a kinship matrix or a pedigree, and adds everything the design needs to know about the units:

argument	meaning	used by
system	"inbred" (default), "clonal", "outcross", "genebank", "hybrid"	kernel, defaults, Part 7
groups, families	named factor: subpopulations, families	stratification, cdpop, quotas
cost	named numeric: cost of phenotyping each unit	budget =
ploidy, dominance, dom_share	polyploid dosage; dominance kernel and its share	clonal and hybrid systems

argument	meaning	used by
base_freq pools, sca_share, crossable	allele frequencies of the base population hybrid parents and which may be crossed	later cycles system = "hybrid"
envs, env_cor, env_noise	environments, their genetic correlation or covariance, their residual variances	sparse multi-environment designs Part 9
major, major_share, major_model, min_carriers	major loci or haplotypes to protect, their variance share, random or fixed, carriers required	
traits, trait_h2, gcor, trait_cost, plot_cost qc	traits, their heritabilities, correlations and costs run <code>tso_qc()</code> on the way in (default TRUE)	multi-trait index designs

The object has units (one row per unit with id, group, family, environment, cost), K (between units), groups, pca, the original geno, and a log of every step. Look at it with `str(tp, max.level = 1)`.

```
head(tp_ex$units, 3)
```

```
#>      id line group family  env line_id cost crossable
#> 1 L001 L001    G1  <NA> <NA>   L001 2.62      TRUE
#> 2 L002 L002    G1  <NA> <NA>   L002 2.73      TRUE
#> 3 L003 L003    G1  <NA> <NA>   L003 2.97      TRUE
```

```
tp_ex$log
```

```
#> [1] "Read example_panel.vcf.gz (VCF) -> 300 lines x 1,000 markers"
#> [2] "Coding VCF GT -> ALT-allele count 0..2 · ploidy 2 (detected)"
#> [3] "Missing 0.0%"
#> [4] "QC dropped 42 markers (MAF < 0.05: 42; call rate < 0.8: 0)"
#> [5] "Hets 0.0% heterozygous calls"
#> [6] "Base freq estimated from these 300 lines and stored (reused for later cycles)"
#> [7] "QC already done by tso_qc(); not repeated"
#> [8] "Kinship additive (VanRaden, base frequencies), mean diagonal scaled to 1"
```

Two figures describe the panel before any design is made:

```
patchwork::wrap_plots(tso_plot(tp, "pca"), tso_plot(tp, "kinship"), ncol = 2)
```

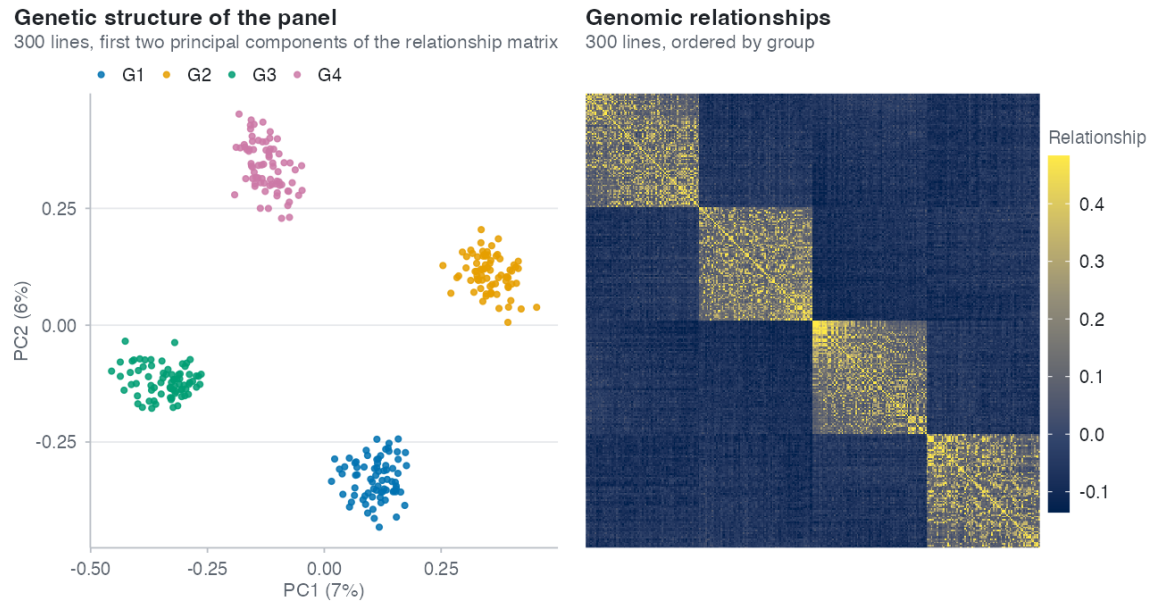


Figure 2: The panel in genetic space (left, PC1 x PC2 with group hulls) and the relationship matrix ordered by group (right).

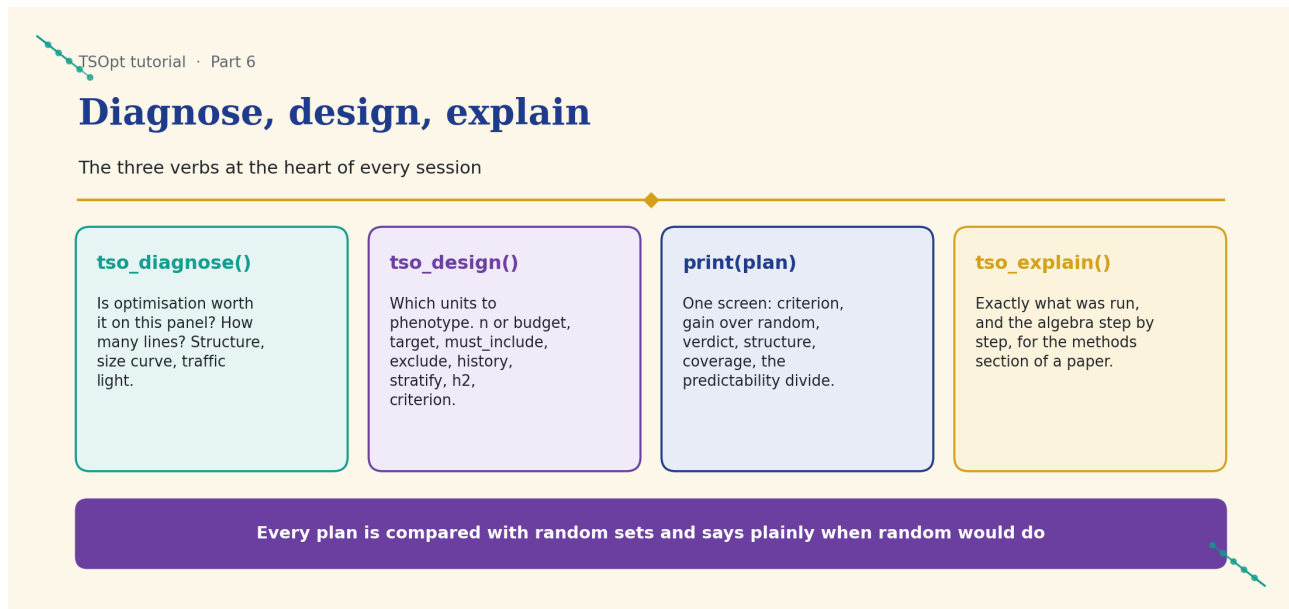
5.9 tso_write(): converting between formats

```
tso_write(geno, "panel.bed")           # PLINK binary
tso_write(geno, "panel.vcf.gz")        # VCF, compressed
tso_write(geno, "panel.hmp.txt")       # HapMap
tso_write(geno, "panel.csv")           # dosages as a table
```

For the breeder

The whole part in one line: `tp <- tso_data("panel.vcf.gz", groups = my_groups)`.
Read the log it prints once, check that the number of lines and markers is what you expected and that the IDs matched your phenotype file, and move on.

6 Diagnose, design, explain



6.1 `tso_diagnose()`: is optimisation worth it here?

Before designing, ask whether designing can help. On some panels every training set of a given size performs about the same (a homogeneous panel with no structure), and the honest advice is to sample at random and spend the effort elsewhere. `tso_diagnose()` answers in seconds.

```
diag <- tso_diagnose(tp, n = 60)
diag
```

```
#> TSOpt diagnosis · 300 candidates · planned n = 60 · h2 = 0.5
#> Verdict      AMBER expected accuracy gain over random at n = 60: 4.9%
#> Advice      Optimisation helps a little; tso_design() is still the better choice, but
#> ↪ random sampling would not be a disaster. The panel has discrete groups: tso_design()
#> ↪ will stratify automatically.
#> Structure    discrete groups (4 groups, silhouette 0.65, supplied groups)
#> Diversity    51 effective independent lines (of 300) · fitted Me = 68
#> Size curve   n=15: 0.34 -> 0.38 · n=30: 0.45 -> 0.48 · n=45: 0.51 -> 0.54 · n=60: 0.56
#> ↪ -> 0.59 · n=90: 0.63 -> 0.65 · n=120: 0.68 -> 0.69 · n=180: 0.74 -> 0.75
#> expected accuracy (sqrt CDmean), random -> optimised
#> Next        tso_plot(diagnosis) · tso_design(data, n = 60)
```

What it computes:

- **Optimisability.** The criterion of n_{random} random sets of size n against the greedy set: their spread and the gap between them, turned into an expected accuracy gain ($\sqrt{\text{CDmean}}$ scale) and a traffic light: GREEN above 8%, AMBER between 2 and 8%, RED below 2%.
- **Structure.** PCA of $\mathbf{K}$, the number of groups (from the silhouette of k -means over the leading PCs, or the supplied groups), and the verdict *continuum*, *mild structure* or *discrete groups*. With discrete groups, `tso_design()` stratifies automatically, because in strongly structured panels stratified sampling can beat naive optimisation (Isidro et al. 2015).
- **Diversity.** The effective number of independent lines (from the eigenvalues of $\mathbf{K}$) and the fitted M_e , the number of independent chromosome segments that makes the size-accuracy curve fit (Daetwyler et al. 2008).
- **Size curve.** Expected accuracy against n for random and optimised sets, so that “how many lines?” gets a curve rather than a guess; sizes = sets the grid.

- **Near-duplicates.** Pairs with $K_{ij} > 0.95$ that would waste plots if both were phenotyped.

```
tso_plot(diag)
```

Diagnosis: AMBER · expected gain 4.9% at n = 60

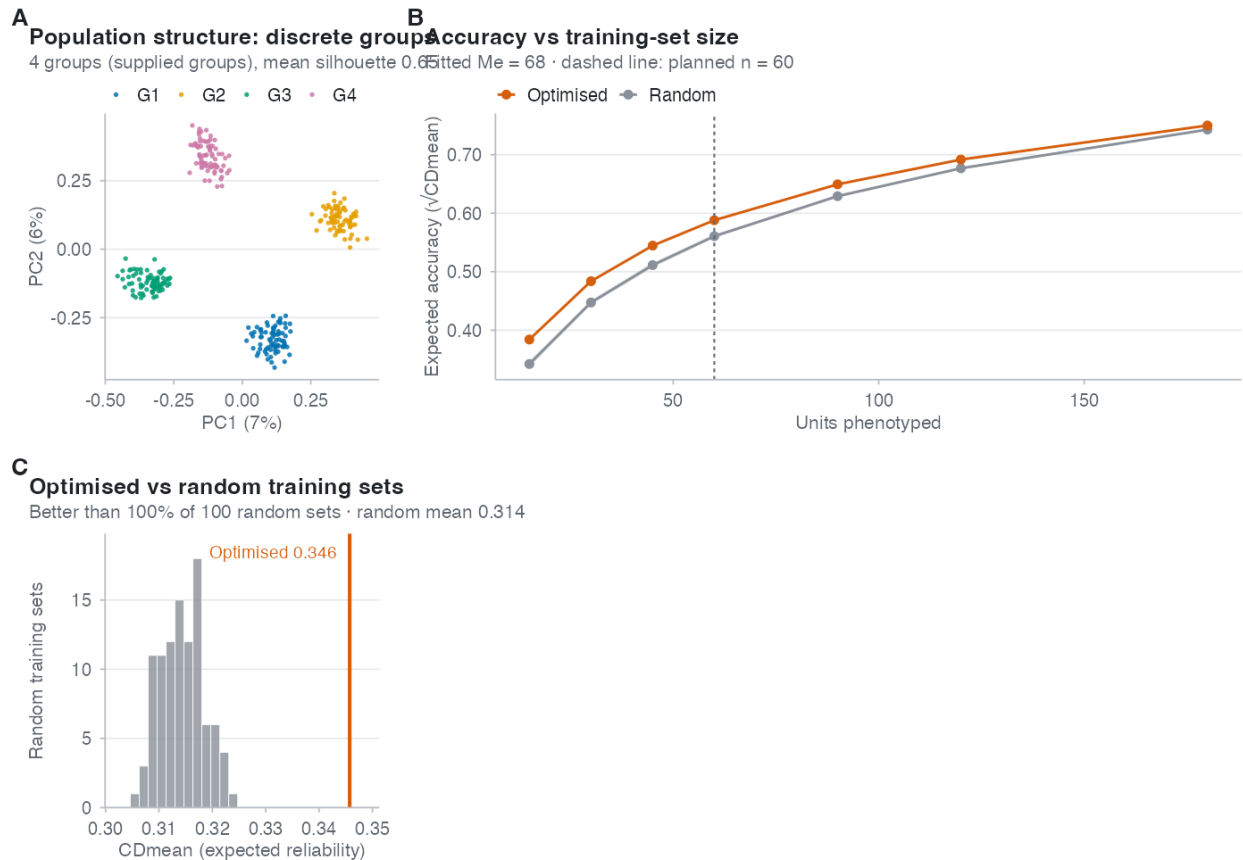


Figure 3: `tso_plot(diagnosis)`: the size curve, the structure and the null distribution at the planned n.

6.2 `tso_design()`: the main verb

```
plan <- tso_design(tp, n = 60)
```

Everything in `print(plan)` comes from the object:

- **Criterion and search** (`plan$spec`): what was optimised, with which h^2 and which search, and the time taken.
- **Expected** (`plan$value`): the criterion of the chosen set, exact (intercept projected), the mean and SD of n_random random sets of the same size, and the percentile of the plan among them.
- **Verdict** (`plan$verdict`): the traffic light with the expected accuracy gain over random.
- **Structure**: the stratification applied, with the allocation per group.
- **Coverage** (`plan$coverage`): the fraction of targets whose closest training relative is at least 0.20 (the threshold below which prediction is little better than the population mean) and the number with none above 0.05 (*unreachable*).
- **Divide** (`plan$divide`): candidates whose reliability under the plan is below 0.06, which no selection decision can distinguish from the mean: the *predictability divide*.

`plan$selected` is the list, in the order the search added the units, with each unit's group, cost, reliability, whether it was forced in, and (since v0.5) its value, the exact CDmean lost if that plot were dropped (Part 11); `plan$units` has the reliability of every candidate; `plan$trace` is the criterion after each step.

```
head(plan$selected[, c("order", "id", "group", "cost", "reliability")])
```

```
#>   order  id group cost reliability
#> 1     1  L234  G4    1     0.5753
#> 2     2  L083  G2    1     0.5864
#> 3     3  L068  G1    1     0.5987
#> 4     4  L185  G3    1     0.5846
#> 5     5  L193  G3    1     0.5694
#> 6     6  L128  G2    1     0.5556
```

6.2.1 Every argument

How many. `n` fixes the number of units. `budget` with `cost` in `tso_data()` fixes the spend instead; units are then added by gain per unit cost (knapsack greedy) until nothing more fits, and `plan$spent` reports the total.

```
plan_budget <- tso_design(tp_ex, budget = 120, verbose = FALSE)
c(units = nrow(plan_budget$selected), spent = round(plan_budget$spent, 1))
```

```
#> units spent
#> 88.0 119.1
```

Who is predicted. `target` names the units to predict; the default is every unit (untargeted). Targets are excluded from the candidates, and the plan reports coverage of the target. `candidates` restricts who may be phenotyped (default: every non-target unit; for hybrids, crosses between crossable parents).

```
this_season <- tp$units$id[241:300] # 60 lines never to be phenotyped
plan_t <- tso_design(tp, n = 60, target = this_season, verbose = FALSE)
unlist(plan_t$coverage[c("frac_020", "frac_005", "n_unreachable")])
```

```
#>   frac_020   frac_005 n_unreachable
#>         1         1             0
```

Who is in or out. `must_include` forces units in (checks, parents already planted) at zero search cost; `exclude` removes units (no seed, quarantined); `history` names units already phenotyped in the past, which enter the design as observed at no cost, with `history_weight` (default 1; use 0.8 to discount old data by the ratio of its residual variance to today's).

```
plan_c <- tso_design(tp, n = 60, must_include = c("L001", "L002"), exclude = paste0("L",
  ↪ 290:300),
  history = paste0("L", 100:130), history_weight = 0.8, verbose = FALSE)
c(forced = sum(plan_c$selected$forced), excluded_present = sum(paste0("L", 290:300) %in%
  ↪ plan_c$selected$id),
  history = plan_c$spec$history)
```

```
#>   forced excluded_present   history
#>         2             0         31
```

Structure. `stratify = "auto"` (default) stratifies when the diagnosis finds discrete groups, "groups" always, "none" never. `group_quota` gives explicit numbers per group; `max_per_family`, `max_per_parent` and `max_per_env` cap the allocation.

```
plan_q <- tso_design(tp, n = 60, group_quota = c(G1 = 30, G2 = 10, G3 = 10, G4 = 10),
  ↪ verbose = FALSE)
table(plan_q$selected$group)
```

```
#>
#> G1 G2 G3 G4
#> 30 10 10 10
```

The model. `h2` (default: the robust grid 0.2/0.5/0.8; one value designs for that value; your own vector is your own prior). `goal` maps to a criterion in breeder language: "accuracy" (CDmean), "families" (CDpop), "information" (D-optimality), "gain" (with `select_top`), "correlation" (r-score), "reference_panel" (untargeted CDmean, stratified). `criterion` overrides the goal with any name from `tso_criteria()` or a function. `value` says what is predicted when the model has non-additive kernels: "total" (default, the worth of the line or clone itself) or "breeding" (the additive part, for choosing parents). The engine needs no change for it: it updates on **K**, the covariance of the data, while the contrasts it scores come from the additive part, which gives the exact reliability of the predicted breeding values. The same device gave exact fixed effects (Part 2). `fixed = "intercept"` (default) treats the intercept exactly; "none" drops it; column names of `data$units` or a numeric matrix add further fixed effects, exactly (Part 2). `trait_uncertainty = "auto"` (default for multi-trait data) hedges over five scenarios of trait heritabilities and genetic correlations, "none" designs for the given values (Part 8); `kernels =` hedges over additive, dominance and Gaussian kernels (Part 11).

Fixed effects have a trap worth knowing. Suppose the lines of an early and a late maturity group are grown in two separate trials, each with its own mean, and the maturity groups follow the population structure:

```
tp_mat <- tp; set.seed(11)
early <- tp$units$group %in% c("G1", "G2")          # maturity follows structure, mostly
flip  <- runif(nrow(tp$units)) < 0.15
tp_mat$units$trial <- ifelse(xor(early, flip), "early", "late")
des <- function(...) tso_design(tp_mat, n = 45, stratify = "none", n_random = 0,
                                verbose = FALSE, ...)

p_ignore <- des()                                   # trial effect ignored
p_trial  <- des(fixed = "trial")                     # trial effect exact (levels opened)
score_trial <- function(ids) des(fixed = "trial", must_include = ids)$value$cdmean_exact
fx_tab <- do.call(rbind, lapply(list(ignore = p_ignore, trial = p_trial), function(p) {
  tr <- tp_mat$units$trial[match(p$selected$id, tp_mat$units$id)]
  data.frame(early = sum(tr == "early"), late = sum(tr == "late"),
              CDmean_trial_model = round(score_trial(p$selected$id), 4))
})))
fx_tab
```

```
#>      early late CDmean_trial_model
#> ignore    22  23          0.2799
#> trial     22  23          0.2879
```

Key point

The first line phenotyped in a trial tells the model nothing about genetics: it is used up estimating that trial's mean. A search that looks one step ahead would therefore never open a second trial and would pile every line into one. TSOpt avoids this trap automatically: every level of a factor fixed effect is opened with two units before the search allocates the rest freely. The trial-aware design above spreads its lines over both trials and reaches CDmean 0.2879 under the trial model, against 0.2799 for the design that ignored the trial effect. Environments in a multi-environment design are opened the same way (`min_per_env`, Part 7).

The search. `search = "auto"` (default), "greedy", "greedy+exchange", "tabu", "ga"; `max_pass` limits exchange passes; `iterations` limits GA generations; `max_contrasts` subsamples the target contrasts for very large targets (default Inf, no subsampling); `restarts` and `cores` run a multi-start search; `direction` says whether a user criterion is maximised or minimised.

Major genes and environments. `min_carriers` overrides, per design, the number of training carriers of each protected allele set in `tso_data()` (Part 9). For line $\times$ environment data, `env_target` says what to predict ("mean", "cells" or one environment), `max_per_line` caps the environments per line and `min_per_env` sets a floor per site (Part 7).

Replicates and reporting. `reps` allows up to that many plots per unit (the same unit may be added again when a second plot is worth more than a new line). `n_random` sets the number of random sets for the comparison (default 100), `seed` makes everything reproducible, `verbose` controls printing.

6.3 `tso_explain()`: what exactly was run

```
tso_explain(plan)
```

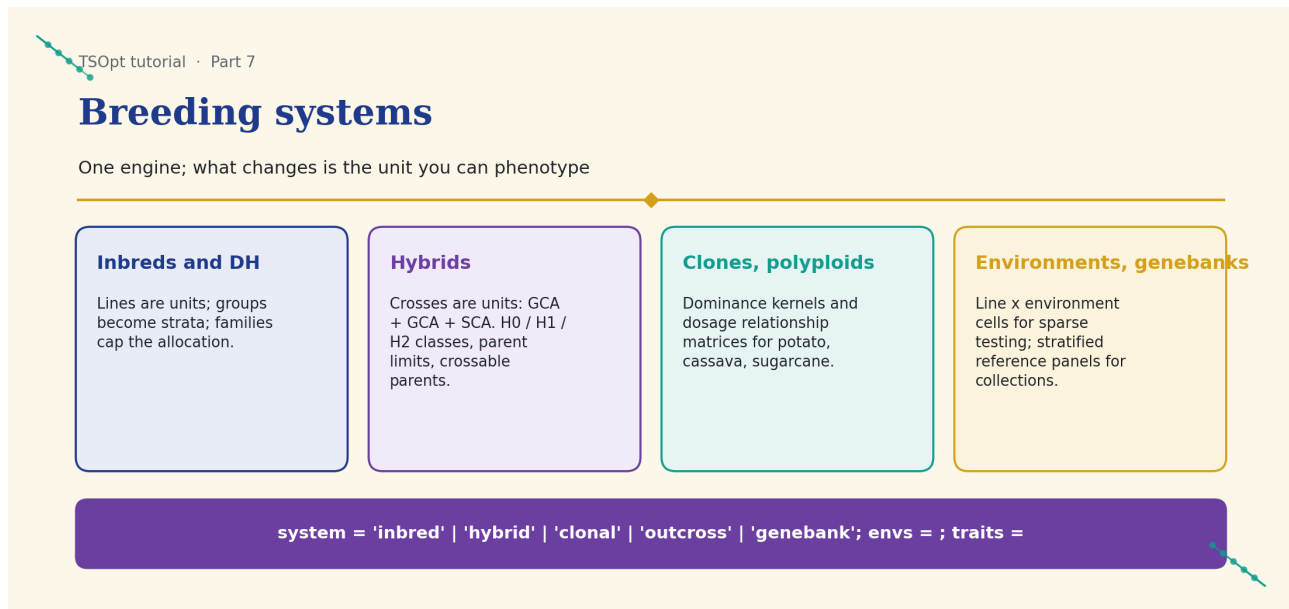
```
#> How this plan was made
#>   System      inbred
#>   Goal        accuracy -> CDmean: the mean coefficient of determination of target
#>   ↪ contrasts (each target minus the target mean),  $CD_i = 1 - c_i' A c_i / c_i' K c_i$ 
#>   Heritability robust: the criterion is averaged over  $h2 = 0.2, 0.5, 0.8$  (nobody knows
#>   ↪  $h2$  before the trial)
#>   Targets      untargeted (whole pool) (300 units)
#>   Candidates    300 units
#>   Size          n = 60
#>   Constraints   group_quota = 15/15/15/15
#>   Structure     discrete groups -> stratified
#>   Search        greedy (robust CDmean: average over  $h2$ )
#>   Fixed effects intercept handled exactly (flat prior)
#>   Reproduce     seed = 1, TSOpt 0.5.0
#>
#> The algebra, step by step
#>   1.  $K$  is the relationship matrix between units, scaled to mean diagonal 1.
#>   2. Before any phenotype, the uncertainty about genetic values is  $A = K$ .
#>   3. Phenotyping unit  $j$  with noise  $\lambda$  shrinks it by a rank-one step:
#>   ↪  $u = A[, j]; A \leftarrow A - u u' / (\lambda + u_j)$  (Sherman-Morrison;  $K$  is never
#>   ↪ inverted)
#>   4. Greedy: at each step add the unit whose rank-one step improves the criterion most.
#>   5. Exchange: try replacing each chosen unit by the best outsider; keep improving
#>   ↪ swaps.
#>   5b. Rank-one tabu search (ROTS): rank every chosen unit by its exact loss in one step,
#>   ↪ remove one of the most removable, add the best outsider, accept the best move even
#>   ↪ if it is worse, and forbid undoing it for a while; keep the best set ever seen.
#>   6. Report: the criterion is recomputed directly, with and without the intercept
#>   ↪ projected out, and compared with random training sets of the same size.
```

The first block is the compiled specification: paste it into a methods section. The second is the algebra in words. `plan$spec` holds the same information as a list, and `plan$call` the call.

For the breeder

A working session is three lines: `tso_diagnose(tp, n)` to see whether it is worth it, `tso_design(tp, n)` to get the list, `tso_export(plan, "trial.csv")` to hand it over. Read the verdict line. If it is RED, random sampling is fine and you have saved yourself an argument; if it is GREEN, the plan is worth 8% or more in accuracy for free.

7 Breeding systems



One engine serves every breeding system. What changes between systems is the *unit* that can be phenotyped (a line, a hybrid, a line in one environment, a trait on a line) and the relationship matrix between units. `tso_data()` builds both from `system =` and a few arguments; the design call is the same.

7.1 Inbred lines and doubled haploids

The default. Lines are units, the additive relationship matrix is the kernel. Groups (from the diagnosis or supplied) become strata; `families =` allows `max_per_family` and the `cdpop` criterion. Early generations (F2:3, F4) are also lines here; give the dosage matrix of the family representatives.

7.2 Hybrids: which crosses to make

A single-cross hybrid's genetic value is the general combining ability of the female plus that of the male plus their specific combining ability, $u_{ij} = g_i + g_j + s_{ij}$. With `system = "hybrid"` and `pools = list(females, males)`, the units are all possible crosses; the kernel is $\mathbf{K}_{GCA}$ (block diagonal over the two pools) plus `sca_share` times the dominance kernel of the crosses. `crossable` names the parents that can actually be crossed this season; parents that are new (not crossable) can still be targets, which is how the classes of untested hybrids arise (Melchinger et al. 2023):

- **H2**: both parents have been tested in other crosses (easiest to predict);
- **H1**: one parent is new;
- **H0**: both parents are new (hardest).

`predict =` chooses which class the design serves ("all" by default), `max_per_parent` limits how many crosses any parent appears in (seed production and flowering synchrony), `plan$selected` lists the crosses, and `tso_plot(plan, "hybrid_grid")` draws them.

```
ph_panel <- tso_simulate_panel(n = 60, markers = 800, groups = 2, fst = 0.3, seed = 2)
id <- rownames(ph_panel$dosage)
th <- tso_data(ph_panel, system = "hybrid", pools = list(id[1:30], id[31:60]),
              crossable = c(id[1:24], id[31:54]), sca_share = 0.2, verbose = FALSE)
ph <- tso_design(th, n = 100, max_per_parent = 6, verbose = FALSE)
```

ph

```
#> TSOpt plan · hybrid · 100 hybrids of 576 candidates · untargeted (whole pool)
#> Criterion CDmean, exact (fixed intercept) (averaged over h2 = 0.2, 0.5, 0.8) Search
  ↳ greedy (robust CDmean: average over h2) (0.6 s)
#> Expected CDmean 0.467 (at h2 = 0.50) (exact, intercept projected: 0.467) vs random
  ↳ 0.440 ± 0.004 (better than 100% of 100 random sets)
#> Verdict AMBER modest gain (~3.0% accuracy over random); worth using, but do not
  ↳ expect much
#> Coverage nearest training relative >= 0.20 for 100% of targets · 0 targets
  ↳ unreachable (< 0.05)
#> Divide 0 candidates effectively unselectable (reliability < 0.06) · see
  ↳ tso_plot(plan, "divide")
#> Plot value dropping one plot costs 0.37% to 0.51% of the CDmean (median 0.44%) · see
  ↳ tso_plot(plan, "value")
#> Classes untested hybrids: H0 = 36, H1 = 288, H2 = 476
#> Next tso_plot(plan) · tso_export(plan, "trial.csv") · tso_explain(plan)
```

```
tso_plot(ph, "hybrid_grid")
```

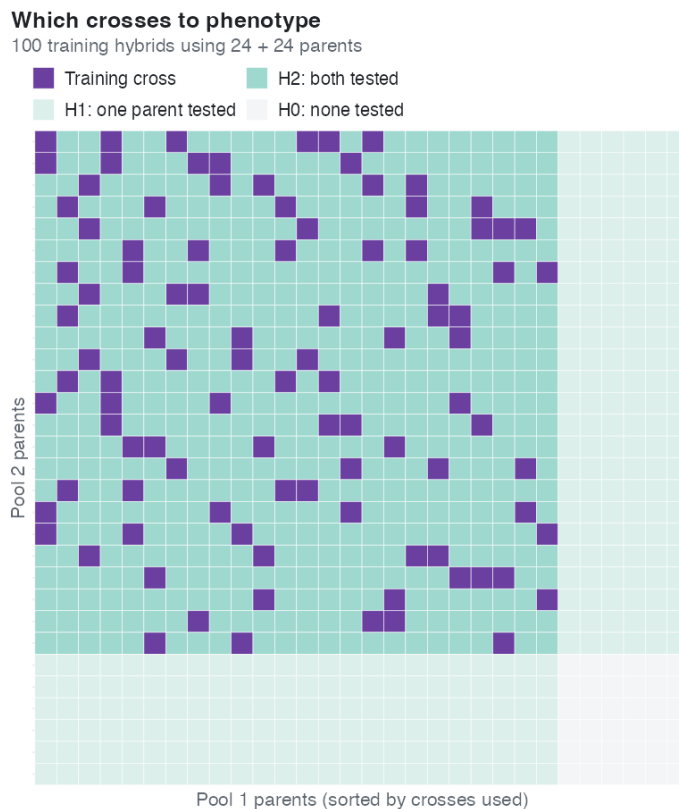


Figure 4: Which crosses to make: females by males, chosen crosses marked, the H0/H1/H2 class of every untested cross in colour.

Watch out

A tester design (every line crossed to one or two testers) is what many programmes run and is the worst design for predicting untested hybrids: it measures GCA in one background and nothing about SCA. `tso_design()` prints a warning when the chosen set looks like one.

7.3 Clonal and polyploid crops

Clones carry their dominance permanently, so `system = "clonal"` adds a dominance kernel with share `dom_share` (default 0.2) to the additive one. Polyploids are read as dosages 0 to ploidy directly (0/0/1/1 genotypes in a VCF, or tetraploid dosage calls), and the additive matrix follows VanRaden's formula with $k = \text{ploidy}$. Planting material is often the binding constraint; give its cost per clone to `cost =` and design with `budget =`.

```
tc <- tso_data(panel, system = "clonal", dom_share = 0.25, verbose = FALSE)
pc <- tso_design(tc, n = 60, verbose = FALSE)
c(system = pc$spec$system, cdmean = round(pc$value$cdmean_exact, 3))
```

```
#>   system   cdmean
#> "clonal" "0.326"
```

7.4 Sparse multi-environment testing

With `envs =`, a unit is a line in one environment and the design decides which line is grown where. The kernel is $\Omega_E \otimes \mathbf{K}$, with Ω_E the genetic correlation between environments, so information flows between sites through the correlation: a line tested in Hyderabad tells you something about its performance in Ludhiana, and the design exploits that. Since v0.4 the model is the full multi-environment model rather than a sketch of it:

argument	what it does
<code>env_cor</code> (in <code>tso_data</code>)	one correlation for every pair, a correlation matrix, or a genetic <i>covariance</i> matrix with site-specific genetic variances on the diagonal (for example from earlier trials)
<code>env_noise</code> (in <code>tso_data</code>)	relative residual variance of each site: a plot at a noisy site (2) is worth about half a plot at a clean one (1)
<code>environment means</code>	added automatically as exact fixed effects (Part 2)
<code>env_target</code>	what the design predicts well: "mean" (default, each line's value averaged over the sites, the target population of environments), "cells" (every line $\times$ site cell) or the name of one site
<code>max_per_line</code>	at most this many sites per line; 1 gives a fully sparse allocation
<code>min_per_env</code> , <code>max_per_env</code>	a floor and a cap on the plots per site

Target lines stand for all their cells: a line named in `target =` is never grown anywhere.

```
p120 <- tso_simulate_panel(n = 120, markers = 600, seed = 3) # a smaller panel,
  ↪ for a readable figure
sites <- c("Hyderabad", "Ludhiana", "Pusa")
Om <- matrix(c(1, .7, .4, .7, 1, .5, .4, .5, 1), 3, dimnames = list(sites, sites))
tm <- tso_data(p120, envs = sites, env_cor = Om, verbose = FALSE,
  env_noise = c(Hyderabad = 1, Ludhiana = 2, Pusa = 1)) # Ludhiana is noisy
pm <- tso_design(tm, n = 120, max_per_line = 1, min_per_env = 30, verbose = FALSE)
pm$spec$fixed
```

```
#> [1] "intercept + 2 fixed column(s) (exact)"
```

```
table(pm$selected$env)
```

```
#>
#> Hyderabad  Ludhiana    Pusa
#>          55         30         35
```

The noisy site Ludhiana gets the floor of 30 plots and no more; every line is grown once. The target matters as much as the noise:

```
alloc <- function(...)
  table(factor(tso_design(tm, n = 120, verbose = FALSE, ...)$selected$env, sites))
met_tab <- rbind(`mean, sparse, floor 30` = table(factor(pm$selected$env, sites)),
  `mean, no constraints` = alloc(),
  `cells` = alloc(env_target = "cells"),
  `Pusa only` = alloc(env_target = "Pusa"),
  `Pusa, floor 20` = alloc(env_target = "Pusa", min_per_env = 20))
met_tab
```

```
#>
#> mean, sparse, floor 30      Hyderabad Ludhiana Pusa
#> mean, no constraints      70         10    40
#> cells                     60         12    48
#> Pusa only                  2          2   116
#> Pusa, floor 20            20         20    80
```

Without constraints the design puts the plots where one plot informs most of the target: for the across-site mean, 70 of 120 go to Hyderabad, the clean site best correlated with the other two, and 10 to noisy Ludhiana; for Pusa as the only target, 116 go to Pusa itself. A floor per site is how a programme keeps every location in the trial (for estimating its mean and its noise, which the design takes as given).

```
tso_plot(pm, "met_allocation")
```

Sparse-testing allocation

120 plots in 3 environments · 120 of 120 lines tested



Figure 5: Line x environment allocation of the sparse design: each line is grown at one site.

The same machinery answers “which sites to test in”: give many environments, the covariance

between them and a small n , and the allocation concentrates in the environments that are most informative about the target environment.

7.5 Testcrosses and three or more heterotic groups

system = "testcross" evaluates candidate lines as testcrosses with testers: a unit is line $\times$ tester, with covariance $(v_{GCA}J + v_{SCA}K_T) \otimes K_L$ (K_T the tester relationships, or I when testers are not genotyped). The structure is that of lines $\times$ environments, so testers get their own means and lines can be spread over them; the design predicts the lines' GCA by default (value = "gca", contrasts from $v_{GCA}J \otimes K_L$). For three or more heterotic groups, pools is a named list and cross_groups the pairs crossed; the single cross $i \times j$ has $\text{Cov}(g_{ij}, g_{kl}) = \sigma_a(K_{ik} + K_{il} + K_{jk} + K_{jl}) + \sigma_s(K_{ik}K_{jl} + K_{il}K_{jk})$.

```
tcx <- tso_data(panel, system = "testcross", testers = rownames(panel$dosage)[1:3],
  ↪ verbose = FALSE)
table(tso_design(tcx, n = 45, max_per_line = 1, verbose = FALSE)$selected$tester)
```

```
#>
#> L001 L002 L003
#> 13 20 12
```

Large networks. Above 8,000 line $\times$ trait $\times$ environment cells the unit covariance $\Omega_E \otimes G_T \otimes K$ is kept factorised (implicit = TRUE): the engine needs columns, small blocks and products with a few columns, all computed from the factors, and index contrasts are kept as $(w'S) \otimes K_c$ and expanded only for the search. Designs are identical to the dense form; 1,000 lines $\times$ 15 sites take about two and a half minutes.

7.6 Outcrossing crops and families

For forages, rye or other outcrossers where the phenotyped unit is a half-sib or full-sib family, system = "outcross" with families = uses family-mean contrasts; the family's representative genotype is the pooled allele frequency of its members. The cdpop criterion is the natural one.

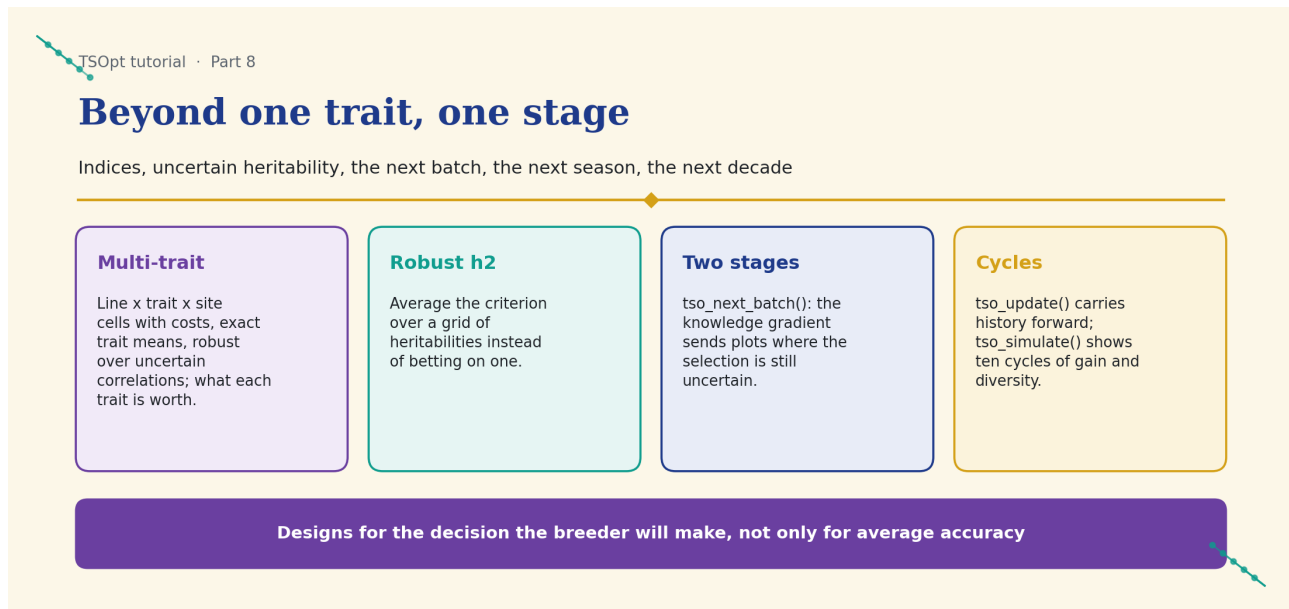
7.7 Genebanks and reference panels

A genebank wants a phenotyped core that predicts the rest of the collection, and its structure is signal, not nuisance. system = "genebank" (or goal = "reference_panel") makes the design untargeted, stratifies whenever groups exist, and favours coverage; criterion = "cdmin" adds a floor so that no accession is left unpredictable.

```
gb <- tso_simulate_panel(n = 300, markers = 800, groups = 5, fst = 0.25, seed = 4)
tg <- tso_data(gb, system = "genebank", verbose = FALSE)
pg <- tso_design(tg, n = 60, goal = "reference_panel", verbose = FALSE)
table(pg$selected$group)
```

```
#>
#> G1 G2 G3 G4 G5
#> 12 12 12 12 12
```

8 Beyond one trait, one stage



8.1 Multi-trait index designs: which trait on which line

Breeders select on an index of several traits, and traits cost different amounts: yield needs a plot and a harvest, protein a lab assay, canopy NDVI almost nothing once the plot exists. With traits $=$, a unit is *one trait measured on one line*; the kernel is $\mathbf{G}_T \otimes \mathbf{K}$ with $\mathbf{G}_T$ the genetic correlations gcor between traits (and trait_h2 setting the residual of each); the contrasts are index contrasts $\mathbf{b} \otimes \mathbf{c}_i$ with the index weights index = (Jia and Jannink 2012). The cost of a cell is trait_cost plus plot_cost, and the plot cost is paid once per line: measuring NDVI on a plot that is already grown for yield costs only the scan. The design then buys the cells that most improve the reliability of the index per unit cost. The expensive trait goes on few lines, the cheap correlated trait on many.

Since v0.5 the model is the one breeders fit: every trait has its own fixed mean, handled exactly (Part 2), and every trait is opened with two units, because the first measurement of a trait only estimates its mean.

```
R <- matrix(c(1, 0.5, 0.6, 0.5, 1, 0.3, 0.6, 0.3, 1), 3,
            dimnames = rep(list(c("yield", "protein", "ndvi")), 2))
tt <- tso_data(panel, traits = c("yield", "protein", "ndvi"), trait_h2 = c(0.3, 0.5, 0.6),
              gcor = R, trait_cost = c(20, 35, 0.5), plot_cost = 10, verbose = FALSE)
pt <- tso_design(tt, budget = 3000, index = c(yield = 1, protein = 0.4), verbose = FALSE)
pt
```

```
#> TSOpt plan · inbred · 196 line x trait cells of 900 candidates · untargeted (all
  ↳ lines), index yield=1 + protein=0.4 + ndvi=0
#> Criterion index CDmean, exact (intercept + 2 fixed effects) (trait heritabilities
  ↳ from the data) Search greedy (robust over 5 trait heritability / correlation
  ↳ scenarios) (1.4 s)
#> Expected CDmean 0.345 (exact, intercept projected: 0.344) vs random 0.275 ± 0.006
  ↳ (better than 100% of 100 random sets)
#> Scenarios CDmean given 0.345 · h2_lower 0.257 · h2_higher 0.416 · gcor_weaker 0.292 ·
  ↳ gcor_stronger 0.423
#> Verdict GREEN optimisation pays off here: expected accuracy ~11.9% above random
#> Coverage nearest training relative >= 0.20 for 100% of targets · 0 targets
  ↳ unreachable (< 0.05)
```

```
#> Divide      0 candidates effectively unselectable (reliability < 0.06) · see
↳ tso_plot(plan, "divide")
#> Traits      yield: 88 lines · protein: 4 lines · ndvi: 104 lines
#> Plot value dropping one plot costs 0.08% to 0.85% of the CDmean (median 0.17%) · see
↳ tso_plot(plan, "value")
#> Budget      spent 2992.0 of 3000.0
#> Next        tso_plot(plan) · tso_export(plan, "trial.csv") · tso_explain(plan)
```

```
pt$traits
```

```
#>   trait lines weight h2
#> 1  yield   88     1.0 0.3
#> 2 protein    4     0.4 0.5
#> 3  ndvi  104     0.0 0.6
```

```
tso_plot(pt, "trait_allocation")
```

Which trait to measure on which line

104 lines grown · index yield 1, protein 0.4, ndvi 0



Figure 6: Which trait is measured on which line, for the budget.

`tso_compare()` includes the naive alternative, *measure every trait on the lines you grow*, so that the value of allocating traits separately is visible.

8.2 Traits and environments together

Since v0.5, `traits` = and `envs` = combine. A unit is then one trait measured on one line in one environment, with genetic covariance $\Omega_E \otimes \mathbf{G}_T \otimes \mathbf{K}$: information flows between traits through $\mathbf{G}_T$ and between sites through Ω_E . Every trait has its own mean at every site, exactly, and every trait $\times$ site cell is opened with two units. A plot is one line in one environment, so the plot cost is paid once per line *per site*: a line grown in Hyderabad and in Ludhiana is two plots. `env_target = "mean"` (the default) designs for the index averaged over the sites; the name of one site designs for the index there.

```
te <- tso_data(p120, traits = c("yield", "ndvi"), trait_h2 = c(0.3, 0.6),
              gcor = matrix(c(1, 0.6, 0.6, 1), 2), trait_cost = c(yield = 20, ndvi = 0.5),
              plot_cost = 10, envs = sites[1:2], env_cor = 0.6, verbose = FALSE)
pe <- tso_design(te, budget = 2500, index = c(yield = 1, ndvi = 0), verbose = FALSE)
table(pe$selected$trait, pe$selected$env)
```

```
#>
#>      Hyderabad Ludhiana
#> ndvi      44      53
#> yield      38      36
```

```
n_plots <- length(unique(paste(pe$selected$line_id, pe$selected$env)))
c(cells = nrow(pe$selected), plots = n_plots, trait_costs = sum(pe$selected$cost),
  plot_costs = 10 * n_plots, spent = pe$spent)
```

```
#>      cells      plots trait_costs plot_costs      spent
#>      171       97      1528      970      2498
```

```
pe$spec$fixed
```

```
#> [1] "intercept + 3 fixed column(s) (exact)"
```

The 171 cells occupy 97 plots; the spend is the trait costs plus 10 per plot, $2498.5 = 2498.5$. The three fixed columns are the four trait $\times$ site means less the one absorbed by the intercept.

8.3 Robust multi-trait design: uncertain correlations and heritabilities

The genetic correlations and trait heritabilities of a multi-trait design are estimates, usually from a few hundred lines of one or two trials, and a correlation of 0.6 estimated that way could well be 0.3 or 0.8. A design that leans on one correlation (put the budget on the cheap proxy, because it predicts yield) is only as good as that number. The multi-trait counterpart of robust CDmean is therefore the default, `trait_uncertainty = "auto"`: the index criterion is averaged over five scenarios, run in lock-step engines exactly as the heritability grid is.

scenario	heritabilities	genetic correlations
given	as given	as given
h2_lower	40% lower	as given
h2_higher	40% higher (at most 0.95)	as given
gcor_weaker	as given	halved towards 0
gcor_stronger	as given	moved halfway towards ± 1 (skipped when not positive definite)

With no correlations between traits (an identity gcor), the two correlation scenarios are dropped. The plan prints its CDmean under every scenario (the Scenarios line of pt above, `plan$value$cd_by_scenario`). To see what the hedge buys, score the robust design and the design for the given values only (`trait_uncertainty = "none"`) under all five:

```
ptn <- tso_design(tt, budget = 3000, index = c(yield = 1, protein = 0.4),
                 trait_uncertainty = "none", verbose = FALSE)
score_scenarios <- function(p) { # exact index CDmean of a plan under each
  ↪ scenario
  d <- p$data; u <- d$units; nl <- p$spec$n_lines_block
  tl <- unique(match(u$line_id[p$index$target], u$line_id[seq_len(nl)]))
  S <- c(p$index$history, p$index$selected)
  sapply(TSOpt:::.trait_scenarios(d$gcor, d$trait_h2), function(sc) {
    Ks <- TSOpt:::.mt_kernel(d, sc$gcor)
```

```

    s2 <- pmax((1 - sc$h2[u$trait]) / sc$h2[u$trait], 1e-6)
    TSOpt:::cd_direct(Ks, S, s2, TSOpt:::contrasts_index(Ks, nl, p$spec$index_blocks,
  ↪ tl, TRUE))
  })
}
stopifnot(isTRUE(all.equal(score_scenarios(pt), pt$value$cd_by_scenario))) # same
  ↪ numbers as the plan
sc_tab <- rbind(robust = score_scenarios(pt), `given values only` = score_scenarios(pntn))
sc_tab <- cbind(sc_tab, mean = rowMeans(sc_tab), worst = apply(sc_tab, 1, min))
round(sc_tab, 4)

```

```

#>               given h2_lower h2_higher gcor_weaker gcor_stronger   mean   worst
#> robust          0.3451   0.2574   0.4158   0.2921   0.4230 0.3467 0.2574
#> given values only 0.3452   0.2576   0.4153   0.2955   0.4179 0.3463 0.2576

```

The two designs share 158 of their 196 and 192 cells, and their CDmeans differ by at most 0.0051 in any scenario: the robust design is ahead in 2 of the 5 scenarios, most under gcor_stronger (+0.0051), and behind most under gcor_weaker (-0.0034). On this example, with moderate parameters, the hedge is cheap insurance rather than a large gain. The scenario row itself is the more useful output: the spread from 0.257 to 0.423 is how much the expected index reliability depends on parameters nobody knows well, and it is the honest range to report.

8.4 What is each trait worth? Proxy and high-throughput traits

Drones, phenomics carts and NIRS make some traits almost free once the plot is grown: canopy NDVI, canopy temperature, early vigour, predicted protein. They are rarely in the selection index, but a cheap, heritable trait that is genetically correlated with yield carries information about yield, and can raise the accuracy of the index more per unit cost than another yield plot (Rutkoski et al. 2016). `tso_trait_value(plan)` measures how much, trait by trait:

- `share_of_budget`: what the design spent on the trait;
- `value_per_cost`: the index reliability one more measurement adds per unit of cost, relative to the best trait (1 = the best buy at the margin);
- `exchange_rate`: how many measurements of the trait carry the information of one measurement of the reference trait (the most weighted one, here yield): the number of NDVI scans that replace one yield plot;
- `loss_without_pct`: the loss of index CDmean when the trait is dropped altogether and its money spent on the other traits (the design is re-run without it, same budget and targets).

```

tv <- tso_trait_value(pt)
tv

```

```

#> TSOpt trait value · 3 traits · index CDmean of the plan 0.345 · exchange rates in yield
  ↪ measurements
#>   trait weight cost lines budget value per cost worth 1 ref CDmean lost without it
#>   yield   1.0 20.0   88   90%       1.00       1.0       15.8%
#> protein   0.4 35.0    4    7%       0.94       0.7        0.5%
#>   ndvi    0.0  0.5  104    3%       0.91       2.6       13.4%
#> -> yield gives the most information per unit of cost at the margin; ndvi (index weight
  ↪ 0) is measured as a proxy.

```

```
tso_plot(tv)
```

What each trait is worth in the design

index CDmean 0.345 · exchange rates in yield measurements

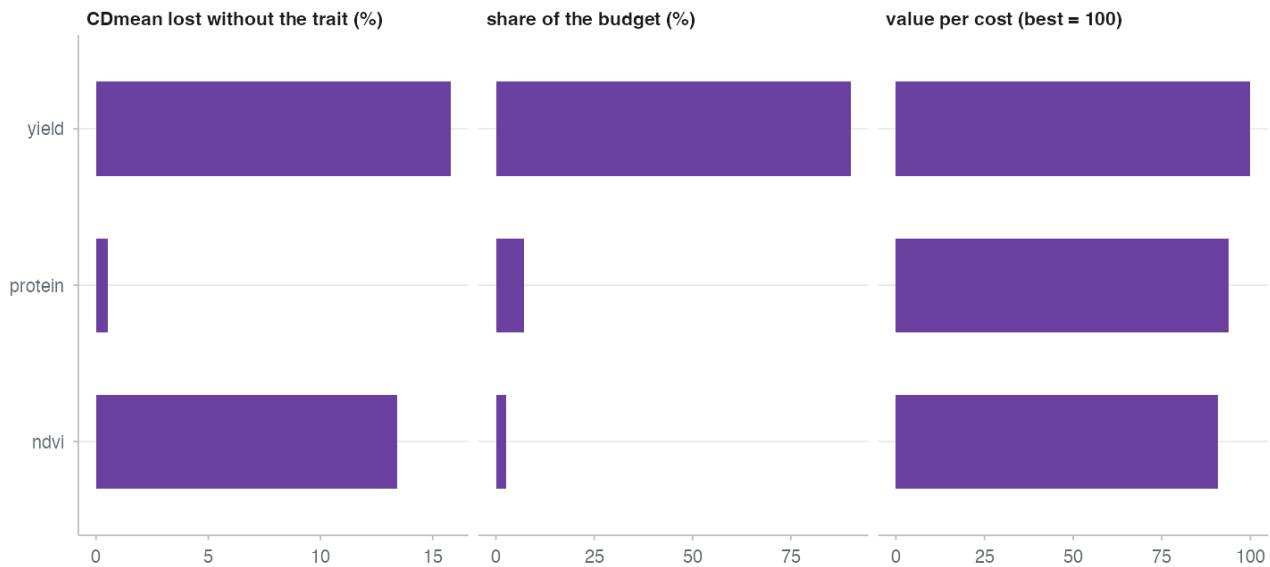


Figure 7: `tso_plot(trait value)`: what each trait costs the budget and what the index loses without it.

For the breeder

NDVI has index weight 0 and takes 2.7% of the budget, yet dropping it (and spending its money on yield and protein) costs 13.4% of the index CDmean; protein, with weight 0.4 but only 4 lines, costs 0.5%. At the margin 2.6 NDVI scans carry the information of one yield plot, and a scan costs 40 times less than a yield measurement. Before cancelling the drone flights to save money, run `tso_trait_value()`; before adding a new sensor, give it a trait, a cost and a plausible genetic correlation, and let the design say whether it pays.

The exchange rates and losses take the genetic correlations as given. The robust design above hedges the allocation; for the value of a proxy, re-run `tso_data()` with a weaker correlation between the proxy and yield and call `tso_trait_value()` again: a proxy whose value vanishes at half the correlation is a bet on that correlation.

8.5 Robust design over an uncertain heritability

Part 3 introduced the default grid. The mechanism is general: give any vector to `h2` and the criterion is averaged over it, which is a discrete prior. `plan$value$cd_by_h2` reports the CDmean of the chosen set at each value, so you can see how much accuracy is at stake:

```
pr <- tso_design(tp, n = 60, h2 = c(0.1, 0.3, 0.5, 0.7), verbose = FALSE)
round(pr$value$cd_by_h2, 3)
```

```
#> h2=0.1 h2=0.3 h2=0.5 h2=0.7
#> 0.102 0.242 0.346 0.437
```

8.6 Two-stage designs: where the next plots go

Before any phenotype exists, the criteria of Part 3 can only use relationships. After a first stage, phenotypes exist, and a Bayesian decision rule can use them. `tso_next_batch()`:

1. estimates h^2 from the stage-1 data by REML (or takes $h^2 =$);
2. computes the posterior mean and variance of every unit's genetic value given the stage-1 phenotypes (the engine state with every observation added, and the posterior mean from the GBLUP);
3. for each candidate plot, computes the **knowledge gradient** (Frazier et al. 2009): the expected improvement in the expected value of the top select_top fraction if that plot were observed (Gauss-Hermite quadrature over the plot's predictive distribution, n_quad nodes);
4. adds the plot with the largest gradient, updates the posterior as if it had been observed (the *kriging believer* rule: means kept, variance shrunk), and repeats until n plots.

Plots go to the candidates whose side of the selection threshold is most uncertain, and to the relatives that resolve them. `retest = TRUE` allows a second plot on a promising phenotyped unit when that is worth more than a new line. `pool` restricts the pool the top fraction is selected from, `candidates` restricts who may receive plots.

```
stage1 <- tso_design(tp, n = 60, verbose = FALSE)
nb <- tso_next_batch(stage1, y[stage1$selected$id], n = 30, select_top = 0.1)
nb
```

```
#> TSOpt next batch · 30 plots · selecting the top 10% (30 of 300) · 60 stage-1
  ↳ observations
#> Posterior      h2 = 0.19 (estimated from stage 1)
#> Batch          29 new units, 1 re-tests of promising phenotyped units
#> Boundary       158 units have a 10-90% chance of being selected: these decide where plots
  ↳ go
#> Next           tso_plot(batch) · batch$selected · batch$posterior
```

```
head(nb$posterior[order(-nb$posterior$p_top), ], 5)
```

```
#>      id   mean    sd p_top phenotyped next_batch
#> L128 L128 0.7275 0.8396 0.247      TRUE      FALSE
#> L187 L187 0.6997 0.8531 0.228      TRUE      FALSE
#> L112 L112 0.6322 0.8513 0.224      TRUE      FALSE
#> L154 L154 0.6127 0.8454 0.203      TRUE      FALSE
#> L114 L114 0.5354 0.8957 0.195     FALSE      FALSE
```

```
tso_plot(nb)
```

Where the next plots go

Dashed: selection threshold (top 10%) · plots go where the ranking is uncertain

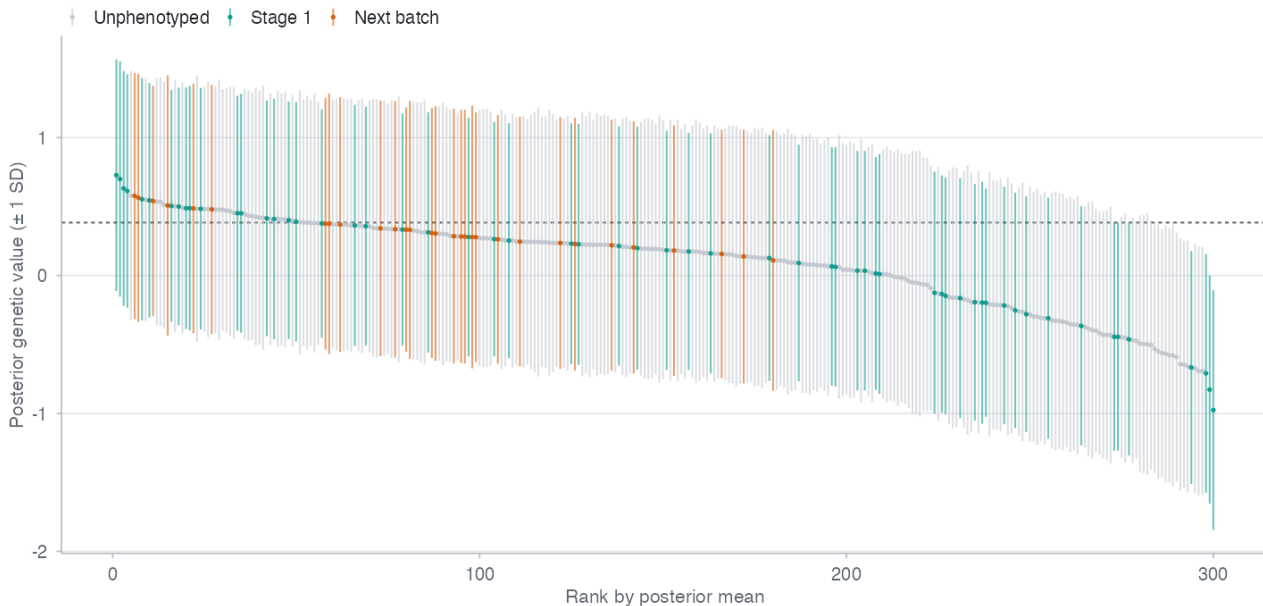


Figure 8: Posterior ranking after stage 1 and where the next 30 plots go.

`nb$posterior` is a ranking with honest uncertainty: posterior mean, SD and probability of being in the top fraction, for every unit. It is worth taking to the selection meeting on its own.

8.7 Across seasons: `tso_update()`

Training sets go stale: a set that predicted this year's candidates well predicts next year's worse, and a programme that never adds phenotypes loses accuracy cycle by cycle (Hussain 2026). `tso_update()` takes last season's plan and this season's genotypes (`geno =`, read with the stored base allele frequencies so that relationships across seasons are correct) or `tso_data` object, marks everything already phenotyped as history with `weight_history_weight` (0.8 by default, to discount older data), and designs the `n` new plots that add most.

```
season2 <- tso_update(plan, data = tp, n = 40, verbose = FALSE)
c(history = season2$spec$history, new = nrow(season2$selected),
  cdmean_all = round(season2$value$cdmean_exact, 3))
```

```
#>   history      new cdmean_all
#>   60.000    40.000     0.423
```

8.8 The long term: `tso_simulate()`

Every published comparison of criteria looks at one cycle of accuracy. `tso_simulate()` runs recurrent genomic selection for several cycles under each strategy and tracks genetic gain, genetic variance, accuracy and favourable alleles lost. Each cycle: a training set is chosen by the strategy, phenotyped, a GBLUP is fitted, the top `n_parents` by GEBV are crossed (`n_crosses`), and `n_candidates` new progeny are simulated with recombination on chromosomes. `window` limits the training data to the last cycles (a sliding window) instead of accumulating.

```
sim <- tso_simulate(strategies = list(random = "random", cdmean = "cdmean"), cycles = 5,
  reps = 2, n_candidates = 150, n_crosses = 15, n_train = 50, n_parents =
    ↪ 15,
```

```

sim                                markers = 600, n_qtl = 100, verbose = FALSE)

#> TSOpt long-term simulation · 5 cycles · 2 replicates · 50 of 150 candidates phenotyped,
↳ 15 parents
#> Strategy Gain (founder SD) Genetic variance (vs founders) Mean accuracy Favourable
↳ QTL lost
#> cdmean    3.25                0.45                0.51        29.0
#> random    2.82                0.15                0.44        31.0
#> Coancestry of parents
#> 0.75
#> 0.88
#> Values at the last cycle, averaged over replicates (accuracy averaged over cycles).

tso_plot(sim)

```

Long-term consequences of the training-set strategy

2 replicates · mean $\pm$ 1 SE · 50 of 150 phenotyped per cycle

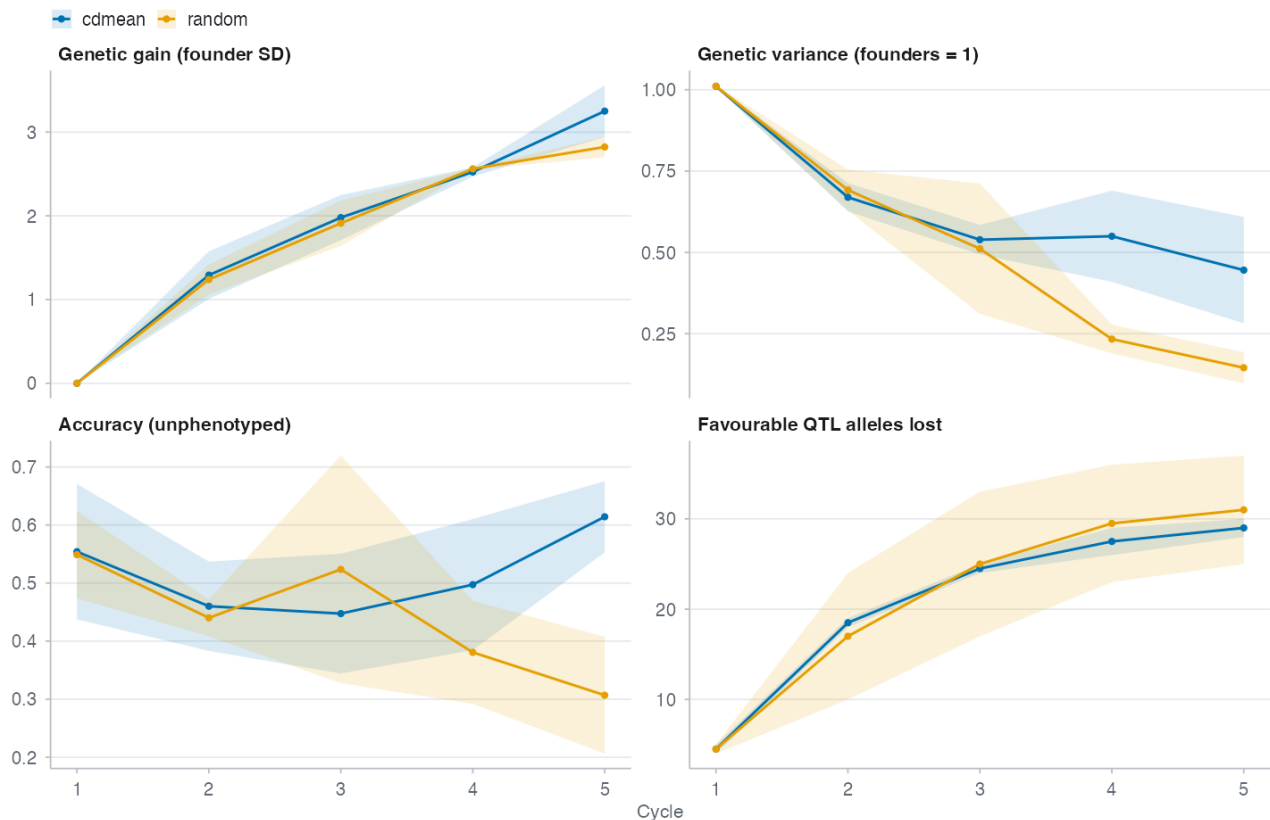


Figure 9: Gain, genetic variance and accuracy over cycles, by strategy.

Use the defaults (six cycles, two replicates, 200 candidates) or more before drawing conclusions: the run above is kept small so that this document builds quickly.

9 Major genes and haplotypes

A genome-wide relationship matrix gives every marker the same weight. A major gene, one locus among thousands, is a vanishing part of it, so CDmean cannot see whether a training set contains both of its alleles. When one allele is absent from the training set, its effect cannot be learned, and every target that carries it is predicted from the polygenic background alone. Resistance genes, introgressed segments, a photoperiod or a dwarfing gene: this is the ordinary case in plant breeding, not an exotic one.

Key point

A training set can be optimal for CDmean and useless for a major gene. TSOpt v0.4 finds major loci (`tso_major_scan()`), builds haplotypes around them (`tso_haplotypes()`), gives them their share of the genetic variance or treats them as fixed effects, and guarantees that every allele the targets carry appears in the training set a minimum number of times (`tso_data(major = , major_share = , major_model = , min_carriers = )`). Every plan, with or without `major =`, reports how many of the targets' alleles and haplotypes the training set covers.

9.1 `tso_major_scan()`: finding the loci

The scan is EMMAX (Kang et al. 2010): one REML fit of the polygenic model, then a generalised least-squares test of every marker with the variance components held fixed, so a genome of markers costs one matrix factorisation. Leads are *clumped*: after the most significant marker is taken, markers within window markers of it on the same chromosome or in LD with it ($r^2 > \text{clump_r2}$) are not reported as separate loci. For each lead the scan reports its effect, its $-\log_{10} p$, and its share of the genetic variance, alone and jointly with the other leads; the joint share is what `tso_data()` uses.

Phenotypes for the scan come from an earlier trial or from the first stage of this one. Here a trait is simulated on the tutorial panel with one planted major QTL explaining 40% of the genetic variance (`tso_simulate_trait(major = , major_share = )`):

```
tr_maj <- tso_simulate_trait(panel, h2 = 0.6, n_qlt = 80, major = 1, major_share = 0.4,
                             seed = 11)
scan <- tso_major_scan(panel, setNames(tr_maj$y, tr_maj$id), verbose = FALSE)
scan
```

```
#> TSOpt major-locus scan · 300 lines · 958 markers · mixed model (EMMAX) (h2 = 0.52)
#> Threshold -log10(p) >= 4.3
#> Major loci 1: m855 (chr 9, -log10 p 8.7, 34% of genetic variance)
#> Next      tso_data(geno, major = scan) (uses a 34% variance share at these loci)
scan$leads[, c("marker", "chrom", "effect", "log10p", "share_genetic", "share_joint")]
```

```
#>   marker chrom  effect log10p share_genetic share_joint
#> 1   m855    9 -0.7341  8.664      0.3563      0.3433
```

The planted locus is m855; the scan's lead is m855 (found: TRUE), with an estimated 34% of the genetic variance for the planted 40%.

```
tso_plot(scan)
```

1 major locus/loci found

mixed model (EMMAX) · 300 lines

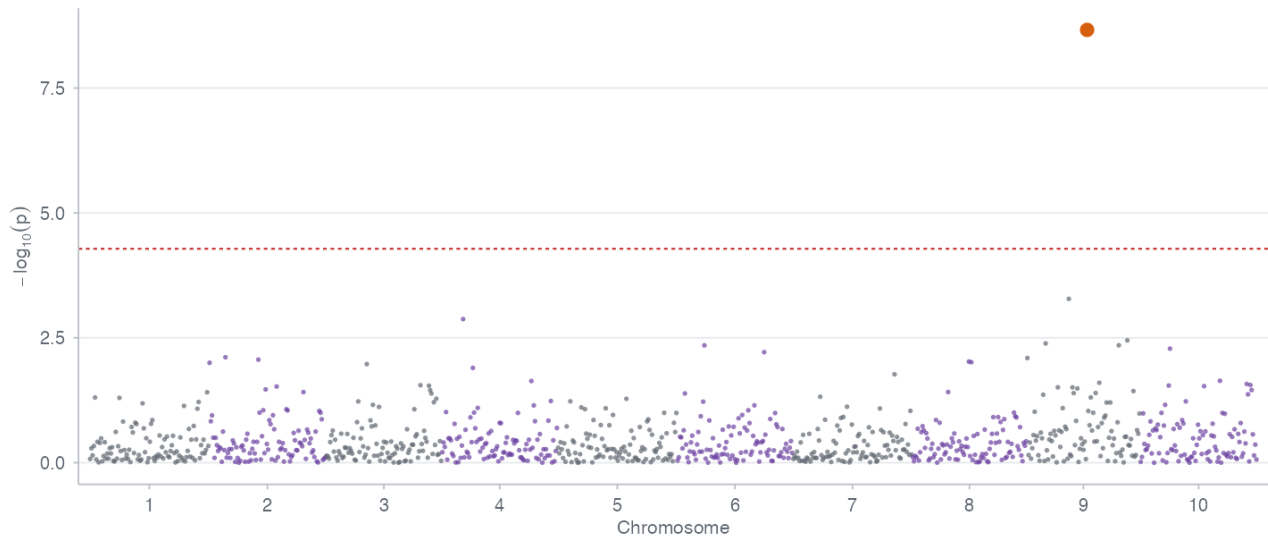


Figure 10: `tso_plot(scan)`: the Manhattan plot, with the Bonferroni threshold and the clumped leads.

Watch out

Scanning and designing on the same phenotypes overstates the share of variance at the lead (the winner's curse), and a scan of a few hundred lines finds only loci of large effect. Treat `major_share` from a small scan as an upper bound, prefer loci confirmed in earlier material, and give known genes (from mapping or from a diagnostic marker) by name.

9.2 `tso_major_effects()`: loci from marker effects you already have

Phenotypes are not the only source of major loci. A published GWAS or QTL study, last season's BayesB model or a cloned gene with a diagnostic marker already gives marker effects. `tso_major_effects()` turns them into the same object as the scan, so `tso_data(major = )`, the plan's Major line treat it identically.

The effects may be a named vector, a data frame or a CSV or tab-separated file. The marker, effect, effect-allele and p-value columns are recognised by their usual names (`sn`, `beta`, `a1`, `pval`, ...). Each effect is aligned to the allele your dosages count, so an effect given for the other allele is flipped. Then every marker's contribution is computed *in your panel*: $b_j^2 \text{Var}(x_j)$ over your candidates, so a large effect at an allele that is rare in your material gets a small share. Leads are clumped as in the scan.

The share is relative to a genetic variance V_G : `var_g`, else `h2 * var_p`, else, for a genome-wide model (100 or more markers, or 20% of them), the variance of the genomic values $\mathbf{Xb}$ the effects predict here. Two kinds of effects need different treatment (type):

- **"joint"**: one model on all markers (BayesB, BayesC, BayesR, LASSO, known genes). A QTL split over several markers in LD is summed over its block (`share_region`, markers with $r^2 \geq \text{region_r2}$ within window).
- **"marginal"**: GWAS, one marker at a time. Marginal effects overlap through LD and are never summed. Only markers with $p \leq p_{\text{max}}$ (Bonferroni by default) are candidates, and V_G must be given. A table with a p-value column is read as GWAS by default. GWAS effects

from a small study are inflated by the winner's curse (the Beavis effect, [Beavis 1994](#)), so the shares they give are upper bounds.

The example panel ships with a file of effects from a genome-wide model for a resistance score: two major genes (about 25% and 10% of the genetic variance) on a polygenic background, one of them reported for the other allele.

```
g_ex <- tso_read(system.file("extdata", "example_panel.vcf.gz", package = "TSOpt"),
  ↪ verbose = FALSE)
eff <- read.csv(system.file("extdata", "example_marker_effects.csv", package = "TSOpt"))
me <- tso_major_effects(g_ex, eff)
```

```
#> TSOpt major loci from marker effects · 300 lines · 290 of 290 markers matched (1 effect
  ↪ alleles flipped)
#> Variance variance of the genomic values the effects predict here (0.120)
#> Major loci 2: m146 (chr 2, MAF 0.20, 22% of genetic variance); m348 (chr 4, MAF 0.23,
  ↪ 9% of genetic variance)
#> Next tso_data(geno, major = effects) (uses a 31% variance share at these loci)
me$leads[, c("marker", "chrom", "maf", "effect", "share_genetic", "share_region")]
```

```
#> marker chrom maf effect share_genetic share_region
#> 1 m146 2 0.2033 0.2003 0.21756 0.21756
#> 2 m348 4 0.2267 -0.1218 0.08705 0.08705
```

```
tso_plot(me)
```

2 major locus/loci from the marker effects

290 marker effects · 300 lines

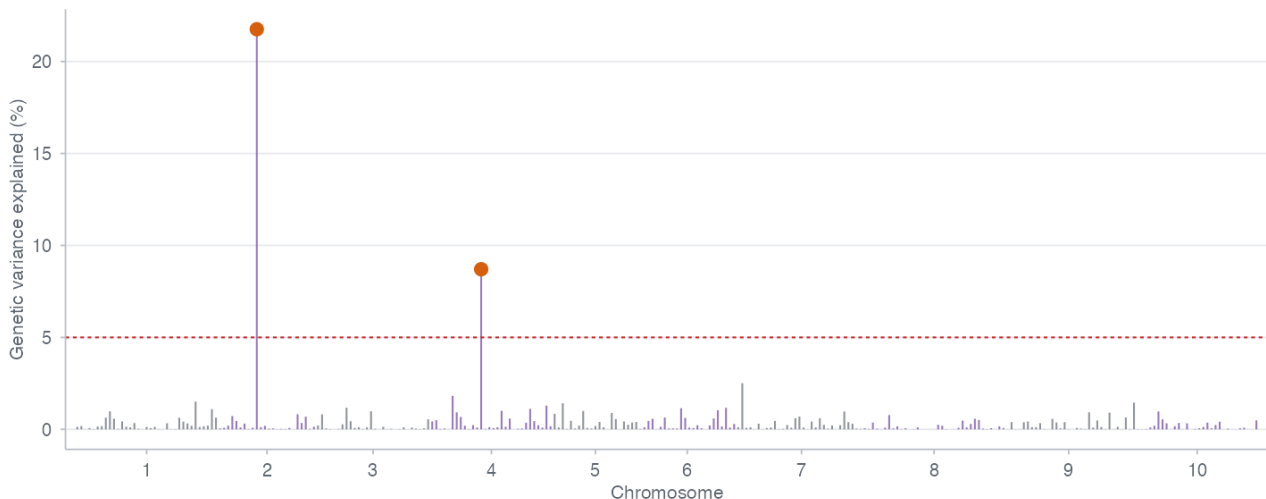


Figure 11: `tso_plot(effects)`: the genetic variance each marker explains in this panel, with the 5% line and the leads.

The same function reads GWAS summary statistics; here the scan's table stands in for a published one:

```
gw <- data.frame(snp = scan$table$marker, beta = scan$table$effect, pval =
  ↪ 10^-scan$table$log10p)
tso_major_effects(panel, gw, h2 = 0.6, var_p = var(tr_maj$y))
```

```
#> TSOpt major loci from marker effects · 300 lines · 958 of 958 markers matched
#> Variance h2 x phenotypic variance (1.068)
#> Major loci 1: m855 (chr 9, MAF 0.19, 31% of genetic variance)
```

```
#> Next      tso_data(geno, major = effects) (uses a 31% variance share at these loci)
```

Watch out

Ridge effects (rrBLUP, GBLUP back-solved to markers) cannot size a major gene: ridge regression shrinks every effect by the same factor and spreads a large QTL over thousands of markers. In simulations, a QTL explaining 30% of the genetic variance received well under 1%. TSOpt recognises dense, near-normal effects and warns. Use GWAS or BayesB effects, or the scan, or give the known loci by name with `major_share`.

With either source, each protected locus is weighted in K_{major} by the share it explains, not equally:

```
tp_me <- tso_data(g_ex, major = me, verbose = FALSE)
c(loci = paste(tp_me$major$loci, collapse = ", "), share = round(tp_me$major$share, 2))
```

```
#>      loci      share
#> "m146, m348" "0.31"
```

9.3 tso_haplotypes(): alleles of a block

A single SNP can be a poor tag of the causal allele; the haplotype of a few markers around it is a better one. `tso_haplotypes()` builds haplotype alleles from runs of adjacent markers: around chosen markers (`around =`, with window markers on each side), for blocks you define (`blocks =`), or tiled along the whole genome. For inbred lines the multi-locus string is the haplotype; for phased heterozygous material (a 0|1 VCF) each chromosome copy gives one; for unphased heterozygous material it is the local multi-locus genotype.

```
hp <- tso_haplotypes(panel, around = scan$leads$marker[1], window = 2)
hp
```

```
#> TSOpt haplotypes · 1 block(s), fixed 5-marker tiles · 300 lines · homozygous lines:
↪ strings are haplotypes
#> markers per block: median 5 (range 5-5) · haplotypes per block: median 19
#> hap_m855      5 markers · 19 haplotypes (17 with freq >= 2%) · top 14%
```

```
round(head(sort(hp$freq[[1]], decreasing = TRUE), 6), 3)
```

```
#>
#> 00011 01001 01011 01010 10110 00111
#> 0.140 0.133 0.117 0.063 0.057 0.053
```

Haplotypes rarer than `min_freq` (default 2%) are pooled as "rare" when they are turned into allele classes. A `tso_haplotypes` object can be passed to `tso_data(major = )` in place of marker names, or together with them as `major = list(markers = , haplotypes = )`.

9.4 Genome-wide haplotypes: blocks, the haplotype kernel and coverage

Blocks. `tso_haplotypes(method = )` tiles the genome into "fixed" blocks of $2w + 1$ markers, "window" blocks of kb kilobases, or "ld" blocks that grow while a new marker's mean r^2 with the block stays at or above r^2 (default 0.3, at most `max_markers`), a greedy rule in the spirit of the Haploview and `plink --blocks` blocks (Gabriel et al. 2002). Blocks from `plink --blocks` (`.blocks`, `.blocks.det`) can be read directly.

Phase. Inbred lines are homozygous and need no phasing (99% homozygous calls count as homozygous). For heterozygous material a phased VCF (0|1) keeps one haplotype per chromosome copy through `tso_read()`, `tso_qc()` and `tso_write()`; without phase, blocks hold local multi-locus genotypes. TSOpt does not phase: use Beagle, SHAPEIT or Eagle.

Kernel. "HAP" in `tso_kernels()` treats block b as a multi-allelic locus with haplotype dosages h_{ibl} (copies of haplotype ℓ carried by line i):

$$\mathbf{K}_{HAP} \propto \sum_b \sum_{\ell} (h_{.bl} - \bar{h}_{b\ell})(h_{.bl} - \bar{h}_{b\ell})',$$

scaled to mean diagonal one, with haplotypes rarer than `min_freq` pooled. With single-marker blocks it is exactly VanRaden's matrix with panel frequencies (a test checks it); longer blocks add identity by descent. It is additive, so it mixes with A under robust shares.

Coverage and the criterion. With `tso_data(haplotypes = )` every plan reports the share of the targets' distinct haplotypes in training, the share of their haplotype copies, and the share with at least two training carriers, per block (`tso_plot(plan, "haplotypes")`). `criterion = "haplotype"` maximises

$$F(S) = \sum_c w_c (1 - 2^{-n_c(S)}),$$

with w_c the target copies of haplotype c and $n_c(S)$ its training carriers. Each extra carrier is worth half the previous one, so a second carrier (needed to estimate an effect) is still rewarded; F is monotone submodular, so the greedy set is within $1 - 1/e$ of the optimum (Nemhauser et al. 1978). `tso_pareto()` has the objective "haplotypes".

```
hp_fx <- tso_haplotypes(panel, method = "fixed", window = 2)
tp_hap <- tso_data(panel, haplotypes = hp_fx, verbose = FALSE)
tgt_h <- rownames(panel$dosage)[seq(3, nrow(panel$dosage), length.out = 60)] # targets
  ↪ across the panel
p_cd <- tso_design(tp_hap, n = 40, target = tgt_h, verbose = FALSE)
p_hc <- tso_design(tp_hap, n = 40, target = tgt_h, criterion = "haplotype", verbose =
  ↪ FALSE)
round(rbind(cdmean = c(unlist(p_cd$alleles$haplotypes[c("frac", "frac2")])), CD =
  ↪ p_cd$value$cdmean),
      haplotype = c(unlist(p_hc$alleles$haplotypes[c("frac", "frac2")])), CD =
  ↪ p_hc$value$cdmean)), 3)
```

```
#>      frac frac2  CD
#> cdmean  0.877 0.633 0.273
#> haplotype 0.900 0.632 0.249
```

```
tso_plot(p_cd, "haplotypes")
```

88% of the targets' haplotypes are in the training set

200 blocks · 163 blocks miss a target haplotype (marked) · 95% of target haplotype copies covered · 63% with ≥ 2 carriers

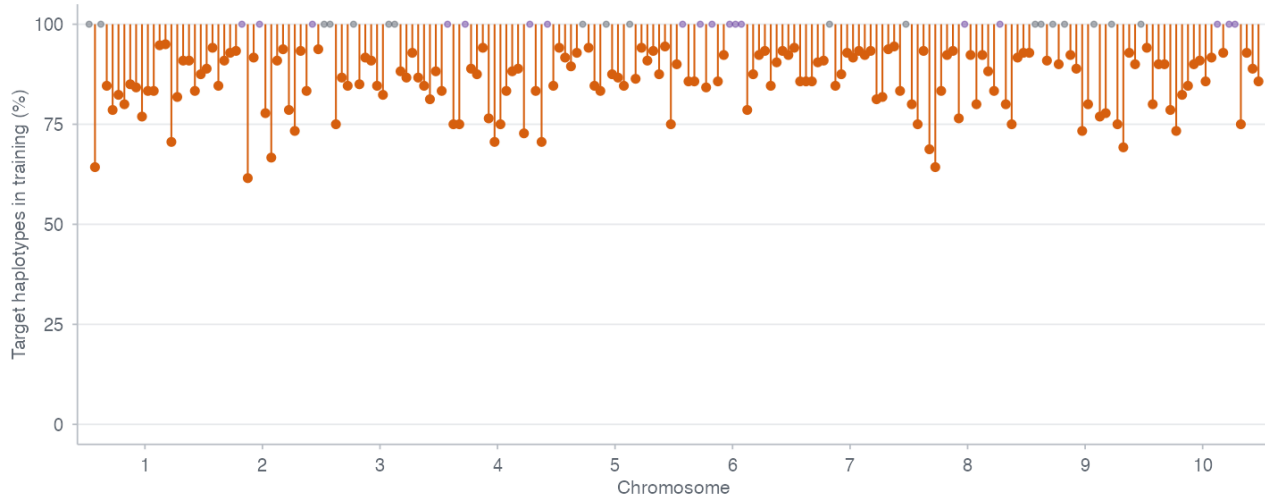


Figure 12: `tso_plot(plan, "haplotypes")`: the share of the targets' haplotypes in the training set, block by block; blocks that miss one are marked.

On four real panels with real phenotypes (120 paired runs, benchmark/HAPLOTYPES.md), designs under the haplotype kernel predicted no better than under the SNP kernel (predictive ability -0.006 to +0.001). The haplotype criterion covered 3-4 more percentage points of the targets' haplotypes in every run, at no average cost in prediction, though it gained 0.05 in maize and lost 0.02-0.04 in rice and sorghum. It is a tool for haplotype representation, not a better predictor.

9.5 Three ways to protect a major gene

`tso_data(major = )` accepts marker names, a scan, marker effects (a `tso_major_effects` result or the table itself), a haplotype object or a list of markers and haplotypes. What it does with them is set by two arguments:

- **major_model = "random"** (default): the relationship matrix becomes $\mathbf{K} = (1 - s)\mathbf{K}_{\text{poly}} + s\mathbf{K}_{\text{major}}$, with $s = \text{major_share}$ (the scan's or the effects' joint estimate, or 0.3). CDmean now values sampling the alleles of the locus in proportion to the variance they explain.
- **major_model = "fixed"**: the loci are fixed effects of the prediction model, as Bernardo (2014) recommends for genes of large effect, and `tso_design()` optimises the polygenic CDmean with the loci projected out exactly (Part 2).
- **min_carriers** (default 2, in either model): every allele class carried by the targets must appear at least this many times in the training set. It is a hard constraint that greedy, exchange and tabu search all honour; 0 switches it off.

your situation	major_model	min_carriers
a known gene with a diagnostic marker, effect large and stable	"fixed"	2 to 3
loci from a scan, effect size uncertain	"random" (share from the scan)	2 to 3
several moderate loci, or the trait is also polygenic	"random"	2
rare introgression present in only a few candidates	either	3 or more; check the Major line
you only want the report, not the constraint	"random"	0

For the breeder

The constraint is what does the work. In the demonstration below, re-weighting the kernel alone leaves the design where it was; the guarantee of carriers is what moves it. Weighting and fixed effects then decide *how much* a carrier is worth and how the prediction model will use it.

9.6 The introgression demonstration

The situation where it matters most: a major gene introgressed into the targets. A structured panel of 400 lines; a major allele carried by 16 of the 60 target lines (all from group G4) and by only 8 candidate lines, all in an unrelated group (G1). Nothing in the genome-wide relationships points the design to those eight lines.

```
set.seed(3)
intro <- tso_simulate_panel(n = 400, markers = 1500, groups = 4, fst = 0.25, seed = 8)
iid <- rownames(intro$dosage)
itg <- iid[intro$groups == "G4"][1:60] # this season's targets
intro$dosage[, "m700"] <- 0
intro$dosage[sample(itg, 16), "m700"] <- 2 # 16 targets carry it...
intro$dosage[sample(iid[intro$groups == "G1"], 8), "m700"] <- 2 # ...and 8 unrelated
  ↪ lines
maj <- function(...) tso_data(intro, major = "m700", verbose = FALSE, ...)
data_sets <- list(standard = tso_data(intro, verbose = FALSE),
  weighted_only = maj(major_share = 0.3, min_carriers = 0),
  random_3_carriers = maj(major_share = 0.3, min_carriers = 3),
  fixed_3_carriers = maj(major_model = "fixed", min_carriers = 3))
tp_std <- data_sets$standard; tp_maj <- data_sets$random_3_carriers
designs <- lapply(data_sets, function(d) tso_design(d, n = 30, target = itg, verbose =
  ↪ FALSE))
carriers <- sapply(designs, function(p) sum(intro$dosage[p$selected$id, "m700"] > 0))
carriers
```

```
#>          standard      weighted_only random_3_carriers fixed_3_carriers
#>              0              0              3              3
```

The standard design, and the design with the re-weighted kernel but no constraint, contain 0 carriers; both protected designs contain 3. The plan says so on its Major line:

```
designs$random_3_carriers
```

```
#> TSOpt plan · inbred · 30 lines of 340 candidates · 60 named targets
#> Criterion CDmean, exact (fixed intercept) (averaged over h2 = 0.2, 0.5, 0.8) Search
  ↪ greedy (robust CDmean: average over h2) (0.0 s)
#> Expected CDmean 0.566 (at h2 = 0.50) (exact, intercept projected: 0.544) vs random
  ↪ 0.326 ± 0.149 (better than 100% of 100 random sets)
#> Verdict GREEN optimisation pays off here: expected accuracy ~35.8% above random
#> Structure discrete groups (4 groups, silhouette 0.61) -> stratified allocation
  ↪ 0/0/0/30 (major-allele carriers take precedence)
#> Coverage nearest training relative >= 0.20 for 100% of targets · 0 targets
  ↪ unreachable (< 0.05)
#> Divide 0 candidates effectively unselectable (reliability < 0.06) · see
  ↪ tso_plot(plan, "divide")
#> Major 1 loci · 2 of 2 allele classes carried by targets have >= 3 training carriers
#> Alleles 100.0% of target alleles present in training (0 markers not covered) · 95.5%
  ↪ of target 5-marker haplotypes
#> Plot value dropping one plot costs 0.34% to 3.75% of the CDmean (median 0.48%) · see
  ↪ tso_plot(plan, "value")
#> Next tso_plot(plan) · tso_export(plan, "trial.csv") · tso_explain(plan)
```

```
tso_plot(designs$random_3_carriers, "major")
```

Major-locus alleles in the training set

1 loci · 2 of 2 allele classes protected

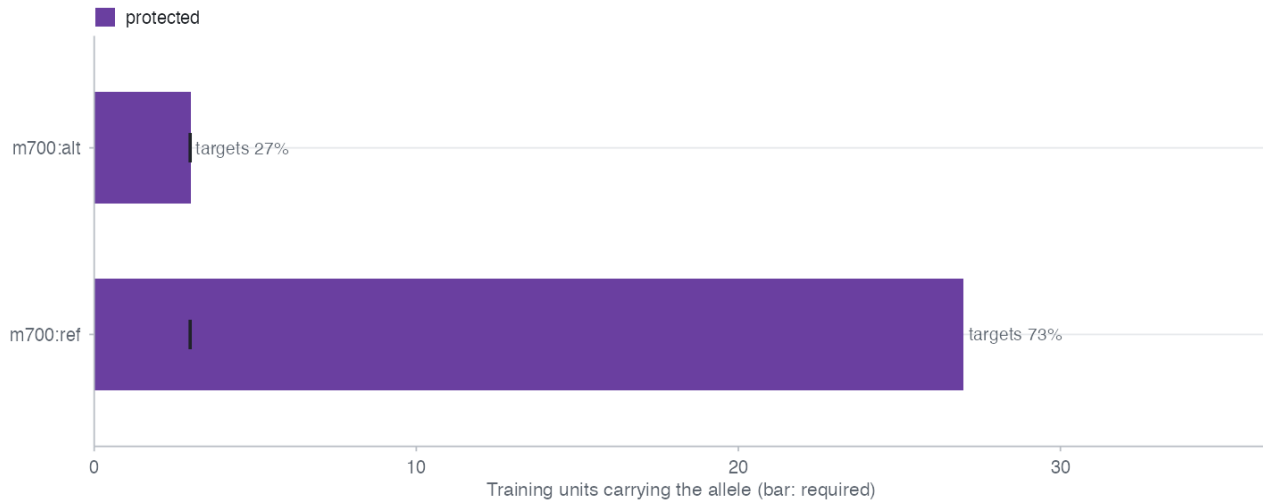


Figure 13: `tso_plot(plan, "major")`: carriers of each allele class of the protected locus among the targets and in the training set.

To see what the carriers are worth, ten traits are simulated in which the gene explains 40% of the genetic variance ($h^2 = 0.5$, 100 further QTL), and each training set is used the way a breeder would once the gene is known: GBLUP with the gene as a fixed effect. When the training set holds no carrier, the effect cannot be estimated and the gene drops out of the model.

```
Ki <- tso_kinship(tso_qc(intro, verbose = FALSE))
predict_major <- function(train, y, h2 = 0.5) { # GBLUP + the gene as a fixed effect
  S <- match(train, iid); x <- intro$dosage[, "m700"]
  V <- Ki[S, S] + diag((1 - h2) / h2, length(S)); Xm <- cbind(1, x[S])
  if (var(x[S]) == 0) Xm <- Xm[, 1, drop = FALSE] # no carrier: effect not estimable
  Vi <- solve(V); b <- solve(crossprod(Xm, Vi %*% Xm), crossprod(Xm, Vi %*% y[train]))
  setNames(as.vector(cbind(1, x)[, seq_len(ncol(Xm)), drop = FALSE] %*% b +
    Ki[, S] %*% (Vi %*% (y[train] - Xm %*% b))), iid)
}
acc_major <- function(plans, ev, n_traits) t(sapply(seq_len(n_traits), function(s) {
  tr <- tso_simulate_trait(intro, h2 = 0.5, n_qtl = 100, major = "m700",
    major_share = 0.4, seed = s)
  y <- setNames(tr$y, tr$id); gv <- setNames(tr$g, tr$id)
  sapply(plans, function(p) cor(predict_major(p$selected$id, y)[ev], gv[ev]))
}))
acc10 <- acc_major(designs, itg, 10)
data.frame(accuracy = round(colMeans(acc10), 3),
  better_than_standard = paste0(colSums(acc10 > acc10[, "standard"]), "/10"))
```

```
#>          accuracy better_than_standard
#> standard          0.222              0/10
#> weighted_only     0.221              5/10
#> random_3_carriers  0.891             10/10
#> fixed_3_carriers   0.894             10/10
```

Accuracy on the targets rises from 0.22 to 0.89 with three carriers in the training set, better in 10 of 10 traits. The weighted kernel without the constraint gains nothing, because it chose no carrier either; the random and fixed models with the constraint are practically tied.

The full benchmark. `benchmark/major_genes_benchmark.R` repeats the comparison of the standard and the protected design with 30 traits, targeted (the 60 G4 lines) and untargeted (the whole pool; the evaluation set is every line in neither training set). It takes a second, so it is run here in full:

```
bench_major <- do.call(rbind, lapply(c("targeted", "untargeted"), function(sc) {
  targ <- if (sc == "targeted") itg else NULL
  p0 <- tso_design(tp_std, n = 30, target = targ, verbose = FALSE)
  p1 <- tso_design(tp_maj, n = 30, target = targ, verbose = FALSE)
  ev <- if (is.null(targ)) setdiff(iid, c(p0$selected$id, p1$selected$id)) else itg
  a <- acc_major(list(p0, p1), ev, 30)
  ncar <- function(p) sum(intro$dosage[p$selected$id, "m700"] > 0)
  data.frame(scenario = sc, carriers_std = ncar(p0), carriers_prot = ncar(p1),
    accuracy_std = round(mean(a[, 1]), 3), accuracy_prot = round(mean(a[, 2]), 3),
    prot_better = paste0(sum(a[, 2] > a[, 1]), "/30"))
}))
pn_kable(bench_major, row.names = FALSE,
  col.names = c("scenario", "carriers, std", "carriers, protected", "accuracy, std",
    "accuracy, protected", "protected better"))
```

scenario	carriers, std	carriers, protected	accuracy, std	accuracy, protected	protected better
targeted	0	3	0.232	0.872	30/30
untargeted	0	4	0.307	0.694	30/30

Key point

Standard CDmean put no carrier of the major allele in the training set; the protected design put 3 (targeted) and 4 (untargeted). Accuracy rose from 0.232 to 0.872 on the targets and from 0.307 to 0.694 untargeted, better in 30/30 and 30/30 traits. No other change to a design in this tutorial comes close. The price is small: three of thirty plots.

9.7 Allele and haplotype coverage in every plan

Even without `major =`, every plan checks the genome: which marker alleles carried by the targets are absent from the training set, and what share of the targets' 5-marker haplotypes (tiled genome-wide) also occur in it. The `Alleles` line of `print(plan)` summarises it and `plan$alleles` holds the detail:

```
plan_cov <- tso_design(tp, n = 60, target = tp$units$id[241:300], verbose = FALSE)
al <- plan_cov$alleles
data.frame(alleles_covered = al$frac, markers_not_covered = al$n_missing_markers,
  haplotypes_covered = round(al$haplotypes$frac, 3), blocks =
    ↪ al$haplotypes$n_blocks)
```

```
#> alleles_covered markers_not_covered haplotypes_covered blocks
#> 1 1 0 0.915 195
```

A missing allele is a warning sign only if it matters for the trait; the list (`plan$alleles$missing`) is what to compare with known genes and scan results.

9.8 Weighted kinship

The third route puts the scan into the kernel itself: `tso_kinship(weights = )` up-weights markers by their evidence, for example $w_m = \max(1, -\log_{10} p_m)$ (Zhang et al. 2014), and the weighted matrix is given to `tso_data(kinship = )`:

```

w_scan <- setNames(pmax(1, scan$table$log10p), scan$table$marker)
geno_p <- tso_gc(panel, verbose = FALSE)
K_w <- tso_kinship(geno_p, weights = w_scan)
K_u <- tso_kinship(geno_p)
up <- upper.tri(K_w)
data.frame(markers_upweighted = sum(w_scan > 1), cor_with_unweighted = cor(K_w[up],
  ↪ K_u[up]))

#>   markers_upweighted cor_with_unweighted
#> 1                88                0.9973

tp_w <- tso_data(kinship = K_w, verbose = FALSE)

```

With one locus among a thousand markers, the weighted matrix is almost the unweighted one; the weighting pays with several loci of moderate effect. For one or a few major genes, major = with its carrier guarantee is the sharper tool.

10 More decision tools

`tso_design()` answers “which n units”. The decisions around it come with their own questions: how large should n be, will this set actually find the best lines, what does accuracy cost in diversity, which parents to cross once the trial is in, and what to do when the panel has twenty thousand lines. Each has one function, and each works on the same `tso_data` object. The examples use the tutorial panel and the 60 targets of Part 6.

```
targets60 <- tp$units$id[241:300]
plan_t60 <- tso_design(tp, n = 60, target = targets60, verbose = FALSE)
```

10.1 `tso_sample_size()`: how many lines do I need?

The *operating curve* of a training set is the expected accuracy at every size, for an optimised design and for random sampling, relative to phenotyping every candidate. One greedy pass of the engine adds the candidates one at a time, which gives the optimised CDmean at every size in a single run: the curve is exact for the greedy design, not a model fitted to noisy maxima of random sets. Expected accuracy is $\sqrt{CD}$ and the relative efficiency is $RE(n) = \sqrt{CD(n)} / \sqrt{CD_{all}}$. The logistic curve of TSDFGS (Ou and Liao 2019) is fitted for smoothing and reporting only; the answers never depend on it.

```
ss_t <- tso_sample_size(tp, target = targets60)
ss_t
```

```
#> TSOpt sample size · 240 candidates · 60 targets · averaged over h2 = 0.2, 0.5, 0.8
#> Reference      expected accuracy 0.470 (phenotyping all 240)
#> → Phenotype 137 lines to reach 95% of the accuracy of phenotyping all 240; random
  ↳ sampling would need 210.
#> Target RE      optimised  random  plots saved
#>          90%        70       183       113
#>          95%       137       210       73
#>          99%       213       234       21
#> Fit           logistic acc(n) = 0.442 / (1 + exp(0.81 - 0.2026 n)), RMSE 0.0222
  ↳ (smoothing only)
#> Method         exact greedy curve (optimised) · mean of 20 random sets per size (random)

ss_u <- tso_sample_size(tp) # untargeted, for comparison
rbind(targeted = ss_t$n_for, untargeted = ss_u$n_for)
```

```
#>          0.90 0.95 0.99
#> targeted    70 137 213
#> untargeted 167 223 282
```

```
tso_plot(ss_t)
```

Phenotype 137 to reach 95% of the accuracy of all 240 candidates

Exact greedy CDmean curve (averaged over h^2) vs mean of 20 random sets

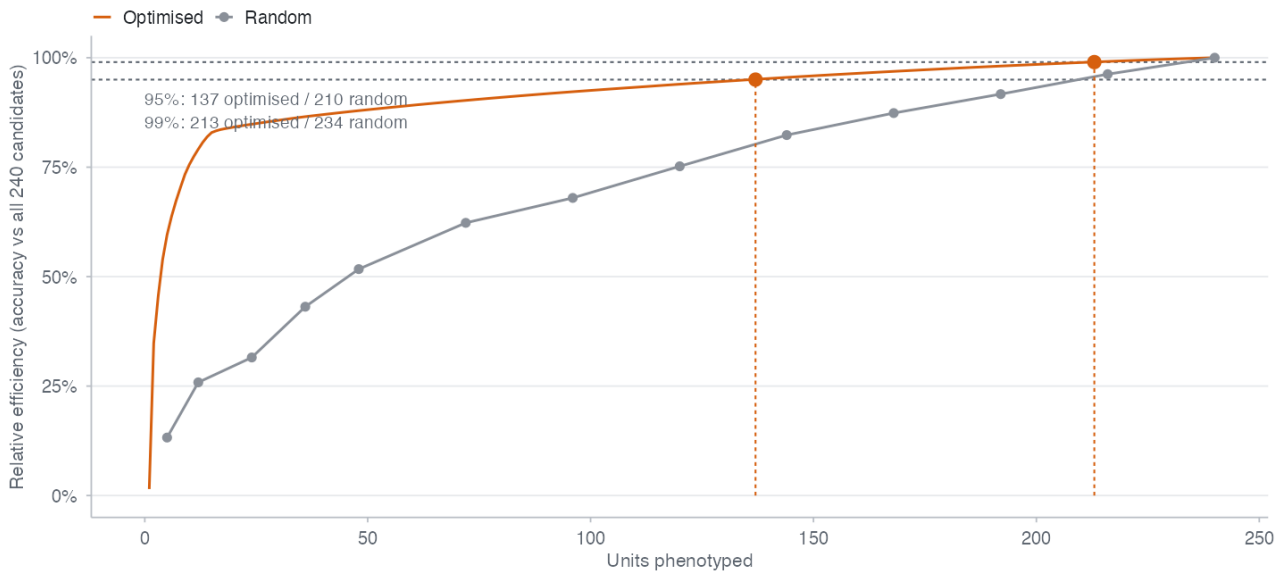


Figure 14: `tso_plot(sample size)`: expected accuracy relative to phenotyping every candidate, optimised against random, with the sizes for 90, 95 and 99%.

For the breeder

Read two numbers. To predict these 60 targets at 95% of the accuracy of phenotyping all 240 candidates takes 137 optimised lines against 210 random ones: the design saves 73 plots. Untargeted, where every line is both candidate and target, the saving at 95% is only 6 plots: a set that large covers the panel whichever way it is drawn. Design pays most when the budget is tight and the target is specific.

10.2 `tso_topk()`: will this set find the best lines?

Accuracy is an average; the breeder selects the top 10%. `tso_topk()` simulates the genomic model behind the design ($\mathbf{g} \sim N(\mathbf{0}, \mathbf{K})$), phenotypes with the design's h^2), fits GBLUP with a fixed intercept on the training set, and records, for every draw, the share of the predicted top k that is truly top k , $\text{NDCG}@k$, the realised gain and the correlation, for the plan and for random training sets sharing the same simulated values.

```
tk <- tso_topk(plan_t60, top = 0.1, n_sim = 300)
tk
```

```
#> TSOpt top-k · 60 training units → rank 60 units · select top 10% (k = 6) · h2 = 0.50
#>
#> optimised [90% int.]    random [90% int.]
#> Top-k overlap      0.271 [0.000, 0.500]    0.186 [0.000, 0.500]
#> NDCG@6            0.735 [0.516, 0.910]    0.660 [0.431, 0.855]
#> Realised gain (SD) 0.715 [-0.044, 1.419]    0.407 [-0.414, 1.166]
#> Correlation        0.422 [0.125, 0.676]    0.241 [-0.115, 0.544]
#> → Of the 6 lines you select, on average 1.6 are truly in the top 6 (random training
  ↪ sets: 1.1); chance alone gives 0.6.
#> Simulation          300 draws (Cholesky) · 20 random sets x 60 draws
```

```
tso_plot(tk)
```

Finding the top 10%: 27% overlap vs 19% for random sets

60 training units ranking 60 units · $h^2 = 0.50$ · 300 simulated traits

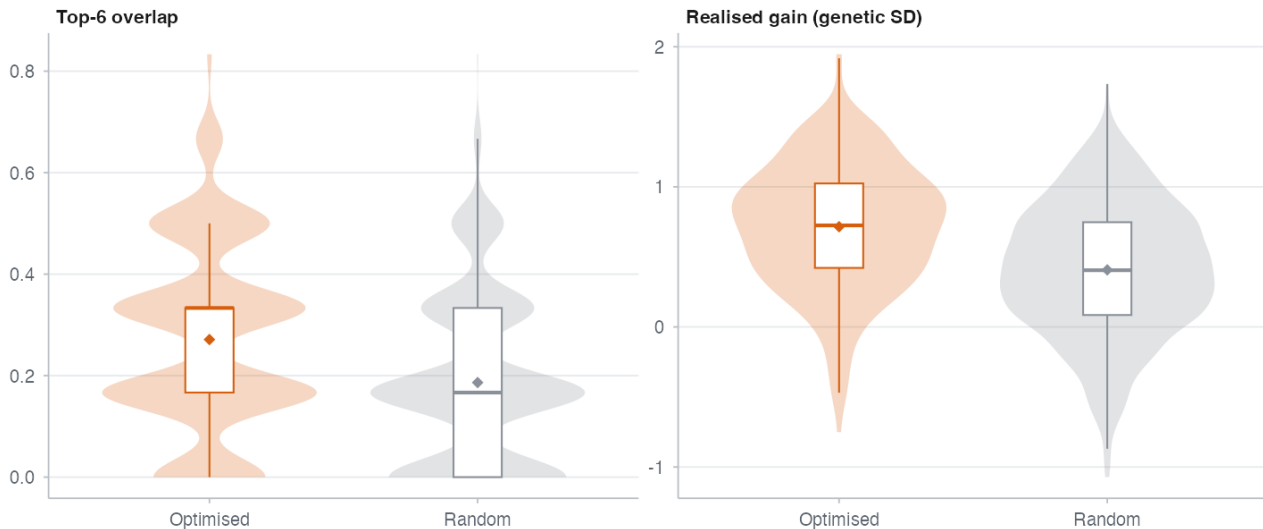


Figure 15: `tso_plot(top-k)`: distribution of each selection metric over simulated traits, plan against random training sets.

`tk$units` gives, for each target, how often it was selected and the probability that it is truly among the best when selected: the honest spread around one list of GEBVs.

10.3 `tso_pareto()`: accuracy against diversity, cost or anything else

Some goals conflict. The most informative training set may also be the most closely related one, or the most expensive. `tso_pareto()` searches for all objectives at once with NSGA-II (Deb et al. 2002) and returns the *Pareto front*: every set that no other set beats on all objectives together. Built-in objectives are "cdmean", "cdmin", "coancestry", "cost", "dopt", "coverage" and "gain"; any function `f(ids, data)` can be added, with `directions =` for the ones to minimise. The first population is seeded with the greedy CDmean set and greedy solutions of weighted sums, and an archive keeps every non-dominated set ever seen, so the best CDmean on the front is never below the greedy one.

```
fr <- tso_pareto(tp, n = 40, objectives = c("cdmean", "coancestry"), target = targets60,
  pop = 60, generations = 60, verbose = FALSE)
fr
```

```
#> TSOpt Pareto front · 51 non-dominated training sets of 40 from 240 candidates · 60
  ↳ named targets
#> cdmean      [max]      0.131 – 0.169 (CDmean (exact))
#> coancestry  [min]     -0.0189 – -0.00588 (Mean coancestry)
#> Knee       set 39: cdmean 0.158 · coancestry -0.0167 (farthest from the line between
  ↳ the extremes)
#> Greedy seed cdmean 0.169 · coancestry -0.00588 (on the front)
#> Search      NSGA-II, pop 60, 60 generations, 3,663 sets evaluated (0.9 s) ·
  ↳ hypervolume 0.986
#> Next        plot(front) · tso_pareto_pick(front) · tso_pareto_pick(front,
  ↳ "best_cdmean")
```

```
knee <- tso_pareto_pick(fr, "knee") # a vector of unit IDs
attr(knee, "values")
```

```
#>      cdmean coancestry
```

```
#> 0.15764 -0.01675
```

```
tso_plot(fr)
```

Pareto front: CDmean (exact) vs Mean coancestry

51 non-dominated sets of 40 (purple) · grey: final population

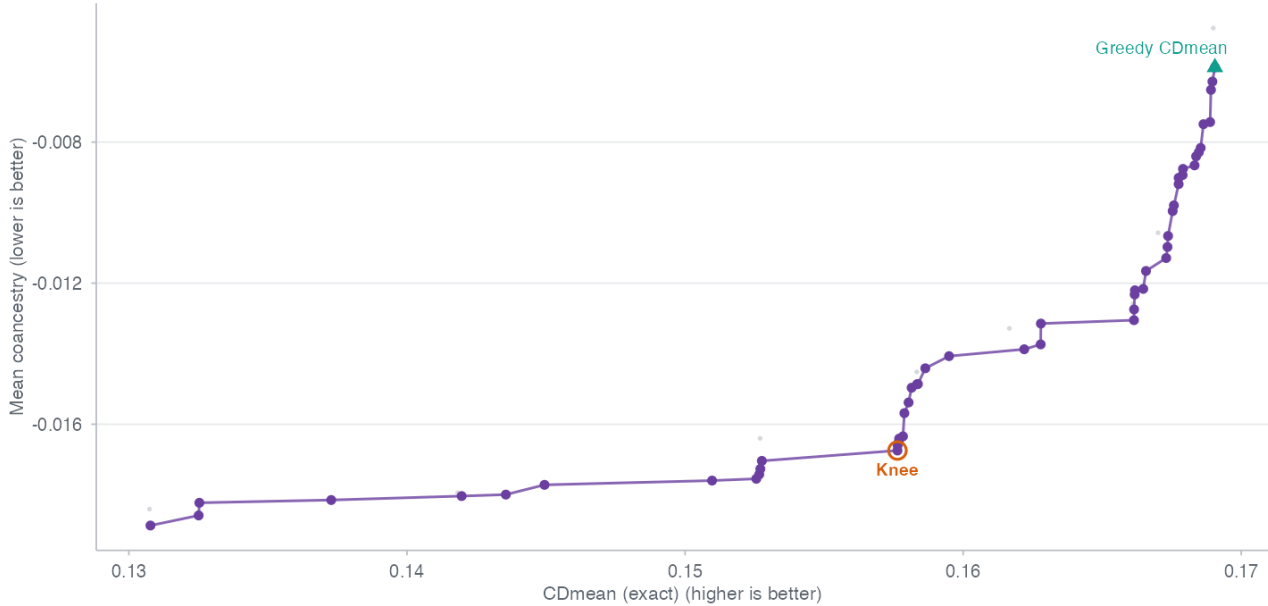


Figure 16: `tso_plot(front)`: the trade-off between CDmean and mean coancestry of the training set; the knee is marked.

The knee (set 39) gives up 0.011 of CDmean for a coancestry of -0.0167 instead of -0.0059. `tso_pareto_pick(fr, "best_cdmean")` takes the most accurate end of the front, and `weights = a` is a weighted compromise; the hypervolume in `fr$spec` measures how much of the objective space the front covers.

10.4 `tso_contribution()`: which parents, and how many crosses each

After the trial, the GEBVs are in and the next question is which lines to cross. Truncation selection on GEBV maximises the gain of one generation and spends diversity fast. Optimum contribution selection (Meuwissen 1997) chooses the contributions $\mathbf{c}$ of the candidates to maximise $\mathbf{c}'\mathbf{b}$ subject to $\mathbf{c}'\mathbf{K}\mathbf{c}/2 \leq C$, $\mathbf{c} \geq 0$, $\mathbf{1}'\mathbf{c} = 1$, where the group coancestry $\mathbf{c}'\mathbf{K}\mathbf{c}/2$ is the expected inbreeding of the offspring. The limit C comes from an accepted rate of inbreeding, $C_t = C_{t-1} + \Delta F(1 - C_{t-1})$ (Grundy et al. 1998). TSOpt solves the quadratic programme exactly by a primal active-set method, with the coancestry constraint binding to 10^{-8} ; `sex =` makes each sex contribute one half, `n_matings =` rounds the contributions to matings, and `n_max` caps the number of parents (heuristically).

```
ev60 <- tso_evaluate(plan, y) # GEBVs of the 240 unphenotyped lines
ebv <- setNames(ev60$predictions$predicted, ev60$predictions$id)
Kp <- tp$K_lines[names(ebv), names(ebv)]
ocs <- tso_contribution(ebv, Kp, delta_F = 0.01, n_matings = 40)
ocs
```

```
#> TSOpt optimum contributions · 240 candidates ·  $\Delta F \leq 0.01 \rightarrow$  group coancestry  $\leq 0.01015$ 
#> Optimum 108 parents · mean EBV of offspring 0.349 · group coancestry 0.0101 ·
↪ expected  $\Delta F$  0.0100
```

```
#> Truncation      top 108 by EBV, equal contributions · mean EBV 0.349 · group coancestry
↳ 0.0149 · ΔF 0.0147
#> Status          coancestry limit binding · lambda = 10.53
#> Parents         L076 3.1% (2), L201 3.1% (2), L102 2.8% (2), L230 2.5% (2), L092 2.5%
↳ (2), L114 2.5% (2), L280 2.3% (2), L285 2.2% (2), L153 2.2% (2), L170 2.1% (2), ...
↳ (98 more)
```

```
ocs_curve <- t(sapply(c(0.005, 0.01, 0.02, 0.05), function(d) {
  o <- tso_contribution(ebv, Kp, delta_F = d)
  c(delta_F = d, parents = sum(o$contributions$selected),
    mean_ebv = round(o$mean_ebv, 3), coancestry = round(o$coancestry, 4))
}))
ocs_curve
```

```
#>      delta_F parents mean_ebv coancestry
#> [1,] 0.005     138   0.284   0.0052
#> [2,] 0.010     108   0.349   0.0101
#> [3,] 0.020      70   0.429   0.0201
#> [4,] 0.050      19   0.522   0.0501
```

```
tso_plot(ocs)
```

Optimum contributions: 108 parents, mean EBV 0.35 vs 0.35 by truncation

Group coancestry 0.0101 (limit 0.01015) vs 0.0149 for the top 108 with equal contributions

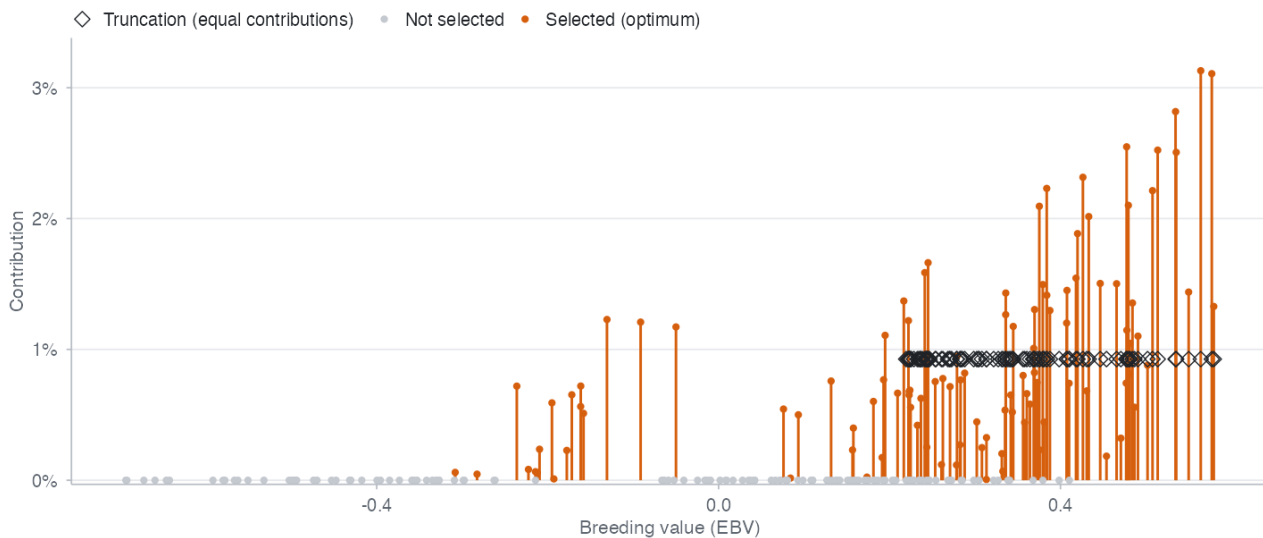


Figure 17: `tso_plot(contributions)`: optimum contribution of each candidate against its GEV.

At $\Delta F = 0.01$ (an effective population size of 50) the optimum uses 108 parents, mean GEV 0.3494 at a group coancestry of 0.0101. Truncation selection of the same number of top lines with equal contributions gives a mean GEV of 0.3485 at a coancestry of 0.0149, a rate of inbreeding of 0.0147 instead of the agreed 0.01. The curve shows the price of diversity: every step up in ΔF buys GEV with fewer parents.

10.5 `tso_design_large()`: panels of tens of thousands

`tso_design()` holds the $N \times N$ relationship matrix, which is out of reach beyond about 15,000 lines (20,000 lines take 3.2 GB for **K** alone). `tso_design_large()` works in marker space instead. A rank- r feature matrix **F** with $\mathbf{K} \approx \mathbf{F}\mathbf{F}' + \text{diag}(\mathbf{e})$ comes from a randomised truncated SVD of the centred genotypes (Halko et al. 2011); what the features miss is kept as line-specific variance. The engine keeps the $r \times r$ posterior covariance of the feature effects and updates it by one

rank-one step per line; the intercept gets a flat prior as an extra all-ones feature. One greedy step costs $O(Nr)$ per heritability, independent of the number of targets.

```
big <- tso_simulate_panel(n = 3000, markers = 1500, seed = 5)
t_big <- system.time(
  plan_big <- tso_design_large(big, n = 200, rank = 200, verbose = FALSE))["elapsed"]
plan_big

#> TSOpt plan (large panel) · 200 lines of 3,000 candidates · 3,000 lines x 1,435 markers
#> ↳ · untargeted (whole pool)
#> Features randomised SVD (1 power iteration), rank 200, 78.5% of trace(K) captured (5
#> ↳ MB, no N x N matrix)
#> Criterion CDmean, exact (fixed intercept) (averaged over h2 = 0.2, 0.5, 0.8) on 3,000
#> ↳ contrasts Search greedy (robust CDmean: average over h2) (2.7 s)
#> Expected CDmean 0.472 (at h2 = 0.50) (exact, intercept projected: 0.472) vs random
#> ↳ 0.437 ± 0.001 (better than 100% of 20 random sets)
#> Verdict AMBER modest gain (~3.9% accuracy over random); worth using, but do not
#> ↳ expect much
#> Coverage nearest training relative >= 0.20 for 100% of targets · 0 targets
#> ↳ unreachable (< 0.05)
#> Next plot(plan) · write.csv(plan$selected, "trial.csv")
```

3,000 lines, 200 chosen in 2.7 s, with 78.5% of trace(**K**) captured by 200 features; raise rank if the share is below 70%. On a 20,000-line panel the package authors measured 1,000 lines chosen in about 39 s. With exact features (rank at least the number of markers) the low-rank engine *is* the dense engine:

```
g600 <- tso_qc(tso_simulate_panel(n = 600, markers = 400, seed = 9), verbose = FALSE)
K600 <- tso_kinship(g600, ridge = 0)
pl <- tso_design_large(g600, n = 40, h2 = 0.5, rank = ncol(g600$dosage), n_random = 0,
  verbose = FALSE) # exact features
pd <- tso_design(tso_data(kinship = K600, verbose = FALSE), n = 40, h2 = 0.5,
  search = "greedy", stratify = "none", n_random = 0, verbose = FALSE)
data.frame(same_set = identical(pl$selected$id, pd$selected$id),
  max_trace_difference = signif(max(abs(pl$trace - pd$trace)), 2))
```

```
#> same_set max_trace_difference
#> 1 TRUE 1.8e-11
```

10.6 Multi-start search

restarts = (Part 4) reruns the exchange or tabu search from perturbed copies of the best set:

```
tabu45 <- function(...) tso_design(tp, n = 45, h2 = 0.5, search = "tabu", stratify =
  ↳ "none",
  n_random = 0, verbose = FALSE, ...)
p_single <- tabu45()
p_multi <- tabu45(restarts = 4)
rs <- p_multi$spec$restarts
data.frame(single = p_single$value$cdmean_exact, restarts_4 = p_multi$value$cdmean_exact,
  runs = rs$runs, improved = rs$improved)
```

```
#> single restarts_4 runs improved
#> 1 0.2974 0.2979 4 TRUE
```

As with tabu search itself, a higher criterion at one assumed h^2 is worth having when the criterion is the deliverable; the robust default does not need it.

10.7 Safe custom criteria

STPGA minimises every criterion, and a criterion written to be maximised is then quietly optimised the wrong way: the search returns the *worst* sets, and published analyses have been caught by it. TSOpt maximises by default, takes `direction = "minimize"`, and probes a user function before optimising it: if a set of near-duplicate relatives scores better than every random set, the function probably measures error or similarity and should be minimised.

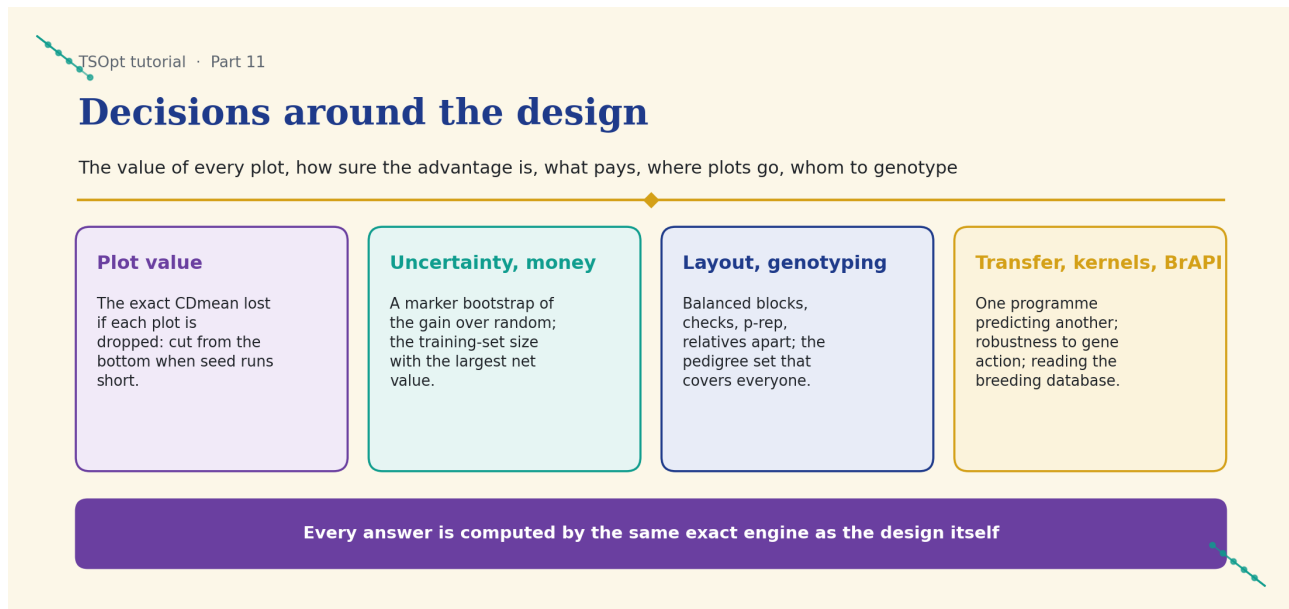
```
similarity <- function(ids, data) mean(data$K[ids, ids]) # to be MINIMISED
msg <- tryCatch(tso_design(tp, n = 30, criterion = similarity, iterations = 200,
                          n_random = 0, verbose = FALSE),
               warning = function(w) conditionMessage(w))
cat(strwrap(msg, 90), sep = "\n")
```

```
#> Your criterion scores a set of near-duplicate relatives higher than every random set.
#> Criteria that measure error, PEV or similarity must be minimised: if that is the case,
#> set direction = "minimize".
```

```
p_div <- tso_design(tp, n = 30, criterion = similarity, direction = "minimize",
                   iterations = 200, n_random = 0, verbose = FALSE)
set.seed(1)
c(designed = similarity(p_div$selected$id, tp),
  random = mean(replicate(20, similarity(sample(tp$units$id, 30), tp))))
```

```
#> designed    random
#> 0.01376 0.03209
```

11 Decisions around the design



A plan answers “which units”. Before and after it come the questions a programme actually argues about. Seed of three lines failed: which plots do we cut? Is the advantage over random real, or an artefact of this SNP chip? How many plots pay for themselves? Where does each plot go in the field? Which individuals should be genotyped in the first place? Will training in our partner’s programme predict ours? What if the trait is not additive? And can all of this read straight from the breeding database? Each has one function (or one argument) in v0.5, and each works on the objects of the earlier parts: `plan` (60 lines, untargeted, Part 6) and `plan_t60` (60 lines for the 60 targets, Part 10).

11.1 The value of each plot

Every plan carries, in `plan$selected$value`, the exact CDmean lost if that plot were dropped, and in `value_pct` the same as a percentage of the plan’s CDmean. The engine gets it cheaply: removing unit j from the final state is the rank-one *downdate* $\mathbf{A} \leftarrow \mathbf{A} + \mathbf{a}_j \mathbf{a}_j' / (\lambda - \mathbf{a}_j \mathbf{a}_j')$ on the flat-prior state of the search (Part 2), one operation for all plots at once, exactly as the tabu search ranks its members (Part 4). The print line `Plot value` summarises the range.

```
pv <- plan_t60$selected[order(-plan_t60$selected$value),
                             c("id", "group", "reliability", "value", "value_pct")]
rbind(head(pv, 4), tail(pv, 3))
```

```
#>      id group reliability      value value_pct
#> 1  L234   G4      0.5758 0.0180694    10.2052
#> 3  L235   G4      0.5515 0.0124319     7.0213
#> 4  L238   G4      0.5435 0.0114603     6.4726
#> 2  L226   G4      0.5670 0.0101727     5.7453
#> 60 L089   G2      0.5554 0.0003748     0.2117
#> 54 L032   G1      0.5571 0.0003628     0.2049
#> 44 L040   G1      0.5952 0.0003496     0.1975
```

```
tso_plot(plan_t60, "value")
```

The value of each plot

the 20 most and 20 least valuable of 60 plots · drop from the bottom when seed or land runs short

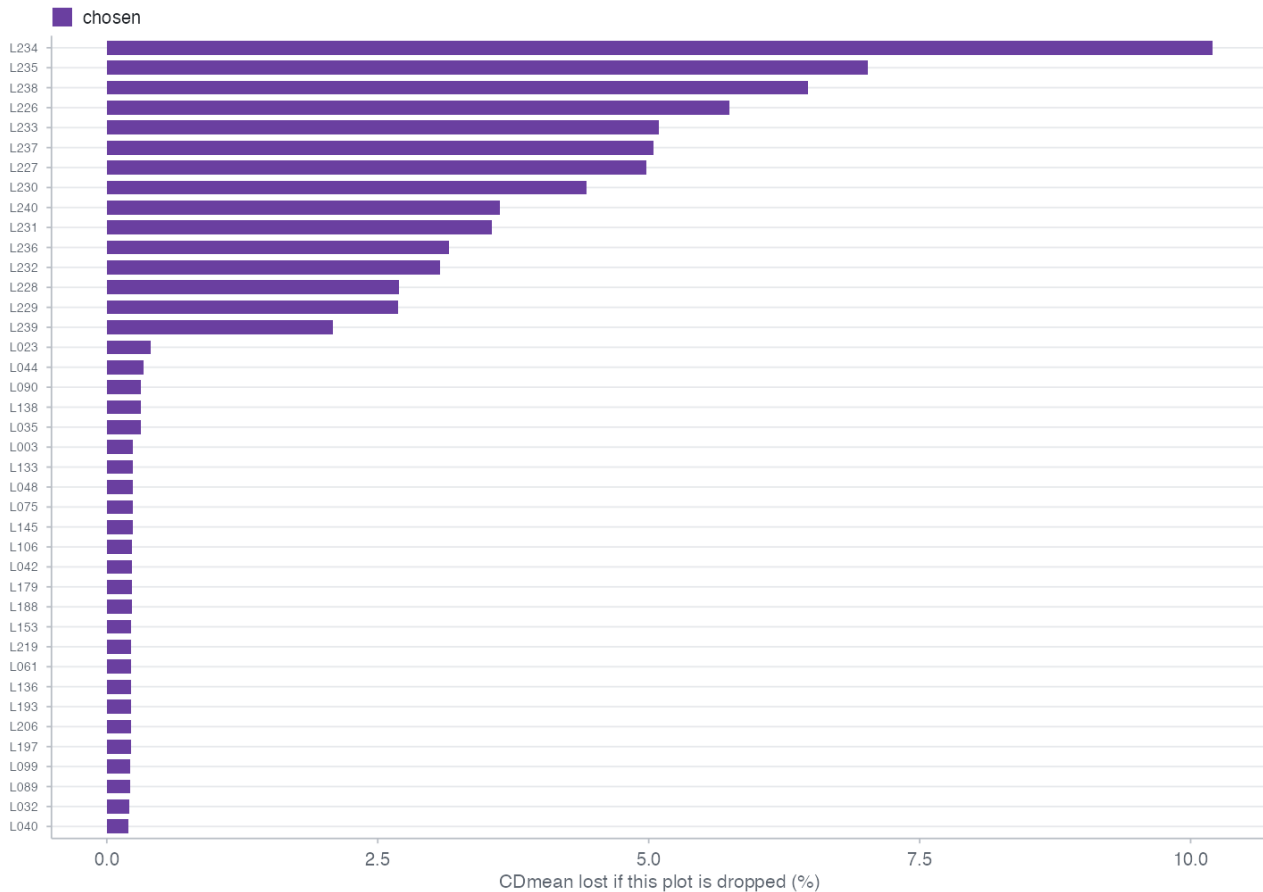


Figure 18: `tso_plot(plan, "value")`: CDmean lost if each plot is dropped, the 20 most and 20 least valuable plots of the targeted plan.

```
cd_drop <- function(drop) # CDmean at h2 = 0.5 without the dropped plots
  tso_score(tp, train = setdiff(plan_t60$selected$id, drop), target = targets60,
    h2 = 0.5, criteria = "cdmean_exact")["cdmean_exact"]
full_cd <- cd_drop(character(0))
brute <- vapply(pv$id[1:5], function(i) full_cd - cd_drop(i), numeric(1))
rbind(value = pv$value[1:5], brute_force = brute)
```

```
#>           L234    L235    L238    L226    L233
#> value      0.01807 0.01243 0.01146 0.01017 0.009013
#> brute_force 0.01807 0.01243 0.01146 0.01017 0.009013
```

```
set.seed(3)
drop6 <- c(full = full_cd, least_valuable = cd_drop(tail(pv$id, 6)),
  random_6 = mean(replicate(20, cd_drop(sample(pv$id, 6)))),
  most_valuable = cd_drop(head(pv$id, 6)))
round(drop6, 4)
```

```
#>           full least_valuable      random_6 most_valuable
#>           0.1771           0.1748           0.1584           0.1043
```

Verify by hand

The reported value of a plot is the CDmean at $h^2 = 0.5$ of the set with the plot minus the set without it, computed from scratch by `tso_score()`: they agree to 2×10^{-8} (the test suite checks it to 10^{-8}).

The values are far from equal. In the targeted plan the most valuable plot is worth 10.2% of the CDmean and the least 0.20%. The top of the list are the lines closest to the targets (15 of the 15 most valuable are from G4, the targets' own group); the bottom are lines whose information is duplicated by relatives in the set. When six plots must go, dropping the six least valuable keeps CDmean 0.1748, six random ones 0.1584 on average, and the six most valuable 0.1043. Values are not additive (two plots of close relatives are each cheap to lose, but not both), so after cutting a few, re-run the design with `must_include` = the survivors, or `exclude` = the lost lines, to re-rank.

For the breeder

Print the value column on the field list. When seed or land runs short, cut from the bottom; when a line fails that is near the top, it is worth replacing, and `tso_design(tp, n = 60, must_include = survivors, exclude = failed)` finds the best replacement.

11.2 `tso_uncertainty()`: is the advantage real?

The verdict line of a plan says how much better than random the set is *on this relationship matrix*. The matrix is an estimate from a finite set of markers; another chip would give another matrix. `tso_uncertainty()` bootstraps the markers (Efron and Tibshirani 1993): for each of `B` replicates it resamples the marker columns with replacement, rebuilds **K** with the base allele frequencies, and recomputes the plan's exact CDmean and that of `n_random` random sets of the same size. The set is *not* re-optimised: it is the plan you would plant, scored under matrices it was not chosen for, so the winner's curse is part of what is measured. Intervals are percentile intervals re-centred on the all-marker estimate (a resampled matrix is slightly noisier than the original, which shifts every CDmean by a similar amount; the shift is the bias column).

```
unc <- tso_uncertainty(plan, B = 50, n_random = 20)
unc
```

```
#> TSOpt uncertainty · marker bootstrap, 50 replicates of 958 markers · 20 random sets
  ↳ each (1.4 s)
#>      estimate [lower, upper] (90% intervals; estimate on all markers)
#> Plan CDmean   0.345  [0.342, 0.350]
#> Random mean   0.311  [0.307, 0.315]
#> Gain (CD)     11.0%  [10.3%, 11.6%]
#> Gain (acc.)   5.4%   [5.0%, 5.6%] (plan reported 4.9%)
#> Beats random  100% of replicates
#> Verdict       ROBUST: the 90% interval of the accuracy gain lies above 2%: the
  ↳ advantage holds whichever markers are used
```

```
tso_plot(unc)
```

Advantage over random: robust

90% interval 5.0% to 5.6% · plan beats random in 100% of 50 marker bootstraps



Figure 19: `tso_plot(uncertainty)`: the plan's CDmean and the random mean over marker-bootstrap replicates, and the accuracy gain with its interval.

The verdict is *robust* when the lower bound of the accuracy gain is above 2%, *small* when the interval is positive but starts below 2%, *uncertain* when it contains 0 and *not supported* when it lies below. Here the gain in accuracy is 5.4% [5.0, 5.6]: ROBUST. The all-marker estimate differs slightly from the plan's own verdict line (4.9%) because the bootstrap scores the exact, intercept-projected CDmean at the median heritability against its own random sets. The function supports line-level plans built from marker genotypes (inbred, clonal, outcross, genebank); hybrid, multi-environment and multi-trait plans give an error.

11.3 `tso_economics()`: how many plots pay off?

`tso_sample_size()` (Part 10) gives accuracy by size; `tso_economics()` turns it into money. With accuracy $r(n) = \sqrt{\text{CDmean}(n)}$ and the best fraction p selected, the response per cycle is the breeder's equation (Falconer and Mackay 1996)

$$R(n) = i(p) r(n) \sigma_g, \quad i(p) = \phi(\Phi^{-1}(1 - p)) / p,$$

its value $\text{horizon} \times \text{value_per_sd} \times R(n)$, its cost $\text{fixed_cost} + \text{cost_per_plot} \times n$. The optimum is the size with the largest net value: there the last plot returned at least its cost and the next would return less. `value_per_sd` is what one genetic standard deviation of gain in the selected material is worth, for example extra revenue per hectare times the hectares sown with it.

```
ec <- tso_economics(tp, cost_per_plot = 40, value_per_sd = 20000, n_random = 10)
ec
```

```
#> TSOpt economics · 300 candidates · select top 10% (i = 1.75) · averaged over h2 = 0.2,
#> ↪ 0.5, 0.8
#> Prices      40 per plot · fixed 0 · 20,000 per genetic SD · 1 cycle(s)
#> → Phenotype 125 lines: net value 19,075; each extra plot now returns 39.9 against a
#> ↪ cost of 40 (the last one returned 40.4)
#>
#>          n      accuracy  value    net    break-even
#> optimised 125      0.686  24,075  19,075          1
#> random   134      0.684  24,024  18,664          1
```

```
#> Advantage   optimised design is worth 412 more at its optimum and uses 9 fewer plots
#> Method      exact greedy accuracy curve (optimised) · mean of 10 random sets at 30
↪ sizes, interpolated (random)

opt_n <- supply(c(`10,000` = 10000, `20,000` = 20000, `40,000` = 40000), function(v)
  if (v == 20000) ec$optimum$opt$n else
  tso_economics(tp, cost_per_plot = 40, value_per_sd = v, n_random = 5)$optimum$opt$n)
opt_n
# the optimum size for three prices of gain

#> 10,000 20,000 40,000
#>    65    125    242

tso_plot(ec)
```

Phenotype 125 lines: net value 19,075

Dashed segments: net value at each design's optimum (optimised 125, random 134) · advantage 412

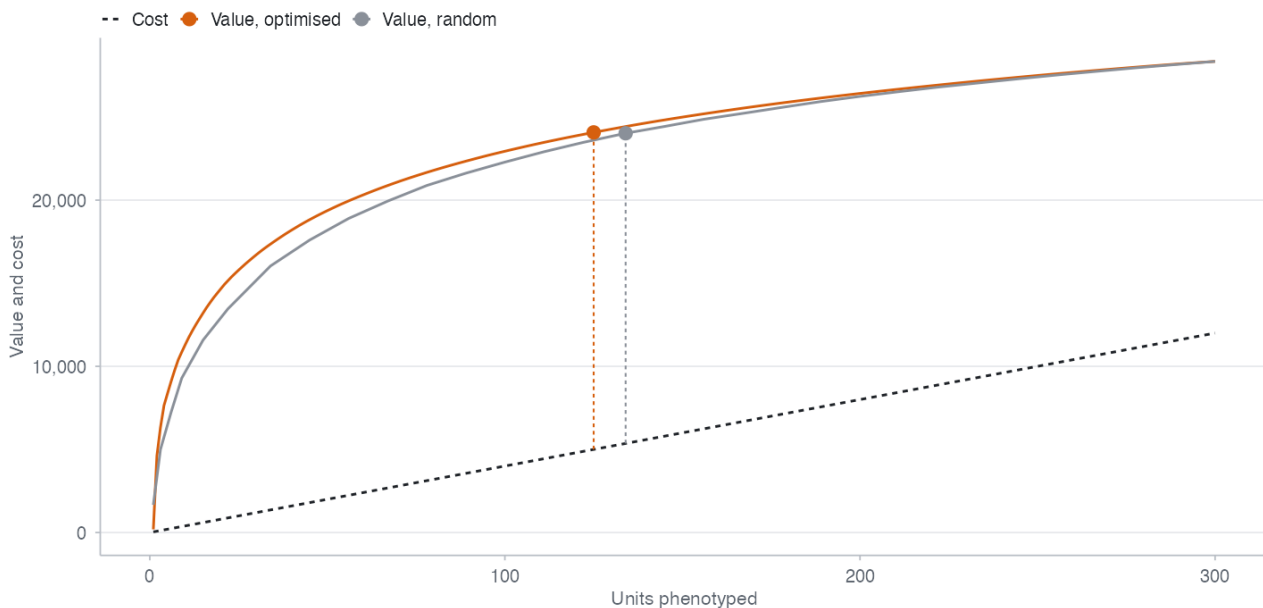


Figure 20: `tso_plot(economics)`: value, cost and net value by training-set size, optimised against random, with the optimum marked.

At 40 per plot and 20,000 per genetic SD the optimised design pays best at 125 lines (net 19,075), random sampling at 134; the design is worth 412 more and uses 9 fewer plots. The optimum moves with the price of gain far more than with the design: 65 lines when gain is worth half as much, 242 when it is worth twice as much. The net-value curve is flat near its top, so any size in the flat region is defensible; `fixed_cost` shifts the net value but never the optimum.

11.4 `tso_layout()`: lay out the field

`tso_design()` decides *which* units; `tso_layout()` decides *where* each plot goes, on a rows × cols grid numbered in serpentine order, with blocks as consecutive runs of that order (bands of rows, as planters and harvesters work). It

- deals the lines round-robin to the blocks within each genetic stratum, so every block gets the same genetic mix and block effects are not confounded with groups;
- spreads the replicated checks systematically through every block, never touching another check when there is room;

- gives a second plot (a partially replicated, p-rep design (Cullis et al. 2006; Williams et al. 2011)) to a fraction prep of the lines, those with the highest plot value, always in a different block;
- then runs simulated annealing on swaps within blocks, and between blocks within a stratum, to minimise the kinship between neighbouring plots, so that close relatives are not side by side and genetic and spatial trends are not confounded.

It reports the exact CDmean with block effects fixed (every plot a record, blocks projected out as in Part 2) and without them: the difference is the information spent on the blocks.

```
lay <- tso_layout(plan, rows = 8, cols = 9, blocks = 3, checks = c("CHK1", "CHK2"),
                  prep = 0.1)
```

```
lay
```

```
#> TSOpt field layout · 8 x 9 plots · 3 blocks (serpentine runs (row bands))
#> Plots      60 lines + 6 second plots (p-rep) + 6 check plots (2 checks) · 72 of 72
  ↳ plots used
#> Blocks      24/24/24 entries per block · balanced over 4 strata (plan strata)
#> Neighbours mean kinship of neighbouring plots -0.018 -> -0.104 after 20,000 annealing
  ↳ steps; close relatives (>= 0.5) adjacent: 1 -> 0
#> Expected   CDmean 0.351 with block effects fixed · 0.356 ignoring blocks (cost of
  ↳ blocking 0.0049)
#> p-rep      second plots chosen by plan$selected$value
#> Next       plot(layout) · as.data.frame(layout) for the field book
```

```
head(as.data.frame(lay), 4) # the field book
```

```
#>  plot row col block  id      type rep stratum
#> 1    1   1   1     1 L275 training  1      G4
#> 2    2   1   2     1 L167 training  1      G3
#> 3    3   1   3     1 L077 training  1      G2
#> 4    4   1   4     1 L182 training  1      G3
```

```
tso_plot(lay)
```

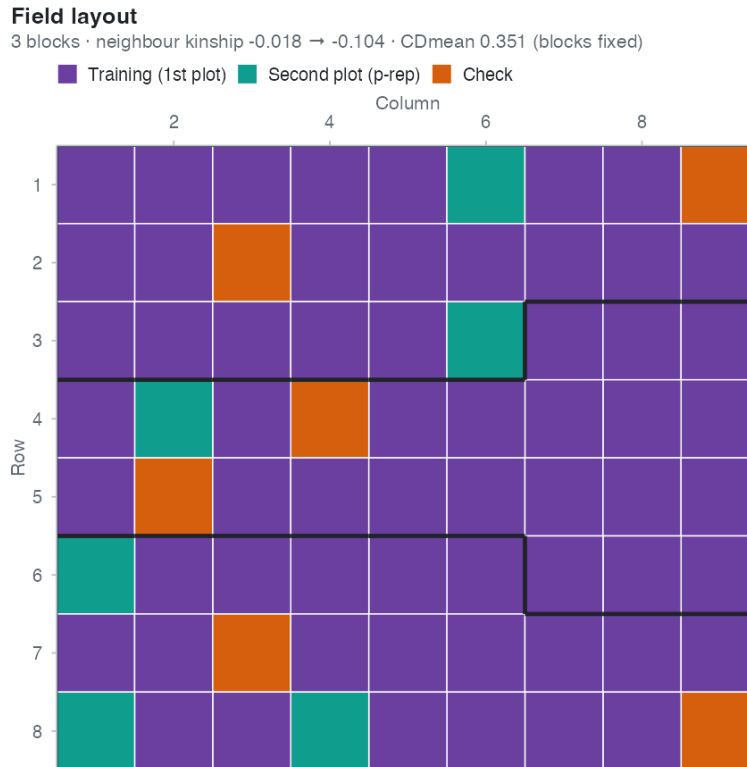


Figure 21: `tso_plot(layout)`: the field map, three blocks in serpentine bands, checks and p-rep second plots marked.

The annealing lowered the mean kinship between neighbours from -0.018 (random placement within blocks) to -0.104 , with 0 pairs of close relatives (≥ 0.5) adjacent instead of 1. Blocking costs 0.0049 of CDmean, a small price for protection against field trends. `plot(layout, fill = "group")` colours the map by genetic group. Multi-environment plans are laid out one field at a time; for spatially optimised designs with more elaborate structures (resolvable row-column, spatial correlation models), export the list and the p-rep choice to DiGger or FieldHub (Murillo et al. 2021) and keep TSOpt's choice of lines.

11.5 `tso_genotype_set()`: choose whom to genotype

The question before phenotyping: with money to genotype n individuals at high density (an imputation reference, or simply a genotyping budget), which ones? The relationships known beforehand come from a pedigree or a cheap low-density panel. On $\mathbf{R} = \text{cov2cor}(\mathbf{K})$ (so 0.5 is a parent or full sib, 0.25 a grandparent or half sib), the default method = "coverage" maximises the mean, over the targets, of the relationship to the closest genotyped individual. The objective is monotone and submodular, so the greedy set is within $1 - 1/e$ of the best possible (Nemhauser et al. 1978); it finds the key ancestors that tie the pedigree together. "cdmean" uses the CDmean engine with a genotype as an almost noise-free record, and "dopt" D-optimality. The set is compared with random sets and with the usual heuristic of choosing the most connected individuals (largest relationship row sums).

```
set.seed(3)
ped <- data.frame(id = paste0("F", 1:20), p1 = NA, p2 = NA, gen = 0) # 20 founders
for (gn in 1:4) { # 4 generations of 60
  prev <- ped$id[ped$gen == gn - 1]; m <- t(replicate(60, sample(prev, 2)))
  ped <- rbind(ped, data.frame(id = paste0("G", gn, "_", 1:60), p1 = m[, 1], p2 = m[, 2],
    ↪ gen = gn))
}
```

```

}
dped <- tso_data(pedigree = ped[, 1:3], verbose = FALSE)
gs <- tso_genotype_set(dped, n = 20)
gs

#> TSOpt genotyping set · 20 of 260 candidates · all 260 individuals · relationships from
  ↳ the pedigree
#> Method      coverage (facility location)    (0.0 s)
#> → Genotype these 20: 92% of the others then have a genotyped relative at ≥ 0.25
  ↳ (grandparent, half sib or closer) and 49% one at ≥ 0.5 (parent, full sib)
#> Coverage    mean best relationship 0.401 · median 0.471 · 10% of targets below 0.250
#> Compared with coverage    mean best ≥ 0.25 ≥ 0.5
#> Optimised      0.447      0.401      92%      49%
#> Most connected 0.302      0.244      45%      17%
#> Random (mean)  0.360      0.307      69%      25%
#> Random        the set beats 100% of 50 random sets of the same size
#> First          F8, G2_7, G3_22, G2_51, G1_59, G1_36, G1_45, G2_1, ...

table(generation = ped$gen[match(gs$selected$id, ped$id)])

#> generation
#> 0 1 2 3
#> 2 5 7 6

tso_plot(gs)

```

Genotyping 20 gives 92% of the others a relative ≥ 0.25

Non-genotyped targets · optimised set vs 50 random sets of the same size (69% ≥ 0.25)

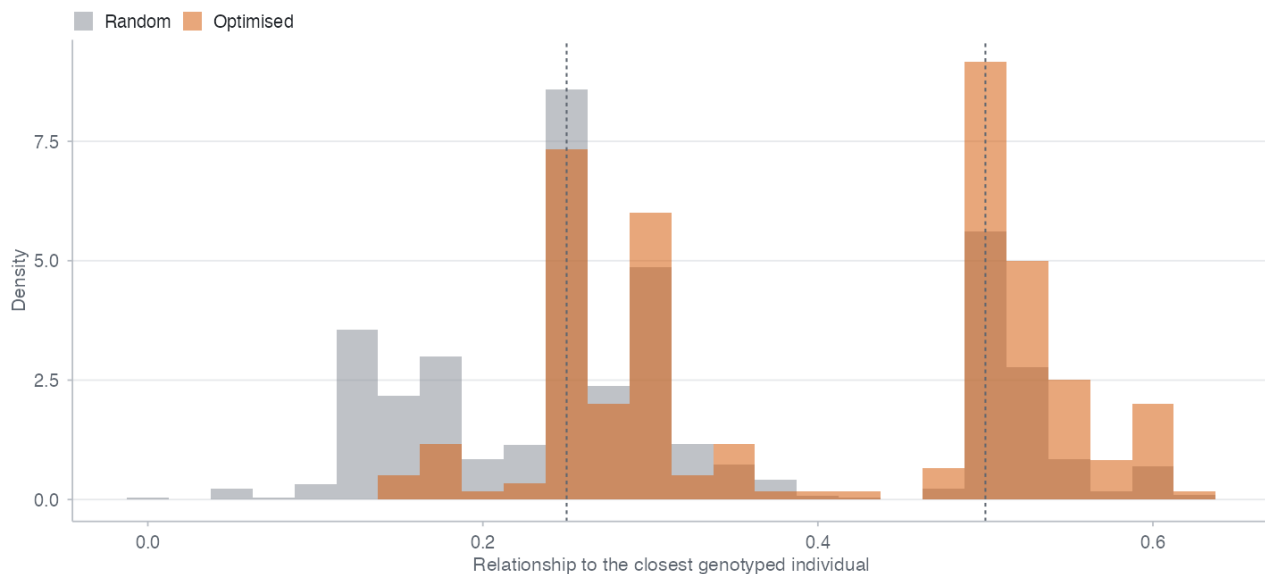


Figure 22: `tso_plot(genotype set)`: relationship of every non-genotyped individual to its closest genotyped relative, the chosen set against the most-connected heuristic and random sets.

With 20 genotyped individuals, 92% of the others have a genotyped relative at 0.25 or closer, against 69% for random sets and 45% for the most-connected individuals. The heuristic does worse than random: the most connected individuals are relatives of each other (one large family), so they cover the same part of the pedigree several times.

11.6 tso_transfer(): predict another programme

Breeding networks share training data: can lines phenotyped in programme A (from) predict the lines of programme B (to), and how many local “bridge” lines phenotyped in B recover the difference? `tso_transfer()` builds three designs of the same size with the robust CDmean engine, all targeted at the lines of B: from A only; from A and B together with at most `n_local` lines of B (the cap is enforced inside the greedy search, so it decides jointly which local lines to add and which lines of A they replace); and from B itself, the in-programme reference. *Transferability* is the accuracy of the first relative to the third, and the *bridge curve* repeats the second for caps from 0 to `n_local`. Here the tutorial panel plays four programmes, its four subpopulations.

```
tr <- tso_transfer(tp, from = c("G1", "G2", "G3"), to = "G4", n = 30, n_local = 10)
tr
```

```
#> TSOpt transfer · train in G1+G2+G3 (225 lines) → predict G4 (75 lines) · 30 plots ·
  ↳ averaged over h2 = 0.2, 0.5, 0.8
#> → Training in G1+G2+G3 predicts G4 with 28% of the accuracy of training in G4; adding
  ↳ 10 local lines raises it to 75%
#>      design      from G1+G2+G3 from G4  CDmean    accuracy  relative
#>      from only      30      0      0.021    0.145    28%
#>      with bridge    20     10     0.145    0.381    75%
#>      in-programme   0      30     0.262    0.512   100%
#> Bridge  0 local: 28% · 2 local: 46% · 4 local: 56% · 6 local: 63% · 8 local: 69% ·
  ↳ 10 local: 75%
#> Related  G4 lines: mean relationship to G1+G2+G3 -0.062 · closest relative in
  ↳ G1+G2+G3 0.028 on average (0.574 within G4) · 0% have a relative ≥ 0.2 in G1+G2+G3
```

```
tso_plot(tr)
```

Transfer from G1+G2+G3 to G4

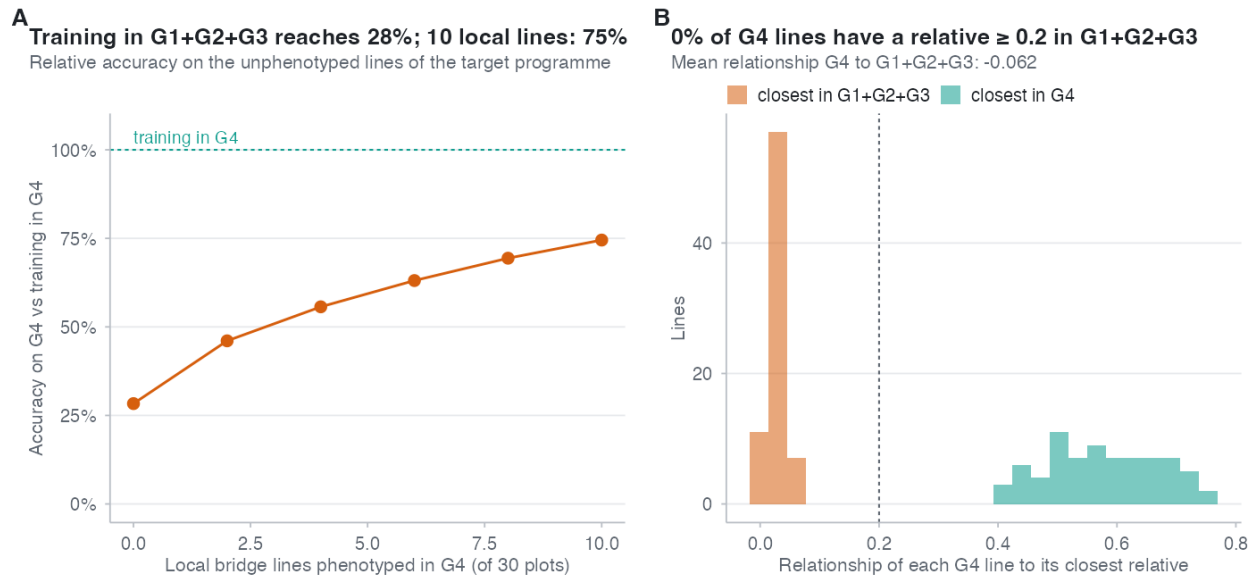


Figure 23: `tso_plot(transfer)`: accuracy for the lines of G4 as local bridge lines replace lines from G1 to G3, relative to training in G4 itself.

Thirty lines from G1 to G3 predict G4 with 28% of the accuracy of 30 lines from G4, because the closest relative of a G4 line in the other groups is at 0.028 on average (against 0.57 within G4). Ten local lines raise it to 75%; the curve is steepest for the first few bridge lines, which is the argument for every partner phenotyping at least a small local set. `cost_local` prices a plot in B

relative to one in A.

11.7 tso_backtest(): forward validation on your own history

`tso_backtest(geno, history, n)` replays a programme cycle by cycle. With `scheme = "within_cycle"` each method chooses which of cycle k 's candidates to phenotype, with all earlier cycles as training data, and GBLUP (a fixed effect per cycle, REML heritability) predicts the rest of cycle k ; with `"from_history"` each method chooses the training set among earlier lines to predict all of cycle k . `tso_simulate_cycles()` produces a realistic history (parents chosen on phenotype) to try it on.

```
hs <- tso_simulate_cycles(cycles = 4, n_candidates = 100, n_crosses = 10, n_parents = 10,
  ↪ markers = 500, seed = 4)
tso_backtest(hs$geno, hs$pheno, n = 0.25, scheme = "from_history", methods = c("random",
  ↪ "default"),
  n_random = 2, verbose = FALSE)
```

```
#> TSOpt forward validation · from_history · 3 test cycles x 1 trait(s) · n = 0.25 per
  ↪ cycle (0.2 s)
#> method          PA   rank  top10  PA vs random (95% CI) · wins
#> random          0.058 0.065 0.100 reference
#> default          0.166 0.165 0.133 +0.108 (-0.132 to +0.347) · 67% of 3
#> Predictive ability within each test cycle, from data available at the time. plot(bt)
  ↪ shows every cycle.
```

In 40 simulated histories (`benchmark/forward_benchmark.R`) the default gained +0.009 within the cycle ($t = 3.1$) and +0.091 from history ($t = 13.4$) over random.

11.8 tso_season(): where the plots go across stages

`tso_season(data, budget)` evaluates two-stage allocations (stage-1 training set n_1 , lines advanced k_1 , stage-2 sites) by simulation on the panel: $g \sim N(0, K)$, genotype x environment variance and plot error set by `ge_ratio` and `h2_plot`, BLUP with the true variance components, precision-weighted means across stages, and the gain of the final `n_select` lines. Common random numbers make the paired differences between allocations precise.

11.9 tso_crosses(): which crosses to make

For inbred parents the progeny variance of a cross (doubled haploids from the F1) is $\frac{1}{4}\delta'R\delta$ with $\delta_j = a_j(x_{P_{1j}} - x_{P_{2j}})$ and $R_{jk} = 1 - 2r_{jk}$ on the same chromosome. With Haldane's map function $1 - 2r = e^{-2d/100}$, so $\sum_{j,k} \delta_j \delta_k e^{-0.02|d_j - d_k|}$ is computed in one pass along each chromosome; for RILs r becomes $2r/(1 + 2r)$. The usefulness is $U = \mu + i\sigma$ (Schnell 1983). Against 3,000 simulated doubled haploids per cross the predicted SD is within 1-2%.

11.10 tso_imputation_plan(): genotyping and plots under one budget

Each split of the budget between high-density reference lines, a low-density panel and plots is scored by the realised reliability of GBLUP that uses the imputed relationships $\hat{K}$ while the truth

follows K :
$$\frac{(c'\hat{K}_{TS}\hat{V}^{-1}K_{ST}c)^2}{c'\hat{K}_{TS}\hat{V}^{-1}(K_{SS} + \sigma^2 I)\hat{V}^{-1}\hat{K}_{ST}c c'K_{TT}c}$$
, which equals CDmean when $\hat{K} = K$.

Imputation is by the k nearest reference lines on the low-density markers of each window.

11.11 tso_omics_plan(): profiles or plots?

On pilot lines that are genotyped and profiled, every split of one budget between profiles and plots is designed. The lines to profile are chosen:

- by genomic coverage;
- as the lines to be phenotyped;
- as the lines to be predicted;
- or at random.

The missing profiles are predicted from genomics, as above, and every plan is scored by the same realised reliability as the imputation plan, with the pilot's full omics kernel as the truth. In the benchmark, profiling paid only when a profile cost about 5% of a plot and half the signal ran through the profiles; profiling part of the lines was never the best plan.

11.12 To the field, the database and other software

tso_export_field() (Field Book, FieldHub, Breedbase), tso_brapi_write_list() and tso_brapi_write_units() (BrAPI write-back), tso_manifest() / tso_verify() (MD5 fingerprints of the inputs), tso_plan_json() (the versioned tsopt-plan/1 file) and tso_analysis_script() (the matching sommer, ASReml-R or rrBLUP model). Outside R, the tsopt command-line tool (inst/cli/tsopt), the REST service (inst/plumber/api.R), the Docker image (Dockerfile) and the Python client (python/tsopt.py) run the same functions.

11.13 Kernel robustness: when the trait is not additive

The additive relationship matrix assumes additive gene action. Dominance matters in clones and hybrids, and the Gaussian kernel of RKHS regression $\exp(-d_{ij}^2/\text{median } d^2)$ on standardised markers captures epistasis (Gianola and van Kaam 2008; de los Campos et al. 2010). A set chosen for one kernel can over-fit it, exactly as a set chosen for one h^2 over-fits that h^2 . kernels = c("additive", "dominance", "gaussian") averages the criterion over the kernels (and over the heritability grid), and plan\$value\$cd_by_scenario reports the CDmean under every kernel $\times h^2$ pair. It works for line-level data with genotypes; the dominance kernel is for diploids (Vitezica et al. 2013) and is not informative on doubled haploids, so the example uses an outbred clonal panel. (To design under one combined model with variance shares, and for breeding values, use tso_data(kernels =) and value = instead; Parts 5 and 6.)

```
po <- tso_simulate_panel(n = 200, markers = 800, groups = 4, fst = 0.15, seed = 7,
                        type = "outbred")
to <- tso_data(po, system = "clonal", verbose = FALSE)
kern3 <- c("additive", "dominance", "gaussian")
p_add <- tso_design(to, n = 40, verbose = FALSE) # the default design
p_ker <- tso_design(to, n = 40, kernels = kern3, verbose = FALSE) # robust over kernels
p_ker$spec$search
```

```
#> [1] "greedy (robust over kernels: additive, dominance, gaussian x h2 grid)"
```

```
score_kernels <- function(ids) # CDmean of a fixed set under every kernel x
  ↪ h2
  tso_design(to, n = 40, must_include = ids, kernels = kern3, n_random = 0,
             verbose = FALSE)$value$cd_by_scenario
kt <- data.frame(default = score_kernels(p_add$selected$id),
                 kernel_robust = p_ker$value$cd_by_scenario)
kt$difference <- kt$kernel_robust - kt$default
round(kt, 4)
```

```
#>
#> additive h2=0.2    0.1790    0.1755    -0.0035
```

```
#> additive h2=0.5    0.3253      0.3216    -0.0037
#> additive h2=0.8    0.4338      0.4329    -0.0008
#> dominance h2=0.2   0.0658      0.0647    -0.0011
#> dominance h2=0.5   0.1502      0.1508     0.0006
#> dominance h2=0.8   0.2307      0.2343     0.0036
#> gaussian h2=0.2    0.0913      0.0915     0.0002
#> gaussian h2=0.5    0.2089      0.2089     0.0001
#> gaussian h2=0.8    0.3267      0.3282     0.0015
```

Scoring a fixed set under every kernel is done by forcing it in with `must_include` (nothing is left to choose). "additive" means the data's own kernel, which for `system = "clonal"` already carries the dominance share of Part 7; "dominance" and "gaussian" are built from the markers. Against the default design, the kernel-robust set changes CDmean by -0.0027 on average under the additive kernel and by +0.0008 under the dominance and Gaussian kernels; the two sets share 23 of 40 lines. As with the heritability grid, the point is not a higher number under one model but no bad surprise under any.

11.14 BrAPI: design from the breeding database

Most programmes keep genotypes and phenotypes in a breeding database (BMS, Breedbase, GIGWA, DeltaBreed) that speaks the Breeding API, BrAPI v2 (Selby et al. 2019). Six functions read from any such server, so a design can start from the database instead of an exported file. They need the `curl` and `jsonlite` packages. The code is shown but not run here, because it needs a network connection; the package tests run it against recorded responses of the public test server.

```
con <- tso_brapi_connect("https://test-server.brapi.org",
                        token = Sys.getenv("BRAPI_TOKEN")) # token optional
con                                     # server name and the endpoints it offers
tso_brapi_studies(con, commonCropName = "Rice")           # trials and studies
tso_brapi_variantsets(con)                               # genotyping data sets
geno <- tso_brapi_geno(con, variant_set = "variantset1")  # a tso_geno object
y <- tso_brapi_trait(con, study = "study1", trait = "Plant height") # germplasm means
ph <- tso_brapi_pheno(con, study = c("study1", "study2")) # every observation
tp_db <- tso_data(geno)                                  # from here on, as in every other part
plan_db <- tso_design(tp_db, n = 150, history = names(y))
```

function	BrAPI v2 endpoints	returns
<code>tso_brapi_connect()</code>	<code>/serverinfo</code>	a connection: URL, token, the services the server offers
<code>tso_brapi_studies()</code>	<code>/studies</code>	studies with trial, location, season, crop
<code>tso_brapi_variantsets()</code>	<code>/variantsets</code>	the genotyping data sets
<code>tso_brapi_geno()</code>	<code>/allelematrix (2.1) or</code> <code>/variantsets/{id}/calls;</code> <code>/variants, /samples, /germplasm</code>	a <code>tso_geno</code> : dosages of any ploidy, map, germplasm names
<code>tso_brapi_pheno()</code>	<code>/observations</code>	one row per observation: germplasm, trait, value, study, season
<code>tso_brapi_trait()</code>	<code>/observations</code>	one trait as a named vector of germplasm means

The connector pages through large results, waits for asynchronous searches (`/search/...` returning a `searchResultsDbId`), retries on network errors and HTTP 429/5xx, and sends an access token as `Authorization: Bearer`. Genotype strings (0/1, 1|1, 0/0/1/1, ./.) become counts of

the alternate allele. Rows are named by germplasm name by default; use the same names = in `tso_brapi_geno()` and `id` = in `tso_brapi_pheno()` so that genotypes and phenotypes match. `tso_brapi_trait()` pools observations with a simple mean; for trials with strong study effects, fit adjusted means (BLUEs) from the long table of `tso_brapi_pheno()` instead.

12 Check, show and share

TSOpt tutorial · Part 12

Check, show and share

Scores, comparisons, evaluation, the figure gallery, exports, reports and the app

tso_score / compare

Any set scored on any criterion; the plan against random, stratified, PAM and kinship.

tso_evaluate()

After the season: predictive ability, NDCG, realised gain, bias; forward vs cross-validation labelled.

tso_plot / save_figure

Every result has a figure, one colour meaning, journal presets: PDF, SVG, EPS, TIFF, PNG.

tso_export / report / app

The list for the field team, a one-page report, and the point-and-click version.

Every figure is an ordinary ggplot2 object: restyle, combine, annotate

12.1 tso_score(): any set, any criterion

Score a training set you already have (last year's trial, a colleague's list, a competitor's output) on the same criteria the design uses:

```
my_list <- tp$units$id[seq(1, 300, by = 5)] # every fifth line, 60 lines
rbind(my_list = tso_score(tp, train = my_list, h2 = 0.5),
      plan     = tso_score(tp, train = plan$selected$id, h2 = 0.5))
```

```
#>      cdmean cdmean_exact pevmean dopt
#> my_list 0.3201      0.3183  0.6795 37.40
#> plan    0.3459      0.3454  0.6537 37.81
```

cdmean is the engine's value, cdmean_exact the projected one, pevmean and dopt as in Part 3. Give target = to score against a target set.

12.2 tso_compare(): the plan against the alternatives

Every plan is compared, on the same target, against random sets (as a distribution), stratified random, PAM medoids, the most-related-to-target set and any other plans or lists you pass in others =:

```
tabu60 <- tso_design(tp, n = 60, h2 = 0.5, search = "tabu", verbose = FALSE)$selected$id
cm <- tso_compare(plan, others = list(every_fifth = my_list, tabu_h2_0.5 = tabu60),
  ↪ n_random = 100)
cm
```

```
#> TSOpt comparison · 60 units each · target: untargeted (whole pool)
#> Method      CDmean vs random Targets >= 0.20 Unreachable Unselectable
#> Optimised (TSOpt) 0.346 4.8% 100% 0 0
#> Random (mean) 0.315 0.0% 100% 0 0
#> Stratified random (mean) 0.317 0.3% 100% 0 0
#> Most related to target 0.308 -1.0% 97% 0 0
#> PAM medoids 0.326 1.7% 100% 0 0
#> every_fifth 0.320 0.8% 100% 0 0
```

```
#> tabu_h2_0.5          0.347  5.0%      100%          0          0
#> 'vs random' is on the accuracy scale (sqrt CDmean). Plot with tso_plot(comparison).
tso_plot(cm)
```

The plan against the alternatives

60 units per method; higher is better

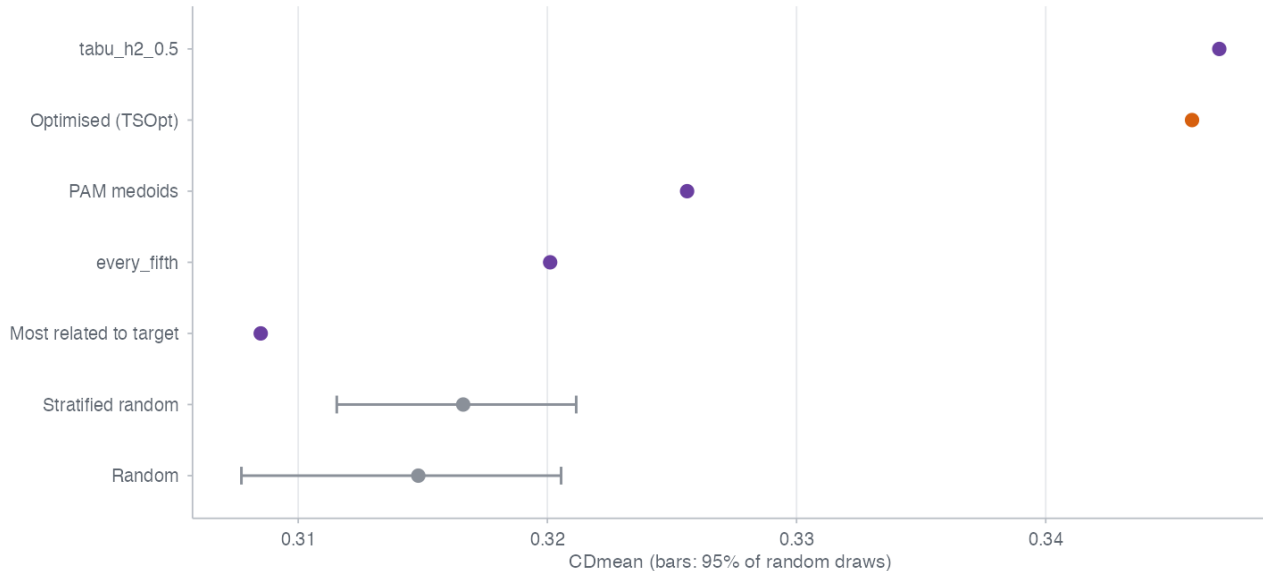


Figure 24: `tso_plot(comparison)`: methods side by side on the accuracy scale.

12.3 `tso_evaluate()`: after the season

Once phenotypes exist, evaluate what the training set actually delivered. `tso_evaluate()` fits a GBLUP (Endelman 2011) (REML heritability, or h^2) on the training units, predicts the targets and reports:

- **predictive ability**, the correlation between predicted and observed values (or the true genetic values, if a simulation provides truth =), labelled *same season* or *forward* (forward = TRUE, when the targets were grown later) because forward accuracy is usually half of within-panel cross-validation;
- **Spearman correlation** and **NDCG@top**, ranking metrics that weight the top of the list;
- **realised gain**: the mean value of the selected top fraction, in SD units above the target mean, which is what the breeder gets;
- **bias slope**: the regression of observed on predicted (1 = unbiased);
- the heritability used.

```
ev <- tso_evaluate(plan, y) # against the phenotypes of the other 240
  ↪ lines
ev
```

```
#> TSOpt evaluation · 60 training units -> 240 targets · against phenotypes
#> Predictive ability    0.434 (same season: optimistic; forward prediction is usually
  ↪ lower)
#> Rank correlation      0.417
#> NDCG@10%            0.723   top 10% overlap 17%
#> Realised gain        0.45 SD above the target mean (selecting the top 10%)
#> Bias slope           2.28 (1 = unbiased; > 1 = predictions shrunk)
#> Heritability used    0.19 (estimated, REML)
```

```
ev_true <- tso_evaluate(plan, y, truth = g) # against the true genetic values
  ↪ (simulation only)
c(vs_phenotype = ev$predictive_ability, vs_truth = ev_true$predictive_ability)
```

```
#> vs_phenotype    vs_truth
#>         0.4343         0.5255
```

ev\$predictions has the predicted and observed value of every target. train = and target = override the plan's sets, so the function also evaluates a training set that was not designed by TSOpt.

12.4 The figure gallery

Every figure is drawn by `tso_plot(object, type)` and returned as an ordinary `ggplot2` object, so that it can be restyled, combined with `patchwork` or annotated. One colour means one thing throughout the package (`tso_colors()`): training set purple, target teal, other candidates light grey, random baseline mid grey, optimised plan orange, warnings red. Palettes are colour-blind safe, and shape or line type is redundant with colour.

object	type	shows
plan	"summary" (default)	four panels: PCA, divide, coverage, baseline
plan	"pca"	the training set in genetic space
plan	"divide"	reliability of every candidate: the predictability divide
plan	"coverage"	closest training relative of each target
plan	"baseline"	the plan against random training sets
plan	"search"	the criterion along the search
plan	"kinship"	relationship heatmap, training marked
plan (hybrid)	"hybrid_grid"	crosses chosen, H0/H1/H2 classes
plan (environments)	"met_allocation"	line by environment allocation
plan (traits)	"trait_allocation"	trait by line allocation
plan (major loci)	"major"	carriers of each protected allele, targets against training (Part 9)
plan	"value"	CDmean lost if each plot is dropped (Part 11)
diagnosis	"summary", "size_curve", "structure", "baseline"	see Part 6
comparison	(default)	methods side by side
data	"pca", "kinship"	the panel before any design
batch	(default)	posterior ranking and the next plots
simulation	(default)	gain, variance, accuracy over cycles
evaluation	(default)	predicted against observed for the targets
major-locus	(default)	Manhattan plot with the clumped leads (Part 9)
scan	(default)	
sample size	(default)	the operating curve (Part 10)
top-k	(default)	selection metrics, plan against random (Part 10)
Pareto front	(default)	the trade-off and its knee (Part 10)
contributions	(default)	optimum contribution against GEBV (Part 10)
large plan	(default)	the criterion along the low-rank search (Part 10)
trait value	(default)	budget share and loss without each trait (Part 8)
uncertainty	(default)	marker-bootstrap distributions and the gain interval (Part 11)

object	type	shows
economics	(default)	value, cost and net value by size, the optimum (Part 11)
layout	(default), fill = "group"	the field map: blocks, checks, p-rep plots (Part 11)
genotype set	(default)	closest genotyped relative, set against heuristic and random (Part 11)
transfer	(default)	the bridge curve (Part 11)

```
tso_plot(plan)
```

Training-set plan: 60 units · AMBER

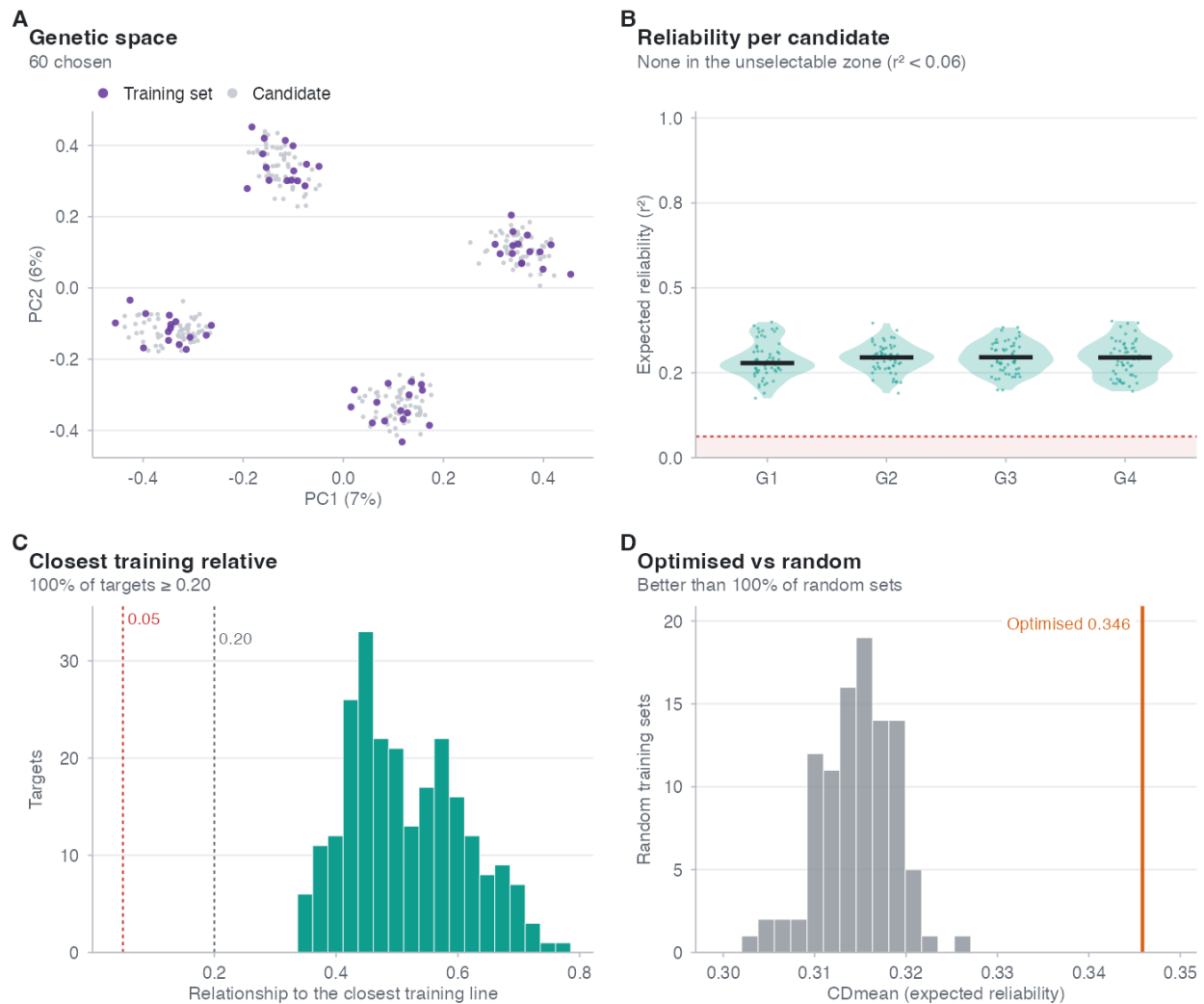


Figure 25: tso_plot(plan): the four-panel summary figure of a plan.

```
tso_plot(plan, "divide")
```

Expected reliability per candidateNone in the unselectable zone ($r^2 < 0.06$)

Figure 26: The predictability divide: reliability of every candidate under the plan; the shaded zone marks candidates no selection decision can distinguish from the mean.

`style = "presentation"` enlarges text and lines for slides; `"course"` uses the PlantNura palette. `tso_theme()` is the ggplot2 theme behind every figure and can be added to your own plots; `title = FALSE` removes the automatic title.

12.5 `tso_save_figure()`: journal-ready files

```
p <- tso_plot(plan, "pca") + labs(title = NULL)
tso_save_figure(p, "Fig2", journal = "tag", width = "single", format = c("pdf", "tiff"))
tso_save_figure(tso_plot(plan), "Fig1", journal = "g3", width = "double", format = "pdf")
```

Journal presets set the column widths ("single", "onehalf", "double", or millimetres): generic, nature, plos, g3, genetics, tag (Theoretical and Applied Genetics), springer, frontiers, plant_genome, crop_science, elsevier. Formats: PDF and SVG (vector), EPS, PNG and TIFF at dpi (default 600). Text and line sizes are scaled to the final width so that a figure saved at 85 mm stays legible.

12.6 `tso_export()`: the list for the field team

```
out <- tempfile(fileext = ".csv")
tso_export(plan, out) # what = "selected" (default) or "all"
head(read.csv(out), 3)
```

```
#>   order  id line group cost reliability forced
#> 1     1  L234 L234   G4     1      0.5753  FALSE
#> 2     2  L083 L083   G2     1      0.5864  FALSE
#> 3     3  L068 L068   G1     1      0.5987  FALSE
```

12.7 `tso_report()`: one page for the programme manager

```
html <- tempfile(fileext = ".html")
```

```
tso_report(plan, html, diagnosis = diag, comparison = cm, title = "Training-set plan,  
↪ 2027 trial",  
          author = "W. Hussain")
```

The report is a self-contained HTML page (PDF through the PlantNura template when rmarkdown and a LaTeX installation are present): how many units, chosen how, expected accuracy and gain against random, the diagnosis, the comparison, the figures, the risks (coverage, divide, structure) and the list. It uses exactly the figure functions above, so what appears in the report is what you can reproduce in code.

12.8 tso_cli(): the command line

The tsopt command-line tool runs the main steps without writing R: in pipelines, on servers and for colleagues who work in a terminal. It reads the same files and writes the same plan files.

```
tsopt design --geno panel.vcf.gz --n 150 --out plan.json  
tsopt diagnose --geno panel.vcf.gz --n 150  
tsopt backtest --geno panel.vcf.gz --pheno history.csv --n 0.2
```

The tool is in `system.file("cli", "tsopt", package = "TSOpt"); tso_cli("help")` lists the commands. The same functions can also serve a web API (`system.file("plumber", "api.R", package = "TSOpt")`) inside an organisation.

13 The head-to-head benchmark

TSOpt tutorial · Part 13

The head-to-head benchmark

Six public panels, 33 scenarios, realised accuracy against STPGA, TSDFGS and a TrainSel-style GA

Design

$n = 15\%$ of candidates; untargeted and targeted scenarios; GBLUP accuracy on never-phenotyped lines.

Result

Robust CDmean: +0.069
PA over random, wins 91% of cases, 0.03 s per design.

Lesson

Optimising the criterion harder at one assumed h^2 did not predict better. Hedging did.

Honesty

Margins over STPGA/TSDFGS are clear; over the GA+SA small; on low- h^2 spruce nobody gains much.

Every number in this chapter comes from benchmark/results/ in the repository

Defaults should come from evidence, not taste. TSOpt's defaults were chosen by running every criterion and search in the package, and the main competing packages, on six public panels, and measuring what a breeder cares about: the accuracy of the predictions that the chosen training set actually produced. The scripts, every raw result and the figures are in benchmark/ in the repository; this part reads the result files directly.

13.1 Design

Panels	The datasets of Fernández-González et al. (2023): Maize (391 lines), RicePS (357), Rice (327), Sorghum (451), Switchgrass (514), Spruce (1,722); all traits of each panel. The soybean file in that distribution is corrupt and was left out.
Scenarios	<i>Untargeted</i> : the whole pool is predicted. <i>Targeted</i> : a random 25% of lines are targets and are never phenotyped; five random splits per panel (two for Spruce, where one TSDFGS run takes over an hour). 33 scenarios in total.
Budget	$n = 15\%$ of the candidates.
Score	Realised predictive ability: GBLUP with REML heritability fitted on the chosen set, correlation between predicted and observed values on the targets, averaged over the panel's traits. Also NDCG@10% , the exact CDmean at $h^2 = 0.5$, and wall-clock time on one core.
Methods	TSOpt: robust CDmean, greedy CDmean, greedy + exchange, tabu, CDmean without the intercept, auto-stratified, cdm, coverage, D-optimality. STPGA 5.2 (genetic algorithm, CDMEAN, 300 iterations, 30 PCs). TSDFGS 2.0 (CD and r-score). A GA + SA from random starts on exact CDmean, as TrainSel searches. Baselines: random (five draws), stratified by the panels' own clusters, PAM, most related to target.
TrainSel	Not run: its licence forbids use by private or for-profit organisations. The GA + SA reproduces its search strategy on TSOpt's exact criterion without its code.

13.2 Results

```

bs <- read.csv("bench/benchmark_summary.csv")
bs$method <- sub("TSOpt Bayesian h2 \\(0.2/0.5/0.8\\)", "TSOpt robust CDmean (default)",
  ↪ bs$method)
tab <- data.frame(Method = bs$method, `PA gain` = sprintf("%+.3f", bs$pa_gain),
  SE = sprintf("%.3f", bs$pa_gain_se), `Beats random` = sprintf("%.0f%%",
  ↪ 100 * bs$win_rate_pa),
  `NDCG gain` = sprintf("%+.3f", bs$ndcg_gain), `Median s` =
  ↪ signif(bs$median_seconds, 2),
  check.names = FALSE)

```

Table 1: Realised predictive ability (PA) gain over random training sets, 33 scenarios, sorted.

Method	PA gain	SE	Beats random	NDCG gain	Median s
TSOpt robust CDmean (v0.3 default)	+0.069	0.012	91%	+0.035	0.034
TSOpt greedy only	+0.064	0.012	85%	+0.035	0.016
TSOpt coverage	+0.061	0.012	85%	+0.025	0.17
GA+SA from random start (TrainSel-style)	+0.061	0.012	85%	+0.028	6.8
TSOpt auto-stratified	+0.060	0.012	85%	+0.030	0.094
TSOpt greedy + exchange	+0.059	0.012	82%	+0.031	0.027
TSOpt CDmean h2=0.5 + tabu (v0.2 default)	+0.058	0.012	82%	+0.031	0.072
TSOpt CDmean without fixed intercept	+0.055	0.012	79%	+0.028	0.065
TSDFGS r-score	+0.044	0.011	82%	+0.020	69
STPGA GA (CDMEAN)	+0.044	0.011	76%	+0.029	68
TSOpt worst-case CD (cdmin)	+0.028	0.012	67%	+0.007	0.44
TSOpt D-optimality	+0.007	0.009	58%	+0.002	0.12
Stratified random (dataset clusters)	+0.006	0.010	61%	-0.001	0
PAM medoids	-0.013	0.011	45%	+0.005	0.13
TSDFGS CD	-0.091	0.021	24%	-0.021	1.5
Most related to target	-0.114	0.029	27%	-0.032	0.11

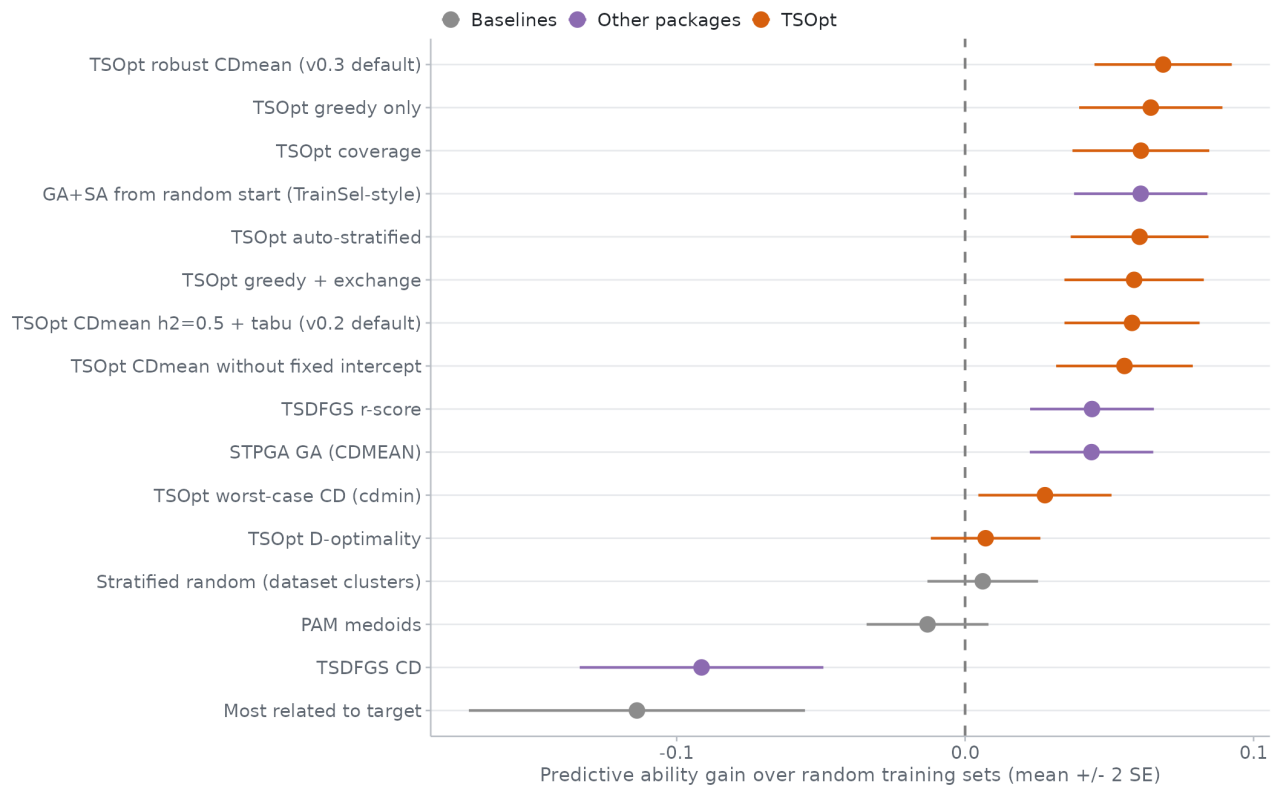
Realised accuracy on six public panels (33 scenarios)GBLUP, all traits, untargeted and targeted scenarios, $n = 15\%$ of candidates

Figure 27: Realised accuracy on six public panels: mean gain over random with 2 SE bars.

Accuracy against run time

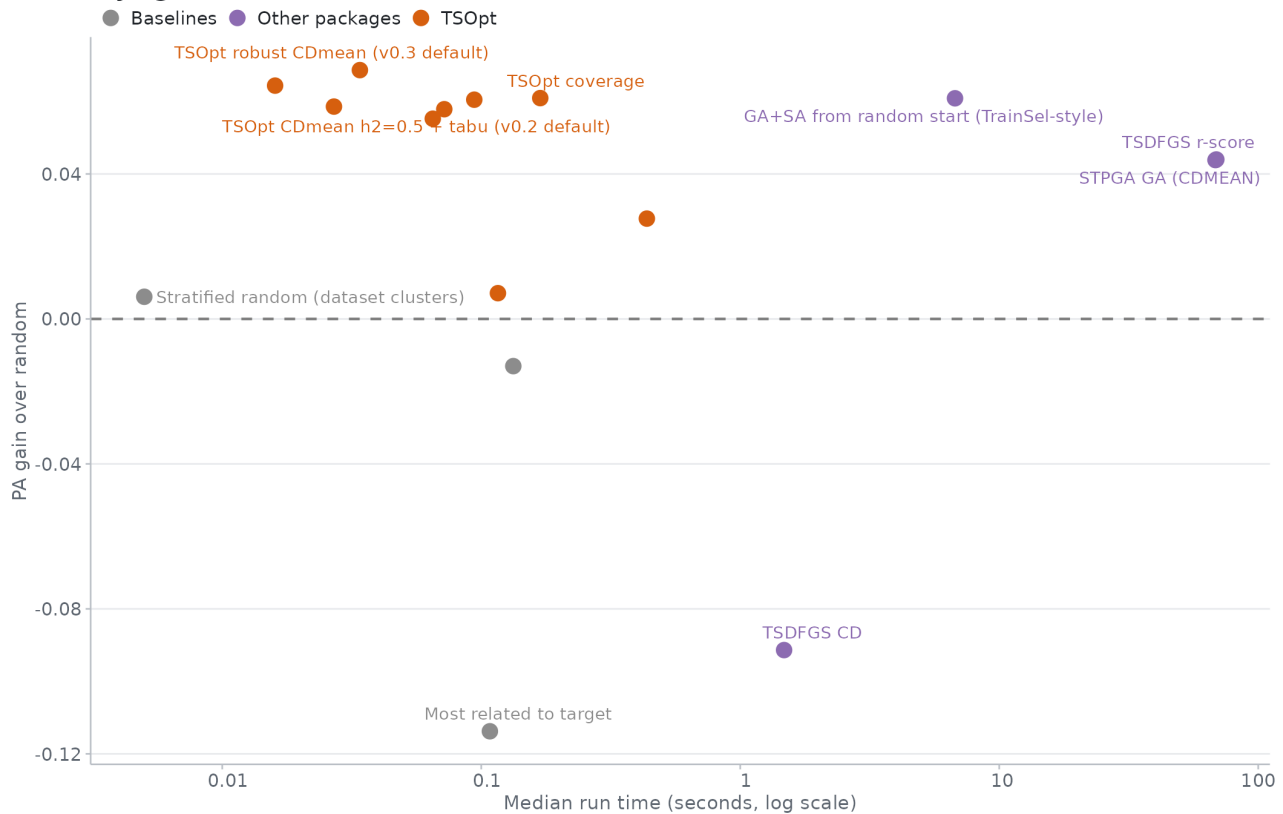


Figure 28: Accuracy against run time (log scale).

The paired comparison is the fair test: on each of the 33 scenarios, the default minus each other method, so that panel and split effects cancel.

```
bp <- read.csv("bench/benchmark_paired_vs_default.csv")
tab2 <- data.frame(Versus = bp$versus, `Default's advantage` = sprintf("%+.3f ± %.3f",
  ↪ bp$pa_advantage, bp$se),
  `Default wins` = sprintf("%.0f%", 100 * bp$default_wins),
  `Sign test p` = signif(bp$p_sign, 2), check.names = FALSE)
```

Table 2: TSOpt default minus each method, realised PA, 33 paired scenarios.

Versus	Default's advantage	Default wins	Sign test p
Most related to target	+0.182 ± 0.028	88%	1.1e-05
TSDFGS CD	+0.160 ± 0.019	97%	7.9e-09
PAM medoids	+0.082 ± 0.016	79%	0.0013
Random	+0.069 ± 0.012	91%	1.4e-06
Stratified random (dataset clusters)	+0.063 ± 0.014	85%	6.6e-05
TSOpt D-optimality	+0.062 ± 0.013	85%	6.6e-05
TSOpt worst-case CD (cdmin)	+0.041 ± 0.013	73%	0.014
STPGA GA (CDMEAN)	+0.025 ± 0.008	61%	0.3
TSDFGS r-score	+0.025 ± 0.010	67%	0.08
TSOpt CDmean without fixed intercept	+0.013 ± 0.005	76%	0.0046
TSOpt CDmean h2=0.5 + tabu (v0.2 default)	+0.011 ± 0.006	76%	0.0046

Versus	Default's advantage	Default wins	Sign test p
TSOpt greedy + exchange	+0.010 ± 0.005	76%	0.0046
TSOpt auto-stratified	+0.008 ± 0.006	73%	0.014
GA+SA from random start (TrainSel-style)	+0.008 ± 0.005	70%	0.035
TSOpt coverage	+0.008 ± 0.006	55%	0.73
TSOpt greedy only	+0.004 ± 0.003	67%	0.08

```

bd <- read.csv("bench/benchmark_pa_gain_by_dataset.csv", check.names = FALSE)
bd$method <- sub("TSOpt Bayesian h2 \\\(0.2/0.5/0.8\\)", "TSOpt robust CDmean (default)",
  ↪ bd$method)
bd[-1] <- lapply(bd[-1], function(v) sprintf("%.3f", v))
bd$method <- sub(" \\\(TrainSel-style\\)", "", sub("Stratified random \\\(dataset
  ↪ clusters\\)", "Stratified random", bd$method))

```

Table 3: PA gain over random, by panel.

method	Maize	Rice	RicePS	Sorghum	Spruce	Switchgrass
GA+SA from random start	+0.034	+0.136	+0.077	+0.084	+0.005	+0.002
Most related to target	+0.018	+0.037	-0.269	-0.075	-0.069	-0.302
PAM medoids	-0.074	-0.006	-0.015	+0.012	-0.029	+0.025
STPGA GA (CDMEAN)	+0.036	+0.123	+0.035	+0.032	-0.000	+0.015
Stratified random	+0.015	-0.003	-0.009	+0.022	+0.001	+0.008
TSDFGS CD	-0.065	-0.013	-0.101	-0.003	-0.074	-0.283
TSDFGS r-score	+0.020	+0.093	+0.041	+0.061	-0.004	+0.028
TSOpt CDmean h2=0.5 + tabu (v0.2 default)	+0.036	+0.131	+0.043	+0.084	+0.019	+0.015
TSOpt CDmean without fixed intercept	+0.024	+0.135	+0.050	+0.082	+0.001	+0.013
TSOpt D-optimality	+0.065	-0.008	-0.000	+0.030	-0.002	-0.046
TSOpt auto-stratified	+0.036	+0.131	+0.063	+0.084	+0.019	+0.009
TSOpt coverage	+0.050	+0.136	+0.063	+0.079	-0.033	+0.022
TSOpt greedy + exchange	+0.036	+0.136	+0.043	+0.084	+0.016	+0.015
TSOpt greedy only	+0.036	+0.136	+0.071	+0.092	+0.009	+0.015
TSOpt robust CDmean (v0.3 default)	+0.041	+0.142	+0.084	+0.088	+0.016	+0.016
TSOpt worst-case CD (cdmin)	+0.055	+0.092	-0.019	+0.016	-0.020	+0.017

13.3 What the benchmark taught us

1. **A better criterion value is not a better prediction.** Tabu search and exchange give the highest CDmean at the assumed $h^2 = 0.5$, and lower realised accuracy than the plain greedy set. They over-fit an assumed heritability that is wrong for most traits: the REML h^2 of the benchmark traits runs from 0.17 (Spruce diameter) to 0.95 (Maize).
2. **Hedging over heritability wins.** Averaging CDmean over 0.2, 0.5 and 0.8 was the most accurate method overall, first on Rice and RicePS, second on Sorghum, and within 0.025 of the best on the other three. A supplementary probe on five panels compared fixed h^2 values from 0.1 to 0.8 with and without exchange; the robust design was always at or near the top, and no single fixed value was.

3. **The exact intercept matters.** Dropping it cost 0.013 PA on average; the default won 76% of the paired comparisons (sign test $p = 0.005$).
4. **STPGA and TSDFGS work in a 30-PC space.** That keeps them affordable and loses information; they also take a thousand to a hundred thousand times longer. TSDFGS's CD mode was worse than random on every panel.
5. **Optimisation is not always worth much.** On Spruce (low h^2 , 1,722 lines) no method gained more than 0.02 over random. This is why `tso_diagnose()` exists and why the verdict line of every plan says so plainly.
6. **The margins are honest ones.** Against STPGA and TSDFGS the default's advantage is about 0.025, clearly above its standard error; against the TrainSel-style GA + SA it is 0.008, small and marginal, at two hundred times the speed.

Key point

The default of `tso_design()` is the method with the best realised accuracy in this benchmark, not the method with the best criterion value. When you report a design, report both: the criterion (`plan$value$cdmean_exact`) and, once phenotypes exist, the realised accuracy (`tso_evaluate()`).

The benchmark above concerns polygenic traits. The major-gene benchmark, a separate and simulated one (the script `major_genes_benchmark.R` in `benchmark/`), and is run in full in Part 9: there the protected design beat standard CDmean in 30/30 traits targeted and 30/30 untargeted.

13.4 Reproducing it

```
export TSO_BENCH_DATA=/path/to/Fernandez-Gonzalez_2022_Comparison/Datasets
export TSO_BENCH_OUT=results
Rscript benchmark/run_benchmark.R RicePS Rice Maize Sorghum Switchgrass Spruce # ~6 h,
↳ mostly STPGA/TSDFGS
Rscript benchmark/analyse_benchmark.R results
Rscript benchmark/major_genes_benchmark.R # about a
↳ second
```

14 Recipes, FAQ and crosswalk

TSOpt tutorial · Part 14

Recipes, FAQ and crosswalk

Copy-paste answers to the questions breeders actually ask

Recipes 150 lines for this season; which crosses to make; cut plots; lay out the field; design from the database.	FAQ Which h2? Why stratify? What if the verdict is RED? How large should n be?	Crosswalk Your STPGA, TSDFGS, TrainSel, FieldHub, AlphaSimR and BrAPI calls, translated.	Troubleshooting Unmatched IDs, unreadable files, slow runs, unexpected verdicts.
---	--	--	--

If a question is not answered here, `tso_explain(plan)` usually answers it

14.1 Recipes

Each recipe is complete; replace the file names and numbers.

1. This season's 150 plots, untargeted reference panel.

```
tp <- tso_data("panel.vcf.gz")
tso_diagnose(tp, n = 150)
plan <- tso_design(tp, n = 150)
tso_export(plan, "trial_2027.csv")
```

2. Predict this season's candidates (they will not be phenotyped).

```
plan <- tso_design(tp, n = 150, target = candidates_2027)
```

3. A budget in money, checks forced in, lines without seed excluded.

```
tp <- tso_data("panel.vcf.gz", cost = cost_per_line) # named vector, currency
  ↳ per unit
plan <- tso_design(tp, budget = 250000, must_include = checks, exclude = no_seed)
plan$spent
```

4. Last year's trial counts, discounted.

```
plan <- tso_design(tp, n = 100, history = phenotyped_2026, history_weight = 0.8)
```

5. Optimise the selection decision, not the average accuracy.

```
plan <- tso_design(tp, n = 150, goal = "gain", select_top = 0.05)
```

6. A genebank core where no accession is left unpredictable.

```
tg <- tso_data("collection.hmp.txt", system = "genebank")
plan <- tso_design(tg, n = 400, criterion = "cdmin")
```

7. Which crosses to make and test.

```
th <- tso_data(parents_vcf, system = "hybrid", pools = list(females, males), crossable
  ↪ = old_parents)
plan <- tso_design(th, n = 200, predict = "H1", max_per_parent = 8)
tso_plot(plan, "hybrid_grid")
```

8. Sparse testing across three sites, each line grown once, every site kept.

```
tm <- tso_data(geno, envs = c("Hyderabad", "Ludhiana", "Pusa"), env_cor =
  ↪ Omega_from_last_years,
  env_noise = c(Hyderabad = 1, Ludhiana = 2, Pusa = 1))
plan <- tso_design(tm, n = 450, max_per_line = 1, min_per_env = 100, target =
  ↪ candidates_2027)
```

9. Which trait on which line, for the lab's capacity.

```
tt <- tso_data(geno, traits = c("yield", "protein", "ndvi"), trait_h2 = c(0.3, 0.5,
  ↪ 0.6),
  gcor = R, trait_cost = c(20, 35, 0.5), plot_cost = 10)
plan <- tso_design(tt, budget = 30000, index = c(yield = 1, protein = 0.4))
```

10. Stage 2: where do the next 40 plots go?

```
nb <- tso_next_batch(stage1_plan, y_stage1, n = 40, select_top = 0.1)
nb$posterior
```

11. Next season, same programme.

```
plan_2028 <- tso_update(plan_2027, geno = "panel_2028.vcf.gz", n = 120)
```

12. A known heritability, or your own prior.

```
plan <- tso_design(tp, n = 150, h2 = 0.35)
plan <- tso_design(tp, n = 150, h2 = c(0.1, 0.2, 0.3))
```

13. The strongest optimiser of the criterion, for a methods comparison.

```
plan <- tso_design(tp, n = 150, h2 = 0.5, search = "tabu")
```

14. Score a list you already have, and compare.

```
tso_score(tp, train = my_list)
tso_compare(plan, others = list(my_list = my_list))
```

15. Journal figures.

```
tso_save_figure(tso_plot(plan), "Fig1", journal = "tag", width = "double", format =
  ↪ c("pdf", "tiff"))
```

16. Protect a resistance gene.

```
scan <- tso_major_scan(geno, y_last_trial) # or give the gene's marker
  ↪ by name
scan <- tso_major_effects(geno, "published_gwas.csv", h2 = 0.5, var_p = 1.8) # no
  ↪ phenotypes: known effects
tp <- tso_data(geno, major = scan, min_carriers = 3) # or major = "Lr34_snp",
  ↪ major_model = "fixed"
plan <- tso_design(tp, n = 150, target = candidates_2027)
plan$major # carriers of every allele class
tso_plot(plan, "major")
```

17. How many lines do I need?

```
ss <- tso_sample_size(tp, target = candidates_2027)
ss$n_for # sizes for 90 / 95 / 99% of
  ↳ the maximum
plan <- tso_design(tp, n = ss$n_for[["0.95"]], target = candidates_2027)
```

18. Balance accuracy and diversity.

```
fr <- tso_pareto(tp, n = 150, objectives = c("cdmean", "coancestry"), target =
  ↳ candidates_2027)
tso_plot(fr)
ids <- tso_pareto_pick(fr, "knee")
plan <- tso_design(tp, n = 150, must_include = ids, target = candidates_2027) # a full
  ↳ plan object for that set
```

19. Choose parents after the trial.

```
ev <- tso_evaluate(plan, y_2027)
ocs <- tso_contribution(ev$predictions, tp$K_lines, delta_F = 0.01, n_matings = 60)
ocs$contributions[ocs$contributions$selected, ]
```

20. A panel of 20,000 lines.

```
plan <- tso_design_large("panel_20k.vcf.gz", n = 1000, rank = 300)
plan$spec$variance_captured # raise rank if below 0.7
```

21. Trials, checks or covariates in the prediction model.

```
tp$units$trial <- trial_of_each_line
plan <- tso_design(tp, n = 150, fixed = "trial") # every trial is opened automatically
  ↳ (Part 6)
```

22. Cut plots when seed runs short.

```
plan$selected[order(plan$selected$value), c("id", "value_pct")][1:10, ] # the cheapest
  ↳ to lose
tso_plot(plan, "value")
plan2 <- tso_design(tp, n = 140, must_include = survivors, exclude = no_seed) # re-rank
  ↳ after cuts
```

23. Is the advantage real?

```
unc <- tso_uncertainty(plan, B = 100)
unc$verdict$label # "robust", "small", "uncertain", "not
  ↳ supported"
```

24. How many plots pay off?

```
ec <- tso_economics(tp, cost_per_plot = 40, value_per_sd = 25000, select_top = 0.1,
  ↳ horizon = 3)
ec$optimum$opt$n # the size with the largest net value
tso_plot(ec)
```

25. Lay out the field.

```
lay <- tso_layout(plan, rows = 12, cols = 14, blocks = 4, checks = c("Check_A",
  ↳ "Check_B"), prep = 0.2)
write.csv(as.data.frame(lay), "field_book_2027.csv", row.names = FALSE)
tso_plot(lay)
```

26. Choose whom to genotype.

```
dp <- tso_data(pedigree = ped) # id, parent 1, parent 2
gs <- tso_genotype_set(dp, n = 200, candidates = with_dna, must_include =
  ↪ already_genotyped)
gs$selected$id
```

27. Predict another programme.

```
tr <- tso_transfer(tp, from = lines_partner, to = lines_ours, n = 150, n_local = 30,
  ↪ cost_local = 1.5)
tr$transferability; tr$bridge # how many local lines are worth it
```

28. Design from the breeding database.

```
con <- tso_brapi_connect("https://my-breedbase.org", token = Sys.getenv("BRAPI_TOKEN"))
geno <- tso_brapi_geno(con, variant_set = "vs_2027")
plan <- tso_design(tso_data(geno), n = 150, history = names(tso_brapi_trait(con, study =
  ↪ "YT_2026", trait = "yield")))
```

29. Measure drone traits instead of yield plots.

```
tt <- tso_data(geno, traits = c("yield", "ndvi"), trait_h2 = c(0.3, 0.6), gcor =
  ↪ matrix(c(1, 0.6, 0.6, 1), 2),
  trait_cost = c(yield = 20, ndvi = 0.5), plot_cost = 10, envs =
  ↪ c("Hyderabad", "Ludhiana"))
plan <- tso_design(tt, budget = 30000, index = c(yield = 1, ndvi = 0)) # robust over
  ↪ trait scenarios
tso_trait_value(plan) # NDVI scans per yield plot, loss
  ↪ without NDVI
```

30. Protect genes from a published GWAS or last year's BayesB model (no phenotypes).

```
me <- tso_major_effects(geno, "bayesb_effects.csv") # marker, effect,
  ↪ effect_allele
me <- tso_major_effects(geno, "gwas_hits.csv", h2 = 0.4, var_p = 2.1) # GWAS: marker,
  ↪ beta, p
tp <- tso_data(geno, major = me) # weighted by the share
  ↪ each explains here
plan <- tso_design(tp, n = 150, target = candidates_2027)
```

31. Epistasis, and choosing parents rather than varieties.

```
tp <- tso_data(geno, kernels = c("A", "AA")) # robust share of A x A
  ↪ epistasis
plan <- tso_design(tp, n = 150, target = parents_2027, value = "breeding")
tp_h <- tso_data(geno, pedigree = ped, kernels = "H") # candidates not genotyped
  ↪ yet
```

32. Cover the haplotypes of a genebank or reference panel.

```
tp <- tso_data(geno, haplotypes = list(method = "ld")) # or "fixed", "window", a
  ↪ PLINK file
core <- tso_design(tp, n = 200, criterion = "haplotype") # untargeted: the whole
  ↪ collection
tso_plot(core, "haplotypes")
front <- tso_pareto(tp, n = 200, objectives = c("cdmean", "haplotypes"))
```

33. Would TSOpt have worked in our programme?

```
hist <- read.csv("history.csv") # id, cycle, yield (entry
  ↪ means)
bt <- tso_backtest(geno, hist, n = 0.2, traits = "yield", methods = c("random",
  ↪ "default", "kinship"))
```

```
plot(bt)
```

34. Estimate heritabilities and site correlations before designing.

```
v <- tso_variance(geno, trials, env = "site", traits = "yield") # one row per line and
  ↪ site
tp <- tso_data(geno, envs = v$groups, variance = v) # robust over the
  ↪ estimates
```

35. Testcrosses with two testers, one tester per line.

```
tc <- tso_data(geno, system = "testcross", testers = c("T1", "T2"))
plan <- tso_design(tc, n = 300, max_per_line = 1) # predicts the lines' GCA
```

36. How should 1,200 plots be split between stages?

```
tso_season(tp, budget = 1200, sites2 = 2:5, n_select = 10, stage1 = "both")
```

37. Which crosses for next season?

```
cr <- tso_crosses(geno, parents = elite_lines, pheno = y, cM_per_Mb = 0.5)
head(cr$table, 20)
```

38. Genotyping everyone at high density, or imputing?

```
tso_imputation_plan(geno_hd, budget = 60000, cost_hd = 35, cost_ld = 10, plot_cost = 30)
```

39. Send the plan to the field team and the database.

```
lay <- tso_layout(plan, rows = 20, cols = 15, checks = c("CHK1", "CHK2"))
tso_export_field(lay, "fieldbook.csv", format = "fieldbook", trial = "YT2027")
tso_brapi_write_units(con, lay, study = "st_2027_01")
tso_plan_json(plan, "plan.json"); tso_analysis_script(plan, "analysis.R")
```

40. Near-infrared spectra of every candidate, and transcriptomes of some.

```
nirs <- tso_omics_kernel("nirs.csv", type = "spectra", batch = "instrument")
tx <- tso_omics_kernel("expression.csv", type = "transcriptome", batch = "run", geno =
  ↪ geno)
tp <- tso_data(geno, kernels = list("A", NIRS = nirs, TX = tx)) # unprofiled lines
  ↪ filled from genomics
v <- tso_variance(tp, history, kernels = list("A", NIRS = nirs, TX = tx))
tp <- tso_data(geno, kernels = list("A", NIRS = nirs, TX = tx), variance = v)
tso_omics_plan(geno_pilot, tx_pilot, budget = 20000, profile_cost = 60, plot_cost = 40)
```

14.2 FAQ

Can I use transcriptomes, metabolomes or spectra? Yes: `tso_omics_kernel()` turns them into a kernel next to the genomic one, and lines without a profile are predicted from genomics. Expect better predictions when much of the signal runs through the profiles and the lines you predict are profiled too; do not expect a different training set. `tso_omics_plan()` tells you whether profiling pays at your costs.

Does it work forward in time? In 40 simulated programme histories it did (+0.009 within the cycle, +0.091 when choosing the training set from history). Run `tso_backtest()` on your own history: that is the test that matters for your programme.

Can it handle our whole multi-location network? Up to 8,000 line x trait x environment cells in dense form, and beyond that in factorised form (1,000 lines x 15 sites in about two and a half minutes, 2.7 GB); limits measured in docs/LIMITS.md.

Do haplotypes help? For the reliability of predictions, not in our benchmark: the haplotype kernel did as well as the SNP kernel. For representation, yes: the haplotype criterion covers more of the targets' haplotypes, with second carriers, at no average cost in prediction. Every plan built with `tso_data(haplotypes = )` reports this coverage, so you can check it either way.

Should I model epistasis in the design? Only if you have a reason. In simulations with 30-50% of the genetic variance additive $\times$ additive (720 paired runs, benchmark/kernels_benchmark.R), designing under the epistatic model raised accuracy by 0.003 over the additive default, not significantly, against 0.055 for optimising at all: the default is robust to epistasis. Use kernels to report reliability under the model you believe, to design for breeding values when choosing parents (+0.005 for breeding values), or, with H, for candidates that are not genotyped yet.

Which h^2 should I give? None: the default averages over 0.2, 0.5 and 0.8 and was the most accurate choice in the benchmark. Give one value only if you have a good estimate from previous trials of the same trait in the same material; give your own grid for a trait you know is at one end of the range.

How many lines? Run `tso_sample_size(tp, target = )`: it reads off the sizes that reach 90, 95 and 99% of the accuracy of phenotyping every candidate, from the exact greedy curve, and the plots the design saves over random sampling (Part 10). `tso_diagnose(tp)` shows the same curve more roughly. Size matters more than composition; optimisation buys most when the budget is tight and the target is specific.

I have marker effects but no phenotypes of these lines. Can I protect the genes? Yes: `tso_major_effects(geno, effects)` reads effects from a GWAS, a QTL study, a BayesB model or known genes, matches them to your markers and alleles, and computes each locus's share of genetic variance in your panel. Pass the result to `tso_data(major = )`. Give `h2` and `var_p` for GWAS results or a short list of genes. Do not use rrBLUP or GBLUP marker effects: ridge regression shrinks a major gene and spreads it over the genome, and TSOpt warns when it sees them.

My trait has a major gene. Does the default handle it? No. A genome-wide kernel does not see a single locus, and the default design can leave out every carrier of a rare allele (Part 9: none of eight carriers was chosen). Give the locus, or a scan, to `tso_data(major = )`; the carrier guarantee (`min_carriers`, default 2) is what protects it. Check the Major line of `print(plan)`. Even without `major =`, the Alleles line tells you which target alleles are missing from the training set.

Random or fixed model for the major gene? Fixed when the gene is known and its effect is large and stable (a diagnostic marker); random, with the share from the scan, when the loci come from a scan of limited size. In the introgression demonstration they gave the same accuracy; the carrier guarantee mattered, the model did not.

The verdict is RED. Did I do something wrong? No. The panel is homogeneous enough that any set of that size does about as well, and random sampling is fine. The plan is still returned and is still at least as good as random.

Why did it stratify? The diagnosis found discrete groups, and in strongly structured panels an unstratified optimum can concentrate in one group. `stratify = "none"` switches it off; `group_quota` sets your own allocation.

The coverage line says some targets are unreachable. No candidate is related to them above 0.05, so no training set will predict them. Either accept that they are outside the model's reach, add relatives to the candidates, or use `criterion = "cdmin"` to give them what can be had.

Can I use my own relationship matrix? Yes: `tso_data(kinship = K)`. Any positive semi-definite matrix with unit IDs as dimnames: a pedigree $\bar{A}$, an H matrix, a Gaussian kernel.

Can I use my own criterion? Yes: `criterion = function(ids, data) ...` returning a number to maximise. It is optimised by the GA (Part 4).

It is slow. The default is a fraction of a second on 1,000 lines and about 15 seconds on 2,000. `search = "tabu"`, `criterion = "rscore"` and the GA are slower by design. Very large targets can be subsampled with `max_contrasts`; `n_random = 0` skips the random comparison; restarts run in parallel with `cores =`.

My panel has 20,000 lines and R runs out of memory. The dense engine holds the $N \times N$ kinship (3.2 GB at 20,000 lines). Use `tso_design_large()`: it works on a rank-rank feature matrix from the markers, never forms an $N \times N$ matrix, and gives the same robust CDmean design (identical to the dense greedy design when the features are exact; Part 10). Check `plan$spec$variance_captured` and raise `rank` if it is below 0.7.

Where is the multi-objective Pareto front? In `tso_pareto()` since v0.4 (Part 10). Robust design over several kernels (`kernels =`) and exact per-trait means in multi-trait designs came in v0.5 (Parts 2, 8 and 11).

My genetic correlations come from one small trial. Can I trust the multi-trait design? The default does not trust them fully: `trait_uncertainty = "auto"` averages over five scenarios (heritabilities 40% lower or higher, correlations halved or strengthened). The Scenarios line of the plan shows how much the index reliability depends on them (Part 8).

Should I keep measuring NDVI (or any proxy) that is not in the index? Ask `tso_trait_value(plan)`: it reports the budget share of each trait and the index CDmean lost without it (Part 8).

Some plots failed. Is the design ruined? Look at `plan$selected$value`: most plots are worth about 1% of the CDmean each; if the lost ones were near the top, re-run with `must_include = survivors` and fill the gap (Part 11).

The verdict is GREEN. Would it stay GREEN with another SNP chip? `tso_uncertainty(plan)` bootstraps the markers and answers with an interval (Part 11).

Does TSOpt make the field design too? `tso_layout()` places the plots: balanced blocks, checks, p-rep second plots on the most valuable lines, relatives kept apart. For spatially optimised row-column designs use DiGGER or FieldHub on the list TSOpt chose (Part 11).

Can TSOpt read from BMS or Breedbase? Yes, from any BrAPI v2 server (Part 11).

Does kernels = work for hybrids or multi-environment data? Not yet: it is for line-level data with marker genotypes. Hybrids already carry a dominance (SCA) kernel through `sca_share`.

Can a design include several trials, blocks or covariates? Yes: `fixed =` with column names of `tp$units` or a numeric matrix, handled exactly (Part 2). Force one line into each trial so that the search can open it (the warning in Part 6).

14.3 Crosswalk from other packages

You used	In TSOpt
<code>STPGA::GenAlgForSubsetSelection(P, Candidates, Test, ntoselect, errorstat = "CDMEAN")</code>	<code>tso_design(tp, n = ntoselect, target = Test, candidates = Candidates)</code>
<code>STPGA ... errorstat = "PEVMEAN" / "DOPT" / "AOPT" / "EOPT"</code>	<code>criterion = "pevmean" / "dopt" / "aopt" / "eopt"</code>
<code>STPGA ... errorstat = "CDMAX" / "PEVMAX"</code>	<code>criterion = "cdmin" / "pevmax"</code>
<code>TSDFGS::optTrain(geno, cand, n.train, method = "rScore")</code>	<code>tso_design(tp, n = n.train, goal = "correlation")</code>
<code>TSDFGS::optTrain(..., method = "CD")</code>	<code>tso_design(tp, n, criterion = "cdmean", h2 = 0.5)</code>

You used	In TSOpt
TSDFGS::SSDFGS(geno) (training-set size)	<code>tso_sample_size(tp)</code> (exact curve, <i>n</i> for 90/95/99%)
TSDFGS / TSOFIBG (identifying the best)	<code>tso_topk(plan)</code>
TrainSel(Data, Candidates, setsizes, settypes = "UOS")	<code>tso_design(tp, n = setsizes)</code>
TrainSel ... settypes = "UOMS" (multiset, replicates)	<code>tso_design(tp, n, reps = 3)</code>
TrainSel ... Stat = f (custom criterion)	<code>tso_design(tp, n, criterion = f)</code>
TrainSel ... Stat = CDMEANOPTwithEnvME (multi-environment)	<code>tso_data(geno, envs = , env_cor = , env_noise =)</code> then <code>tso_design(env_target = , max_per_line = , min_per_env =)</code> <code>tso_design(tp, n, fixed = X)</code>
TrainSel ... Stat = CDMEANX (CDmean with fixed effects)	
TrainSel with funOptProp (continuous proportions)	<code>tso_contribution(ebv, K, delta_F =)</code>
TrainSel / STPGA multi-objective (NSGA-II)	<code>tso_pareto(tp, n, objectives =)</code> then <code>tso_pareto_pick()</code>
TrainSel multi-trait CDmean via user code	<code>tso_data(traits = , gcor = , envs =)</code> then <code>tso_design(index =)</code> , robust over trait scenarios
DiGGER, FieldHub, agricolae::design.dau (augmented and p-rep layouts)	<code>tso_layout(plan, rows, cols, blocks, checks, prep)</code> ; export the list to them for spatial row-column designs
AlphaSimR (Gaynor et al. 2021) (simulating whole programmes)	<code>tso_simulate()</code> for the training-set strategy; or design inside an AlphaSimR loop with <code>tso_data(kinship =)</code> and <code>tso_design()</code>
BrAPI clients (QBMS, brapir)	<code>tso_brapi_connect()</code> , <code>tso_brapi_geno()</code> , <code>tso_brapi_pheno()</code> , <code>tso_brapi_trait()</code>
BGLR / rrBLUP::kin.blup(GAUSS = TRUE) (RKHS Gaussian kernel)	<code>tso_design(tp, n, kernels = c("additive", "gaussian"))</code>
BGLR multi-kernel models with omics kernels (GOBLUP, phenomic selection)	<code>tso_data(geno, kernels = list("A", NIRS = tso_omics_kernel(spectra)))</code> , then <code>tso_design()</code> ; <code>tso_omics_plan()</code> for whom to profile
Key-ancestor or imputation-reference choice by contribution	<code>tso_genotype_set(tso_data(pedigree = ped), n)</code>

14.4 Troubleshooting

Symptom	Cause and fix
"IDs in groups/cost not found"	IDs in the genotype file differ (case, whitespace, prefixes) from your metadata; <code>tp\$units\$id</code> shows the IDs as read.
File not recognised	Give <code>format =</code> explicitly; for tables with markers as rows, <code>markers_in = "rows"</code> ; for an ID column not in first position, <code>id_col =</code> .
Every criterion value is NA	The kinship matrix is not positive definite (duplicated lines, all-missing markers); <code>tso_qc()</code> and <code>ridge</code> in <code>tso_kinship()</code> fix most cases.

Symptom	Cause and fix
The plan ignores my target	Targets are excluded from the candidates by definition; check that target names exist in <code>tp\$units\$id</code> .
“Allele class(es) with fewer training carriers than required”	Too few candidates carry an allele the targets carry (or exclude/quota removed them); lower <code>min_carriers</code> , add carriers to the candidates, or accept the shortfall knowingly.
A custom criterion warns about its direction	It scores near-duplicate relatives above every random set; if it measures error, PEV or similarity, give <code>direction = "minimize"</code> .
<code>tso_design()</code> runs out of memory	The panel is too large for an $N \times N$ matrix; use <code>tso_design_large()</code> .
Compiled code fails to install	Windows needs Rtools, macOS the Xcode command-line tools; Linux needs nothing extra.

15 Reference

TSOpt tutorial · Part 15

Reference

Every function, every argument, the glossary and the literature

Function index
 Every exported function, grouped by task, with every argument and one line of meaning.

Glossary
 CD, PEV, contrast, reliability, lambda, entry-mean, heritability, target, candidate, unit.

References
 The papers behind each criterion and algorithm, with DOIs.

Citing
 How to cite the package, this tutorial and the benchmark.

TSOpt is GPL-3 and lives at github.com/whussain2/Training_Optimization_Pnura

15.1 Function index

TSOpt 0.5.0 exports 65 functions. The first table gives each one in a line, grouped by task; the second gives every argument and its meaning. Both are taken from the installed package and its documentation at build time, so they cannot drift from the code.

Table 4: The 65 exported functions of TSOpt 0.5.0.

Task	Function	Purpose
Licence and documents	<code>tso_licence()</code>	Activate your TSOpt licence key
	<code>tso_licence_request()</code>	Request a free TSOpt licence key
Data in	<code>tso_docs()</code>	Open the TSOpt guides, tutorial and manual
	<code>tso_read()</code>	Read genotypes from any common format
	<code>tso_qc()</code>	Quality control and imputation of genotypes
	<code>tso_kinship()</code>	Relationship (kinship) matrices
	<code>tso_kernels()</code>	Genetic kernels: additive, dominance, epistasis, pedigree and single-step
Design	<code>tso_data()</code>	Assemble everything the design engine needs
	<code>tso_write()</code>	Write genotypes to another format
	<code>tso_diagnose()</code>	Should I optimise, and how many units do I need?
	<code>tso_design()</code>	Design a training set: which units to phenotype
	<code>tso_explain()</code>	Explain exactly what a plan did
	<code>tso_criteria()</code>	The criteria TSOpt knows, and which one to use
Major genes	<code>tso_design_large()</code>	Design a training set in a very large panel (no N x N kinship)
	<code>tso_major_scan()</code>	Find major-effect loci before designing a training set
	<code>tso_major_effects()</code>	Major-effect loci from marker effects you already have
	<code>tso_haplotypes()</code>	Haplotype blocks and haplotype alleles

Task	Function	Purpose
Decision tools	<code>tso_sample_size()</code>	How many units do I need to phenotype?
	<code>tso_topk()</code>	Will this training set find the best candidates?
	<code>tso_pareto()</code>	Multi-objective training-set design: the Pareto front
	<code>tso_pareto_pick()</code> <code>tso_contribution()</code>	Take one training set from a Pareto front Optimum contribution selection: which parents, and how much of each
Around the design	<code>tso_trait_value()</code>	What is each trait worth? Proxy and high-throughput traits in multi-trait designs
	<code>tso_uncertainty()</code>	How sure is the plan's advantage? Marker-bootstrap uncertainty
	<code>tso_economics()</code>	How many plots pay off? The economic size of a training set
	<code>tso_layout()</code>	Lay out a training set in the field: joint genetic and field design
	<code>tso_genotype_set()</code>	Whom to genotype: choose the individuals to genotype at high density
	<code>tso_transfer()</code>	Across-programme transfer: can training in one programme predict another?
Programme	<code>tso_variance()</code>	Estimate variance components from historical data
	<code>tso_backtest()</code>	Forward validation: how each method would have done in your programme
	<code>tso_season()</code>	Plan a season across stages: where the plots should go
	<code>tso_crosses()</code>	Predict crosses: progeny mean, progeny variance and usefulness
	<code>tso_imputation_plan()</code>	Plan genotyping, imputation and phenotyping under one budget
	<code>tso_omics_kernel()</code>	Omics kernel: transcriptome, metabolome or spectra
	<code>tso_omics_plan()</code>	Plan omics profiling and phenotyping under one budget
	<code>tso_simulate_omics()</code>	Simulate omics profiles and a phenotype that depends on them
Field, database, software	<code>tso_export_field()</code>	Export a plan or field layout for field tools
	<code>tso_brapi_write_list()</code>	Write a plan to a BrAPI database: a germplasm list
	<code>tso_brapi_write_units()</code>	Write the plots of a plan or layout to a BrAPI study
	<code>tso_manifest()</code>	A manifest of what produced a plan
	<code>tso_verify()</code>	Check that a plan was made from these data
	<code>tso_plan_json()</code>	Write a plan as a versioned JSON file
	<code>tso_analysis_script()</code>	The analysis model that matches the design
	<code>tso_cli()</code>	Run TSOpt from the command line
Databases	<code>tso_brapi_connect()</code>	Connect to a BrAPI v2 server
	<code>tso_brapi_studies()</code>	List the studies on a BrAPI server
	<code>tso_brapi_variantsets()</code>	List the variant sets (genotyping data sets) on a BrAPI server
	<code>tso_brapi_geno()</code>	Read genotypes from a BrAPI v2 server

Task	Function	Purpose
Check	<code>tso_brapi_pheno()</code>	Read phenotypes from a BrAPI v2 server
	<code>tso_brapi_trait()</code>	One trait from a BrAPI server as a named vector
	<code>tso_score()</code>	Score any training set
	<code>tso_compare()</code>	Compare a plan with the alternatives a breeder would otherwise use
Stages, cycles	<code>tso_evaluate()</code>	Evaluate a training set once phenotypes are in
	<code>tso_next_batch()</code>	Choose the next batch of plots from stage-1 results (two-stage design)
	<code>tso_update()</code>	Refresh the training set for a new season
	<code>tso_simulate()</code>	Simulate recurrent genomic selection under different training-set strategies
Simulate	<code>tso_simulate_panel()</code>	Simulate a breeding panel for practice and testing
	<code>tso_simulate_trait()</code>	Simulate a trait for a panel
	<code>tso_simulate_cycles()</code>	Simulate a breeding programme's historical data, cycle by cycle
Show, share	<code>tso_plot()</code>	Publication-ready figures for every TSOpt object
	<code>tso_theme()</code>	The TSOpt figure theme
	<code>tso_colors()</code>	Colours used throughout TSOpt
	<code>tso_save_figure()</code>	Save a figure ready for a journal, a slide or a report
	<code>tso_export()</code>	Export a plan for the field team
	<code>tso_report()</code>	One-page report for a programme manager

Function	Argument	Meaning
<code>tso_licence()</code>	key	The licence key (a text starting with TSO1.), or NULL to show the current licence.
<code>tso_licence_request()</code>	save	Save the key for future sessions (default TRUE).
	name, email, organisation, country	Who the licence is for.
	use	One or two sentences on what you will use TSOpt for (crop, programme).
<code>tso_docs()</code>	open	Open the email in your mail program (default: in interactive sessions).
	which	NULL (list), or one of "tutorial", "manual", "walkthrough", "guide1" to "guide5", "walkthrough-rmd".
<code>tso_read()</code>	x	A file path (or PLINK prefix), a matrix or a data frame.
	format	"auto" (default) or one of "vcf", "hapmap", "plink", "ped", "raw", "dart", "table", "excel", "rds".
	ploidy markers_in	Ploidy. Detected from the data when NULL. For tables only: "auto", "columns" or "rows". With "auto", a table with more rows than columns is assumed to have markers in rows (genotype panels almost always have more markers than lines).

Function	Argument	Meaning
tso_qc()	id_col	For tables only: the column holding line IDs. Detected when NULL (a first column that is not genotype data).
	vcf_field	For VCF only: “GT” (default) or “DS” (dosage).
	binary_as	For tables coded only 0/1: “homozygous” (default; 0/1 is read as 0/2, the usual coding for inbred lines) or “dosage”.
	verbose	Print the read report.
	geno	A tso_genos object (or anything tso_read() accepts).
	maf	Minimum minor-allele frequency (default 0.05).
	call_rate	Minimum call rate per marker (default 0.8).
tso_kinship()	line_call_rate	Minimum call rate per line (default 0.5).
	impute	“mean” (default; adequate for relationship matrices) or “none”.
	base_freq	Allele frequencies of the base population, named by marker. When given they are kept for all later relationship matrices (so relationships stay comparable across breeding cycles). When NULL, frequencies are estimated from this panel and stored.
	verbose	Print what was done.
	geno	A tso_genos object (after tso_qc()) or a dosage matrix.
	method	“additive”, “dominance” or “pedigree”.
	base_freq	Optional named vector of base allele frequencies. The vector stored by tso_qc() carries the counted allele of each marker (as the attribute “counted”), and genotypes are re-oriented to it, so frequencies stay about the same allele across cycles even for letter-coded files (HapMap, allele-pair tables), which are coded by each file’s own minor allele.
	pedigree	For method = “pedigree”: a data frame with columns id, parent 1, parent 2 (unknown parents as NA or 0).
	scale	If TRUE (default), rescale so the mean diagonal is 1. The heritability used by the design engine is then the entry-mean heritability, whatever the ploidy or inbreeding of the panel.
	ridge	Small value added to the diagonal for numerical stability.
tso_kernels()	weights	Optional marker weights for the additive kernel, named by marker (unnamed: in column order): $G = Z W Z' / (k \sum w p(1-p))$. Use it to up-weight markers near known QTL (e.g. $-\log_{10} p$ from tso_major_scan()), or give weight 0 to drop markers (e.g. major loci modelled separately through tso_data(major =)). Markers without a weight get weight 1.
	geno	Genotypes (tso_genos, a file, or a tso_data with genotypes).

Function	Argument	Meaning
tso_data()	pedigree	Pedigree: data frame with columns id, parent 1, parent 2 (unknown parents NA or 0).
	which	Kernels to build (see Details). Default: “A”, plus “D” and “AA” for diploid marker data, or “A_ped” for a pedigree only.
	bandwidth	Multiplier of the median distance for the Gaussian and exponential kernels (default 1).
	blend	Weight of A22 in the blended G used by “H” (default 0.05).
	base_freq haplotypes	Optional base allele frequencies for “A” and “D”. For “HAP”: a tso_haplotypes() object, or a list of its arguments (e.g. list(method = “fixed”, window = 3)). Default: LD blocks.
	geno	Genotypes: a file path (any format tso_read() accepts), a tso_genotype object, or a matrix.
	kinship	Optional relationship matrix to use instead of (or without) genotypes.
	pedigree	Optional pedigree (id, parent 1, parent 2) used when no genotypes or kinship are given, and by the pedigree and single-step kernels (kernels =).
	system	One of “inbred”, “clonal”, “outcross”, “genebank”, “hybrid”.
	groups	Optional group labels per line (e.g. subpopulations, programmes), named by line ID.
	families	Optional family labels per line, named by line ID.
	cost	Optional cost per unit (named by unit ID, or in unit order). Defaults to 1 for every unit.
	ploidy	Ploidy, if it cannot be detected from the data.
	qc	Run tso_qc() with its defaults (TRUE) or pass a list of arguments for it, or FALSE to skip.
	base_freq	Base-population allele frequencies (see tso_qc()).
	dominance	Add a dominance kernel (TRUE by default for diploid clonal crops and for hybrids’ SCA).
	dom_share	Share of genetic variance given to dominance when dominance = TRUE (default 0.2).
	pools	For system = “hybrid”: a list of two ID vectors (female pool, male pool), or of three or more heterotic groups (named); crosses are then made between the groups in cross_groups.
	sca_share	For hybrids: share of hybrid genetic variance due to SCA (default 0.2; GCA of the two pools share the rest equally).
	crossable	For hybrids: parents that may be used in training crosses (default: all). Hybrids involving other parents can be targets but not training units.

Function	Argument	Meaning
	cross_groups	For three or more heterotic groups: the pairs of groups that are crossed, as a list of length-2 vectors of group names (default: every pair). The genetic value of the single cross $i \times j$ is $a_i + a_j + s_{ij}$ with covariance $\sigma^2 a (K_{ik} + K_{il} + K_{jk} + K_{jl}) + \sigma^2 s (K_{ik}K_{jl} + K_{il}K_{jk})$.
	testers	For system = "testcross": the tester IDs. Testers that are genotyped get their genomic relationships (for the SCA part); otherwise they are taken as unrelated.
	envs	Optional environment names, for line x environment designs.
	env_cor	Genetic correlation between environments: a single value, a correlation matrix, or a genetic covariance matrix (environment-specific genetic variances on its diagonal), e.g. estimated from earlier trials.
	env_noise	Relative residual variance of each environment (named by environment; default 1 everywhere). A noisy site (2) is worth half a plot at a clean one (1): heteroscedastic multi-environment designs.
	traits	Optional trait names, for multi-trait (line x trait) designs: a unit is then one trait measured on one line, and <code>tso_design()</code> can optimise a selection index with <code>index =</code> .
	trait_h2	Entry-mean heritability of each trait (same order as traits).
	gcor	Genetic correlation matrix between traits (default: identity).
	trait_cost	Cost of measuring each trait on one line (default 1).
	plot_cost	Cost of growing a line at all, paid once per line the first time any trait is measured on it (default 0).
	kernels	Genetic kernels to combine: kernel names ("A", "D", "AA", "AD", "DD", "gaussian", "exponential", "A_ped", "D_ped", "AA_ped", "H"; see <code>tso_kernels()</code>), a <code>tso_kernels()</code> object, a <code>tso_omics_kernel()</code> object (transcriptome, metabolome, spectra; the genomic kernel is added, and genotyped lines without a profile are predicted from genomics), or a named list of your own matrices (name the additive one "A"), or a list mixing all of these (<code>list("A", "AA", NIRS = om)</code>). The relationship matrix becomes $K = \sum_k w_k K_k$. Replaces kinship and dominance.

Function	Argument	Meaning
	<code>kernel_share</code>	Share of genetic variance of each kernel (named, or in the order of kernels), or “robust” (default with several kernels): the design is then averaged over non-additive shares of 10%, 30% and 50% (split equally over the non-additive kernels), and the plan reports the 30% centre. See Genetic kernels.
	<code>haplotypes</code>	Haplotype blocks for the haplotype-coverage criterion, the per-block coverage report of every plan and the “HAP” kernel: TRUE (LD blocks), a list of <code>tso_haplotypes()</code> arguments (e.g. <code>list(method = “fixed”, window = 3)</code>), or a <code>tso_haplotypes</code> object.
	<code>variance</code>	A <code>tso_variance()</code> fit of historical data. It supplies what would otherwise be assumed: the heritabilities and genetic correlations of traits, the genetic covariance and residual variances of envs, or the shares of kernels; and, for robust designs, scenarios drawn from the estimates’ sampling distribution (for the heritability, its 10%, 50% and 90% quantiles replace $h^2 = 0.2, 0.5, 0.8$).
	<code>implicit</code>	Keep the unit covariance of traits and environments in factorised form ($\mathbf{E} \otimes \mathbf{G}_T \otimes \mathbf{K}$, never formed as one matrix). NULL (default) switches it on above 8,000 line x trait x environment cells; designs are identical to the dense form, and 1,000 lines x 15 environments fit in memory. Not for hybrids yet.
	<code>major</code>	Major-effect loci to protect in the design: marker names, a <code>tso_major_scan()</code> result (from phenotypes), a <code>tso_major_effects()</code> result or a table of marker effects you already have (from a GWAS, QTL study or earlier model: no phenotypes needed), a <code>tso_haplotypes()</code> object, or <code>list(markers = , haplotypes =)</code> . See Major genes and haplotypes.
	<code>major_share</code>	Share of genetic variance at the major loci together (for <code>major_model = “random”</code>). Default: the scan’s or the marker effects’ estimate, or 0.3.
	<code>major_model</code>	“random” (default): the relationship matrix becomes $(1 - s) K_{\text{poly}} + s K_{\text{major}}$, so the design values sampling every allele of the major loci. “fixed”: the loci are fixed effects in the model (Bernardo 2014); <code>tso_design()</code> then optimises the polygenic CDmean with the exact projection for them.
	<code>min_carriers</code>	Minimum number of training units carrying each allele (haplotype) of the major loci that the targets carry (default 2; 0 turns the constraint off). Can be changed per design in <code>tso_design()</code> .
	<code>verbose</code>	Print what was done.

Function	Argument	Meaning
tso_write()	x	A tso_geno object, or a dosage matrix (lines x markers).
	file format	Output file name. “auto” (from the extension) or an explicit format name.
tso_diagnose()	data	A tso_data object or anything tso_data() accepts.
	n	The number of units you plan to phenotype (used for the verdict). Default: 20% of the candidates.
	target	Optional target units (IDs).
	h2	Assumed entry-mean heritability.
	sizes	Training-set sizes for the size curve.
	n_random	Random sets drawn at each size.
tso_design()	seed	Random seed.
	data	A tso_data object, or anything tso_data() accepts (genotype file, tso_geno object, marker matrix).
	n	Number of units to phenotype.
	budget	Alternatively, a budget in the same currency as the unit costs given to tso_data(). Units are then chosen by gain per unit cost.
	target	Units you want to predict (IDs). NULL means untargeted: the whole pool.
	goal	What you are trying to do; see the table above.
	criterion	Expert override of the criterion (see above).
	select_top	For goal = “gain”: the fraction you will select (default 0.1).
	h2	Assumed entry-mean heritability. The default (NULL) is the robust CDmean design: the criterion is averaged over $h2 = 0.2, 0.5$ and 0.8 , which won the head-to-head benchmark on six public panels (see benchmark/). It falls back to $h2 = 0.5$ when an expert search, a criterion other than CDmean / PEVmean / CDpop, or several traits are used. Give one value to design for a known heritability, or several values for your own prior grid.
	candidates	Units that may be phenotyped. Default: every unit that is not a target (for hybrids, only crosses between crossable parents).
	must_include	Units that must be in the set (checks, parents, lines already planted). They count towards n and budget.
	exclude	Units that cannot be phenotyped (no seed, lost, off-type).
	history	Units already phenotyped in earlier seasons. They inform the design at no cost and are not counted in n.
	history_weight	Weight of historical records relative to new ones (1 = as informative as a new plot; smaller for older or less related seasons).

Function	Argument	Meaning
	stratify	"auto" (default): stratify when the panel has discrete groups; "groups": always stratify by the groups in data; "none".
	group_quota	Optional named vector of the number of units per group.
	max_per_family, max_per_parent, max_per_env reps	Optional caps. Maximum number of plots per unit (replicates). With reps > 1, the engine may put several plots on the same unit when that is more informative.
	index	For multi-trait data (see traits in tso_data()): the selection-index weights, named by trait, per genetic standard deviation. The design then maximises the reliability of the index (index CDmean), or the expected gain on the index with goal = "gain".
	predict	For hybrids: which untested hybrids matter most: "all", "H2" (both parents tested), "H1" (one new parent), "H0" (both parents new).
	search	"greedy+exchange" (default), "greedy", or "swap" (random swaps; used automatically for criteria without a fast update).

Function	Argument	Meaning
	fixed	Fixed effects of the prediction model. "intercept" (default) uses the exact fixed-intercept form of CDmean / PEVmean / CDpop (Rincen's "exact" CDmean); "none" treats the data as having a known mean. For any other fixed effects give the names of columns of <i>dataunits</i> (factors become indicator columns, e.g. "group" trial, check or covariate effects. <i>Environmentmeans</i> (major environment data) and <i>fixedmajorloci</i> (<i>tso_data</i>(major_m "fixed")) are added automatically. Every fixed effect is handled by one engine (the contrasts stay on the genetic values), at no cost for 3 for pools up to 1,500 units, 1 above (exchange rarely adds subsetting biases to the search towards the sampled targets. the minimum number of training units carrying each allele the value given to <i>tso_data</i>(2).0 switches the constraint to <i>environmentdata</i> : what the design should predict well. "mean" (default) : the lines' genetic values averaged over the environments (per every line x environment cell; or the name of one environment (or multi — trait) data : the maximum number of environments (traits) in which <i>environmentdata</i> : the minimum number of units in every environment (as in 2 per environment. <i>Environmentmeans</i> are fixed effects step search would never open a new environment; two plots startrunsoftheexchange/tabusearchfromperturbedstates supplied criterion function : "maximize" (default) or "minimize". TSOpt warns when <i>traitdata</i> : "auto" (default) hedged the design over uncertain trait parameters the given heritabilities and genetic correlations, heritability fit on a model of gene action. Line — level data with genotypes only. 'value' What the design predicts "testcross") the default is "gca", the lines' general combining additive kernels (<i>tso_data</i>(kernels =)) : "total" (default), the total genetic value (the worth of a variety use it to choose parents). The data are modelled with the very kernel "robust" in <i>tso_data</i>() the design averages over the share of "Recommended", "Specialist", "Classical", "Model-free" or "Baseline". 'tso_design_large'() 'geno' At <i>tso_genotype</i> the whole pool. 'h2' Assumed entry — mean heritability. NULL (default) is robust CDmean, average 0.2, 0.5 and 0.8, as in <i>tso_design</i>(). Give one value, or several every line that is not a target. 'must_include' Lines that must be in <i>atso_genotype</i>, <i>atso_data</i> with genotypes, or anything to <i>read</i>() a line IDs, e.g. from a previous trial or the stage — 1 training set. 'maf' Minimum minor — allele frequency among the phenotyped lines. 'n_pc' Principal component (the kinship already models structure). 'threshold' Significance level on log10(p) scale. Default : Bonferroni at 5 'clump_r2' LD threshold for clumping. <i>atso_genotype</i>, <i>atso_data</i> with genotypes, or anything to <i>read</i>() a

Function	Argument	Meaning
tso_variance()	K	Relationship matrix with row and column names covering the IDs of ebv (e.g. <code>dataK_lines</code>). `max_oancestry` Upper limit C for the IDs) with values "M"/"F"; each sex then contributes one half of the parent takes part in about 2N _c of the N matings, and with trait tso _p lan. `reference` The trait the exchange rate is set to the trait with the largest index weight). `redesign` If TR run the design without each trait (same budget or size, same bootstrap replicates (default 50)). `n_r, andom` Random trait (1), per cycle. `select_op` Fraction of the targets selected sets sizes at which random sampling is evaluated (passed to about 30 sizes spread over the whole candidate pool). `fixe up, genotyping already budgeted, analysis`. Shift then set value per genetic SD). `horizon` Number of selection cycles, number of rows and columns of plots. `blocks` Number of rounds (sqrt(n/10)) for n entries (at least 1; at least 2 with pr 0). `checks` Optional check IDs. They need not be in the plot (an augmented design). A single number or one value per checker annealing iterations (default 20,000). `seed` Random seed, kinship = or low – density genotypes), anything tso _a ata() accepts, or a pedigree all, e.g. excluding animals or plants with no DNA left). `n_r, andom` Random set of the same size use IDs, a logical vector over units, or group name(s) matched
	to	Lines of programme B, in the same forms.
	n	Number of training units (plots) in each design.
	n_local	Maximum number of local bridge lines from B in the bridge design (default 0: no bridge).
	cost_local	Cost of one plot in B relative to one plot in A (default 1).
	h2	Assumed heritability; NULL averages over 0.2, 0.5 and 0.8.
	geno	Genotypes (tso_geno, a file) or a tso_data, whose additive relationship matrix is used (unless kernels).
	pheno	Data frame with an id column (line IDs) and the trait column(s); with env, one row per line and environment.
	traits	Trait column(s). Default “y”.
	env	For multi-environment data: the environment column.
tso_backtest()	kernels	For kernel shares (one trait): kernel names, a tso_kernels() object or a named list of matrices.
	draws	Scenarios drawn from the sampling distribution (default 5).
	maxit, tol	Iteration limit and convergence tolerance on the log-likelihood.
	verbose	Print iterations.
	geno	Genotypes of the lines (tso_geno, a file, or a tso_data).

Function	Argument	Meaning
tso_season()	pheno	A data frame with one row per line: id, cycle (year, cohort; numeric or sortable) and the trait column(s).
	n	Lines to choose per cycle: a number, or a fraction (< 1) of the candidates.
	methods	Methods to compare: a character vector or a named list. Each element is “random”, “default” (tso_design() defaults), a criterion name from tso_criteria(), a list of tso_design() arguments (e.g. list(criterion = “cdmean”, h2 = 0.3)), or a function f(data, n, candidates, target, history) returning line IDs.
	traits	Trait column(s) in pheno (default “y”).
	scheme	“within_cycle” or “from_history”.
	min_history	Cycles of history needed before a cycle is tested (default 1).
	window	Use only the last window cycles as history (default all).
	h2	Heritability for GBLUP; default REML on each training set.
	n_random	Repeats of random methods.
	seed	Random seed.
	verbose	Print progress.
	data	A tso_data object (line level) or genotypes.
	budget	Plots available (in units of plot_cost).
	n1, k1, sites2	Values to try for the stage-1 training-set size, the lines advanced, and the stage-2 sites (all combinations that fit the budget are evaluated). Defaults cover a useful range.
	sites1, reps1, reps2	Stage-1 sites and replicates, stage-2 replicates.
	n_select	Lines selected at the end (varieties or new parents).
	h2_plot	Heritability of a single plot.
	ge_ratio	Genotype x environment variance relative to the genetic variance.
	plot_cost	Cost of one plot (the budget is in the same units).
tso_crosses()	stage1	“tsopt” (default), “random”, or “both”: TSOpt and random stage-1 training sets on the same simulated seasons, with the paired difference (what the design adds within a season).
	reps	Simulated seasons per allocation.
	seed	Random seed.
	verbose	Print progress.
	geno	Genotypes of the candidate parents (homozygous lines) and, when effects are estimated, of the phenotyped lines.
	parents crosses	IDs of the candidate parents (default: all lines). Optional two-column matrix or data frame of crosses to evaluate (default: every pair of parents).

Function	Argument	Meaning
<code>tso_imputation_plan()</code>	<code>effects</code>	Marker effects (named numeric vector, per copy of the counted allele), or NULL to estimate them from pheno.
	<code>pheno</code>	Named numeric vector of phenotypes (names = line IDs), used when effects is NULL.
	<code>cM_per_Mb</code>	Map density when the map has no cM column.
	<code>progeny</code>	“DH” (doubled haploids from the F1, default) or “RIL”.
	<code>select</code>	Fraction of progeny selected (default 0.1), which sets the selection intensity i .
	<code>accuracy</code>	Accuracy of selection among the progeny (default 1: on true genetic values); $U = \frac{1}{i} + i \cdot \text{accuracy}$.
	<code>geno</code>	High-density genotypes of the candidates (<code>tso_geno</code> , a file, or a <code>tso_data</code>).
	<code>budget</code>	Total budget, in the units of the costs.
	<code>cost_hd</code> , <code>cost_ld</code>	Cost of one high- and one low-density genotype.
	<code>plot_cost</code>	Cost of one phenotyping plot.
<code>tso_omics_kernel()</code>	<code>n_ref</code> , <code>ld_markers</code>	Values to try (all combinations are evaluated).
	<code>reference</code>	“coverage” (default) or “random” reference lines.
	<code>target</code>	Lines to predict (default: the lines not phenotyped).
	<code>h2</code>	Heritability for the design and its score.
	<code>k</code> , <code>window</code>	Neighbours and window size (in markers) of the imputation.
	<code>seed</code>	Random seed.
	<code>x</code>	Profiles: a numeric matrix or data frame with one row per sample (line IDs as row names, or in column id), or a CSV/TSV file of the same.
	<code>id</code>	Name of the ID column (default: “id”, else row names, else the first column if it is not numeric).
	<code>type</code>	“auto”, “transcriptome”, “metabolome”, “spectra” or “other”. “auto” recognises spectra (many columns named by wavelength whose neighbours are highly correlated).
	<code>batch</code>	Batch of each sample (vector in row order, or the name of a column of <code>x</code>).
	<code>covariates</code>	Other sample covariates to adjust for (a data frame in row order, or names of columns of <code>x</code>).
	<code>transform</code>	“auto”, “none”, “log2” or “log”.
	<code>spectra</code>	Pre-treatment of spectra: “snv_d1” (default), “snv”, “d1” or “none”.
	<code>window</code>	Savitzky-Golay window (odd number of points).
	<code>method</code>	“linear” (default) or “gaussian”.
	<code>bandwidth</code>	Multiplier of the median distance for “gaussian”.
	<code>top</code>	Keep the top most variable features (default: all).
	<code>max_missing</code>	Largest share of missing values a feature may have.
	<code>outlier_z</code>	Robust z-score above which a line is flagged.

Function	Argument	Meaning
tso_omics_plan()	geno	Optional genotypes (tso_geno, a file or a tso_data) for the heritability and agreement checks.
	fill	Predict the omics relationships of genotyped lines without a profile from their genomic relationships in tso_data() (default TRUE).
	blend	Weight of the genomic relationships blended into the profiled block before filling (keeps it invertible; default 0.05).
	name	Name of the kernel (default from type: "TX", "MET", "NIRS" or "OMICS").
	geno	Genotypes of the pilot lines (tso_geno, a file or a tso_data).
	omics	A tso_omics_kernel() of the same lines (all profiled).
	budget	Total budget, in the units of the costs.
	profile_cost,	Cost of one omics profile and of one
	plot_cost	phenotyping plot.
	n_profile	Numbers of lines to profile (default 25%, 50%, 75%).
tso_simulate_omics()	share	Share of the omics kernel in the genetic + omics variance (e.g. from tso_variance()).
	h2	Heritability (signal over signal plus noise).
	strategy	One or more of "coverage", "phenotyped", "targets", "random".
	target	Lines to predict (default: the lines not phenotyped).
	seed	Random seed.
	panel	A tso_geno (e.g. from tso_simulate_panel()).
	features	Number of features.
	h2_features	Range of feature heritabilities.
	omics_share	Share of the phenotype's signal that runs through the profiles.
	h2	Heritability of the phenotype (signal over signal plus noise).
tso_export_field()	causal	Share of features that affect the phenotype.
	type	"other" (a matrix of standardised features) or "spectra" (curves over 400 wavelengths: every feature an absorption band, plus light scatter, baseline offsets and instrument noise).
	seed	Random seed.
	x	A tso_plan or tso_layout.
tso_brapi_write_list()	file	Output file (.csv). NULL returns the table only.
	format	"fieldbook", "fieldhub", "breedbase" or "csv".
	trial	Trial or study name, used in plot names and IDs.
	con	A connection from tso_brapi_connect().
	x	A tso_plan, a tso_layout, or a character vector of germplasm names or IDs.
	name	List name.
	description	Optional list description.
	resolve	Match germplasm names to germplasmDbIds first (default TRUE); FALSE sends the IDs as they are.

Function	Argument	Meaning
tso_brapi_write_units()	dry_run	Return the request body without sending it.
	con	A connection from tso_brapi_connect().
	x	A tso_layout (with field positions) or a tso_plan.
	study	The studyDbld of the study (trial at one location).
tso_manifest() tso_verify()	prefix	Prefix of the observation-unit names.
	resolve	Match germplasm names to germplasmDblds first (default TRUE); FALSE sends the IDs as they are.
	dry_run	Return the request body without sending it.
	plan	A tso_plan.
tso_plan_json()	plan	A tso_plan (or a manifest from tso_manifest() / a plan file from tso_plan_json()).
	data	The tso_data the plan should come from.
	file	A tso_plan.
tso_analysis_script()	plan	Output file (.json); NULL returns the JSON text.
	file	A tso_plan.
	engine	Output .R file; NULL returns the code.
tso_cli()	trait	"sommer", "asreml" or "rrBLUP".
	args	Name of the trait column in the phenotype file.
	args	Character vector of arguments (as from commandArgs(TRUE)).
tso_brapi_connect()	url	Base URL of the server. /brapi/v2 is added when missing, so " https://test-server.brapi.org " and " https://test-server.brapi.org/brapi/v2 " are the same.
	token	Optional access token. It is sent as the header Authorization: Bearer .
	page_size	Records requested per page (BrAPI pageSize).
	timeout	Seconds to wait for one request, and for an asynchronous search to finish.
	retries	How many times a failed request (network error, HTTP 429 or 5xx) is retried, with a growing pause between tries.
tso_brapi_studies()	verbose	Print what was found.
	con	A connection from tso_brapi_connect().
	...	Query parameters for /studies, e.g. commonCropName = "Rice", programDbld = "p1" or trialDbld = "t1".
tso_brapi_variantsets()	con	A connection from tso_brapi_connect().
	...	Query parameters for /variantsets, e.g. studyDbld = "s1".
tso_brapi_geno()	con	A connection from tso_brapi_connect().
	variant_set	The variantSetDbld to read. When NULL and the server has exactly one variant set, that one is used.
	germplasm	Optional germplasm names or ids (or call set names or ids) to keep; all lines are kept when NULL.
	...	Further query parameters passed to the genotype endpoint, e.g. expandHomozygotes = TRUE.
	method	"auto", "allelematrix" or "calls".

Function	Argument	Meaning
tso_brapi_pheno()	names	Row names: “germplasm” (germplasm name, the default), “germplasmDbld” or “callset” (call set name). Use the same choice as id in tso_brapi_pheno() so that genotypes and phenotypes match.
	ploidy	Ploidy, if you want to fix it instead of reading it from the genotype strings.
	verbose	Print a short reading log.
	con	A connection from tso_brapi_connect().
	study	Optional studyDbld(s) (or study names).
tso_brapi_trait()	trait	Optional observation variable name(s) or id(s).
	...	Further query parameters for /observations, e.g. observationLevel = “plot” or seasonDbld = “2023”.
	germplasm	Optional germplasm names or ids to keep.
	id	What identifies a line in column id (and in the means): “germplasm” (germplasm name, the default) or “germplasmDbld”. A warning is given when one name covers several germplasm ids, because their observations would then be pooled.
	verbose	Print a short summary.
tso_score()	con	A connection from tso_brapi_connect().
	study	Optional studyDbld(s).
	trait	Observation variable name or id. May be NULL when the selection contains only one trait.
	...	Passed to tso_brapi_pheno().
	data	A tso_data object (or anything tso_data() accepts).
tso_compare()	train	Training units (IDs or indices). May repeat (replicates).
	target	Target units; NULL = all units not in train... for untargeted scoring, all units.
	h2	Assumed entry-mean heritability.
	criteria	Any of “cdmean”, “cdmean_exact” (intercept projected out), “pevmean”, “dopt”, “rscore”.
	value	“total” (default) or “breeding”: score the reliability of total genetic values or of breeding values (the additive part), for data with non-additive kernels (tso_data(kernels =)).
tso_evaluate()	plan	A tso_plan from tso_design().
	methods	Any of “random”, “stratified”, “kinship”, “pam”.
	others	A named list of extra training sets (unit IDs), e.g. the list you would have planted anyway.
	n_random	Number of random (and stratified) draws.
	seed	Random seed.
tso_evaluate()	x	A tso_plan, or a tso_data object together with train.
	y	Named numeric vector of phenotypes (names = unit IDs). Training units must be present; targets are evaluated where present.
	train	Training units, when x is a tso_data object.

Function	Argument	Meaning
tso_next_batch()	target	Units to evaluate on. Default: the plan's targets (or all units not in training).
	truth	Optional named vector of true genetic values (simulations); when given, metrics are computed against it instead of y.
	h2	Heritability for GBLUP. NULL (default) estimates it by REML from the training data (bounded to 0.02-0.95).
	top	Fraction selected, for NDCG and realised gain (default 0.1).
	forward	Is this forward prediction (later season)?
	plan	The stage-1 tso_plan from tso_design().
	y	Named vector of stage-1 phenotypes (names = unit IDs). May also contain earlier data.
	n	Number of plots in the next batch.
	select_top	Fraction you will finally select (default 0.1).
	pool	Units you will select among. Default: the plan's targets if it was targeted, otherwise every unit.
	candidates	Units that may get a plot in the next batch. Default: every unit in the pool or the plan's candidates.
	retest	Allow a second plot on units already phenotyped (as when the best lines of a preliminary trial are tested again). Default TRUE.
	h2	Heritability of one plot. NULL (default) estimates it from y by maximum likelihood.
	n_screen	Candidates scored exactly at each step (the rest are screened out with a fast boundary score).
tso_update()	n_quad	Gauss-Hermite quadrature nodes.
	n_draws	Posterior draws for the probability of being selected.
	seed	Random seed.
	previous geno	A tso_plan from an earlier season. Genotypes of the whole current panel (old and new lines), in any format tso_read() accepts. Alternatively give data.
tso_simulate()	data	A tso_data object for the current panel.
	n	Number of new units to phenotype.
	history_weight	Weight of last season's records (default 0.8).
	...	Other arguments passed to tso_design().
	strategies	A named list: each element is a strategy name or a function (see Details).
	cycles	Number of selection cycles.
	reps	Number of independent replicates.
	n_candidates	Doubled haploids per cycle.
	n_crosses	Crosses per cycle.
	n_train	Candidates phenotyped per cycle.
	n_parents	Parents selected per cycle.
	n_founders	Founder lines.
	markers	Number of genotyped markers.
	n_qtl	Number of QTL (not genotyped).
	chromosomes	Number of chromosomes.

Function	Argument	Meaning
tso_simulate_panel()	h2	Heritability of one plot in the founder generation. The error variance is then held fixed, so heritability falls as selection uses up genetic variance, as in a real programme.
	window	Cycles of training data kept (Inf = cumulative).
	seed	Random seed.
	verbose	Print progress.
	n	Number of lines.
	markers	Number of markers.
	groups	Number of subpopulations.
	fst	Differentiation between subpopulations.
	founders	Founders per subpopulation.
tso_simulate_trait()	chromosomes	Number of chromosomes (markers are spread evenly).
	type	“dh” (doubled haploids, dosages 0/2) or “outbred”.
	seed	Random seed.
	x	A tso_genotype or tso_data object.
	h2	Entry-mean heritability.
	n_qtl	Number of polygenic QTL (use a small number for an oligogenic trait).
tso_simulate_cycles()	major	Number of major QTL, or the names of the markers to use as major QTL (default 0: none). Randomly chosen major QTL have a minor-allele frequency of at least 0.1.
	major_share	Share of genetic variance explained by the major QTL together (default 0.3).
	seed	Random seed.
	cycles	Number of cycles (years, cohorts).
	n_candidates	Lines per cycle.
	n_crosses	Crosses per cycle.
	n_parents	Lines selected as parents for the next cycle.
	n_founders	Founder lines.
	markers, n_qtl	Genotyped markers and (ungenotyped) QTL.
	chromosomes	Number of chromosomes.
tso_plot()	h2	Heritability of one line’s phenotype in the founder generation.
	year_sd	Standard deviation of the year effects, in genetic SD.
	seed	Random seed.
	x	A result object from TSOpt: a plan, diagnosis, comparison, evaluation, data, batch, simulation, Pareto front, sample-size curve, top-k analysis, contribution plan or major-locus scan.
	type	Figure type (see table). NULL gives the default for x.
	style	Figure style passed to tso_theme().
	title	Optional title; NULL keeps the automatic one, FALSE drops it.
	...	Passed to the individual figure functions.

Function	Argument	Meaning
tso_theme()	style	“publication” (default; 8 pt base text, for figures saved at 85-180 mm), “presentation” (larger text and lines for slides) or “course” (PlantNura teaching style).
tso_colors()	base_size	Override the base text size (points).
	base_family	Font family (default: the device’s sans serif).
tso_save_figure()	which	“roles” (default) or “groups”.
	n	For “groups”: how many colours.
tso_export()	plot	A ggplot (or patchwork) object.
	file	File name. With an extension, that format is written; without one, every format in format is written.
tso_report()	journal	A journal preset (see table).
	width	“single”, “onehalf”, “double” or a number in mm.
tso_report()	height	Height in mm (default: 0.72 x width, or 0.85 x width for multi-panel figures).
	format	One or more of “pdf”, “svg”, “eps”, “png”, “tiff”. Default: c(“pdf”, “png”).
tso_report()	dpi	Resolution for raster formats (default 600).
	plan	A tso_plan.
tso_report()	file	Output file (.csv). If NULL, nothing is written.
	what	“selected” (default) or “all” units with their role and expected reliability.
tso_report()	diagnosis	Optional tso_diagnosis to include (size curve and structure).
	comparison	Optional tso_comparison to include; computed when NULL.
tso_report()	title	Report title.
	author	Author line.

15.2 Glossary

Term	Meaning
Candidate	A unit that may be phenotyped.
Target	A unit whose genetic value is to be predicted; targets are never phenotyped.
Training set	The candidates chosen to be phenotyped.
Unit	Anything one plot can measure: a line, a hybrid, a line in an environment, a trait on a line.
K	The relationship (kinship) matrix between units, scaled to a mean diagonal of one.
λ	$(1 - h^2)/h^2$, the ratio of residual to genetic variance; the noise of one plot.
Entry-mean heritability	The heritability of the phenotype on the unit as it will be recorded (a plot mean over reps, a family mean).
A (<i>S</i>)	The prediction error variance operator after phenotyping <i>S</i> ; the posterior covariance of the genetic values.
PEV	Prediction error variance of a contrast, $\mathbf{c}'\mathbf{A}\mathbf{c}$.

Term	Meaning
CD	Coefficient of determination of a contrast, $1 - \mathbf{c}'\mathbf{Ac}/\mathbf{c}'\mathbf{Kc}$; the squared expected accuracy.
Contrast	A vector $\mathbf{c}$ defining what is predicted: a line minus the mean, a family mean, a hybrid, an index.
CDmean	The mean CD over the target contrasts (each target minus the target mean).
Robust CDmean	CDmean averaged over $h^2 \in \{0.2, 0.5, 0.8\}$; the default.
Flat-prior intercept	Treating the fixed intercept as a random effect with a huge variance, $\mathbf{K} + c\mathbf{J}$, so that the projected criterion keeps the rank-one form.
Rank-one update	The Sherman-Morrison step $\mathbf{A} \leftarrow \mathbf{A} - \mathbf{aa}'/(\lambda + a_{jj})$ that adds one plot without inverting anything.
ROTS	Rank-One Tabu Search, TSOpt's search algorithm.
Reliability	The CD of a unit's own genetic value under the plan (identity contrast).
Coverage	The relationship between a target and its closest training relative.
Predictability divide	The set of candidates whose reliability under the plan is too low for any selection decision.
Untargeted / targeted	The target is the whole pool / a named set.
Knowledge gradient	The expected improvement in the selection decision from one more observation; the rule of <code>tso_next_batch()</code> .
General fixed effects	Trials, checks, covariates, environment means or major loci in $\mathbf{X}$, handled exactly by the flat-prior kernel $\mathbf{K} + c(\mathbf{11}' + \mathbf{XX}')$ with contrasts from $\mathbf{K}$.
Major locus	A locus explaining a large share of the genetic variance; found by <code>tso_major_scan()</code> or from known effects by <code>tso_major_effects()</code> , protected by <code>tso_data(major =)</code> .
Allele class	One allele (or haplotype) of a protected locus; each class the targets carry needs <code>min_carriers</code> training carriers.
Allele coverage	The share of the targets' marker alleles and haplotypes that also occur in the training set; reported by every plan.
Operating curve	Expected accuracy against training-set size, relative to phenotyping every candidate; <code>tso_sample_size()</code> .
Pareto front	The sets that no other set beats on all objectives at once; <code>tso_pareto()</code> . The knee is the balanced compromise.
Optimum contribution	Parent contributions maximising the offspring's mean breeding value at a fixed rate of inbreeding; <code>tso_contribution()</code> .
Low-rank engine	The feature-space engine of <code>tso_design_large()</code> , $\mathbf{K} \approx \mathbf{FF}' + \text{diag}(\mathbf{e})$, with no $N \times N$ matrix.
First-unit trap	The first unit at a level of a fixed factor (trial, site, trait) only estimates that level's mean, so a one-step search never opens the level; TSOpt opens every level with two units.
Trait scenarios	The five sets of trait heritabilities and genetic correlations the robust multi-trait design averages over (<code>trait_uncertainty = "auto"</code>).
Proxy trait	A cheap trait with index weight 0 (NDVI, canopy temperature, NIRS) measured for the information it carries about the index traits.
Exchange rate	Measurements of a trait that carry the information of one measurement of the reference trait; <code>tso_trait_value()</code> .
Plot value	The exact CDmean lost when one plot is dropped from the plan (a rank-one downdate); <code>plan\$selected\$value</code> .
Marker bootstrap	Resampling markers to rebuild $\mathbf{K}$ and re-score the plan and random sets; <code>tso_uncertainty()</code> .
Net value	Value of the genetic gain a training set delivers minus the cost of its plots; maximised by <code>tso_economics()</code> .

Term	Meaning
p-rep design	A partially replicated field design: a fraction of the lines get a second plot, in another block.
Serpentine order	Plot numbering that runs left to right in one row and right to left in the next; <code>tso_layout()</code> blocks are runs of it.
Coverage (genotyping)	The relationship of an individual to its closest genotyped relative; <code>tso_genotype_set()</code> maximises its mean.
Transferability	Accuracy of training in one programme for another, relative to training in the other programme itself; <code>tso_transfer()</code> .
Bridge lines	Local lines phenotyped in the target programme to connect it with the training programme.
Gaussian kernel	$\exp(-d_{ij}^2/\text{median } d^2)$ on standardised markers, the RKHS kernel that captures epistasis.
BrAPI	The Breeding API, a standard web interface to breeding databases (BMS, Breedbase, GIGWA).

15.3 Session information

```
#> R version 4.5.3 (2026-03-11)
#> TSOpt 0.5.0 | platform aarch64-apple-darwin20
#> Built 30 September 2026, 12:17
```

References

- Aguilar, Ignacio, Ignacy Misztal, D. L. Johnson, Andres Legarra, Shogo Tsuruta, and T. J. Lawlor. 2010. "Hot Topic: A Unified Approach to Utilize Phenotypic, Full Pedigree, and Genomic Information for Genetic Evaluation of Holstein Final Score." *Journal of Dairy Science* 93: 743–52.
- Akdemir, Deniz, Julio I. Sanchez, and Jean-Luc Jannink. 2015. "Optimization of Genomic Selection Training Populations with a Genetic Algorithm." *Genetics Selection Evolution* 47: 38. <https://doi.org/10.1186/s12711-015-0116-6>.
- Atkinson, A. C., A. N. Donev, and R. D. Tobias. 2007. *Optimum Experimental Designs, with SAS*. Oxford University Press.
- Beavis, William D. 1994. "The Power and Deceit of QTL Experiments: Lessons from Comparative QTL Studies." *Proceedings of the 49th Annual Corn and Sorghum Industry Research Conference* (Washington, DC), 250–66.
- Bernardo, Rex. 2014. "Genomewide Selection When Major Genes Are Known." *Crop Science* 54: 68–75. <https://doi.org/10.2135/cropsci2013.05.0315>.
- Cook, R. Dennis, and Christopher J. Nachtsheim. 1980. "A Comparison of Algorithms for Constructing Exact D-Optimal Designs." *Technometrics* 22 (3): 315–24. <https://doi.org/10.1080/00401706.1980.10486162>.
- Cullis, Brian R., Alison B. Smith, and Neil E. Coombes. 2006. "On the Design of Early Generation Variety Trials with Correlated Data." *Journal of Agricultural, Biological, and Environmental Statistics* 11 (4): 381–93. <https://doi.org/10.1198/108571106X154443>.
- Daetwyler, Hans D., Beatriz Villanueva, and John A. Woolliams. 2008. "Accuracy of Predicting the

- Genetic Risk of Disease Using a Genome-Wide Approach." *PLoS ONE* 3 (10): e3395. <https://doi.org/10.1371/journal.pone.0003395>.
- de los Campos, Gustavo, Daniel Gianola, Guilherme J. M. Rosa, Kent A. Weigel, and José Crossa. 2010. "Semi-Parametric Genomic-Enabled Prediction of Genetic Values Using Reproducing Kernel Hilbert Spaces Methods." *Genetics Research* 92 (4): 295–308. <https://doi.org/10.1017/S0016672310000285>.
- Deb, Kalyanmoy, Amrit Pratap, Sameer Agarwal, and T. Meyarivan. 2002. "A Fast and Elitist Multiobjective Genetic Algorithm: NSGA-II." *IEEE Transactions on Evolutionary Computation* 6 (2): 182–97. <https://doi.org/10.1109/4235.996017>.
- Efron, Bradley, and Robert J. Tibshirani. 1993. *An Introduction to the Bootstrap*. Chapman & Hall.
- Endelman, Jeffrey B. 2011. "Ridge Regression and Other Kernels for Genomic Selection with R Package rrBLUP." *The Plant Genome* 4 (3): 250–55. <https://doi.org/10.3835/plantgenome2011.08.0024>.
- Falconer, D. S., and Trudy F. C. Mackay. 1996. *Introduction to Quantitative Genetics*. 4th ed. Longman.
- Fedorov, V. V. 1972. *Theory of Optimal Experiments*. Academic Press.
- Fernández-González, Javier, Deniz Akdemir, and Julio Isidro y Sánchez. 2023. "A Comparison of Methods for Training Population Optimization in Genomic Selection." *Theoretical and Applied Genetics* 136: 30. <https://doi.org/10.1007/s00122-023-04265-6>.
- Frazier, Peter, Warren Powell, and Savas Dayanik. 2009. "The Knowledge-Gradient Policy for Correlated Normal Beliefs." *INFORMS Journal on Computing* 21 (4): 599–613. <https://doi.org/10.1287/ijoc.1080.0314>.
- Gabriel, Stacey B., Stephen F. Schaffner, Huy Nguyen, et al. 2002. "The Structure of Haplotype Blocks in the Human Genome." *Science* 296: 2225–29.
- Gaynor, R. Chris, Gregor Gorjanc, and John M. Hickey. 2021. "AlphaSimR: An R Package for Breeding Program Simulations." *G3: Genes, Genomes, Genetics* 11 (2): jkaa017. <https://doi.org/10.1093/g3journal/jkaa017>.
- Gianola, Daniel, and Johannes B. C. H. M. van Kaam. 2008. "Reproducing Kernel Hilbert Spaces Regression Methods for Genomic Assisted Prediction of Quantitative Traits." *Genetics* 178 (4): 2289–303. <https://doi.org/10.1534/genetics.107.084285>.
- Glover, Fred. 1989. "Tabu Search—Part I." *ORSA Journal on Computing* 1 (3): 190–206. <https://doi.org/10.1287/ijoc.1.3.190>.
- Grundy, B., B. Villanueva, and J. A. Woolliams. 1998. "Dynamic Selection Procedures for Constrained Inbreeding and Their Consequences for Pedigree Development." *Genetical Research* 72: 159–68. <https://doi.org/10.1017/S0016672398003474>.
- Halko, N., P. G. Martinsson, and J. A. Tropp. 2011. "Finding Structure with Randomness: Probabilistic Algorithms for Constructing Approximate Matrix Decompositions." *SIAM Review* 53 (2): 217–88. <https://doi.org/10.1137/090771806>.
- Henderson, C. R. 1975. "Best Linear Unbiased Estimation and Prediction Under a Selection Model." *Biometrics* 31 (2): 423–47. <https://doi.org/10.2307/2529430>.

- Henderson, Charles R. 1985. "Best Linear Unbiased Prediction of Nonadditive Genetic Merits in Noninbred Populations." *Journal of Animal Science* 60: 111-17.
- Holland, John H. 1975. *Adaptation in Natural and Artificial Systems*. University of Michigan Press.
- Hussain, Waseem. 2026. *Chapter 14: Training Set Optimisation*. In: *Genomic Selection and Data-Driven Breeding, a PlantNura / KultivateEd Masterclass*. Course material, PlantNura Consulting & Solutions, Hyderabad.
- Isidro, Julio, Jean-Luc Jannink, Deniz Akdemir, Jesse Poland, Nicolas Heslot, and Mark E. Sorrells. 2015. "Training Set Optimization Under Population Structure in Genomic Selection." *Theoretical and Applied Genetics* 128: 145-58. <https://doi.org/10.1007/s00122-014-2418-4>.
- Isidro y Sánchez, Julio, and Deniz Akdemir. 2021. "Training Set Optimization for Sparse Phenotyping in Genomic Selection: A Conceptual Overview." *Frontiers in Plant Science* 12: 715910. <https://doi.org/10.3389/fpls.2021.715910>.
- Jia, Yi, and Jean-Luc Jannink. 2012. "Multiple-Trait Genomic Selection Methods Increase Genetic Value Prediction Accuracy." *Genetics* 192 (4): 1513-22. <https://doi.org/10.1534/genetics.112.144246>.
- Jiang, Yong, and Jochen C. Reif. 2015. "Modeling Epistasis in Genomic Selection." *Genetics* 201: 759-68.
- Kang, Hyun Min, Jae Hoon Sul, Susan K. Service, et al. 2010. "Variance Component Model to Account for Sample Structure in Genome-Wide Association Studies." *Nature Genetics* 42: 348-54. <https://doi.org/10.1038/ng.548>.
- Kaufman, Leonard, and Peter J. Rousseeuw. 1990. *Finding Groups in Data: An Introduction to Cluster Analysis*. Wiley.
- Kirkpatrick, S., C. D. Gelatt, and M. P. Vecchi. 1983. "Optimization by Simulated Annealing." *Science* 220 (4598): 671-80. <https://doi.org/10.1126/science.220.4598.671>.
- Laloë, D. 1993. "Precision and Information in Linear Models of Genetic Evaluation." *Genetics Selection Evolution* 25: 557-76. <https://doi.org/10.1186/1297-9686-25-6-557>.
- Legarra, Andres, Ignacio Aguilar, and Ignacy Misztal. 2009. "A Relationship Matrix Including Full Pedigree and Genomic Information." *Journal of Dairy Science* 92: 4656-63.
- Melchinger, Albrecht E., Rohan Fernando, Anna J. Melchinger, Peter Schmidt, and Matthias Frisch. 2023. "Genomic Prediction in Hybrid Breeding: I. Optimizing the Training Set Design." *Theoretical and Applied Genetics* 136: 176. <https://doi.org/10.1007/s00122-023-04413-y>.
- Meuwissen, T. H. E. 1997. "Maximizing the Response of Selection with a Predefined Rate of Inbreeding." *Genetics Selection Evolution* 29: 225-31. <https://doi.org/10.1186/1297-9686-29-2-225>.
- Meuwissen, T. H. E., B. J. Hayes, and M. E. Goddard. 2001. "Prediction of Total Genetic Value Using Genome-Wide Dense Marker Maps." *Genetics* 157 (4): 1819-29. <https://doi.org/10.1093/genetics/157.4.1819>.
- Murillo, Didier A., Salvador A. Gezan, Ana M. Heilman, Thomas C. Walk, Johan S. Aparicio, and Richard D. Horsley. 2021. "FieldHub: A Shiny App for Design of Experiments in Life Sciences." *Journal of Open Source Software* 6 (61): 3122. <https://doi.org/10.21105/joss.03122>.

- Nemhauser, G. L., L. A. Wolsey, and M. L. Fisher. 1978. "An Analysis of Approximations for Maximizing Submodular Set Functions—I." *Mathematical Programming* 14: 265–94. <https://doi.org/10.1007/BF01588971>.
- Ou, Jen-Hsiang, and Chen-Tuo Liao. 2019. "Training Set Determination for Genomic Selection." *Theoretical and Applied Genetics* 132: 2781–92. <https://doi.org/10.1007/s00122-019-03387-0>.
- Riedelsheimer, Christian, Angelika Czedik-Eysenberg, Christoph Grieder, et al. 2012. "Genomic and Metabolic Prediction of Complex Heterotic Traits in Hybrid Maize." *Nature Genetics* 44 (2): 217–20. <https://doi.org/10.1038/ng.1033>.
- Riedelsheimer, Christian, Jeffrey B. Endelman, Michael Stange, Mark E. Sorrells, Jean-Luc Jannink, and Albrecht E. Melchinger. 2013. "Genomic Predictability of Interconnected Biparental Maize Populations." *Genetics* 194 (2): 493–503. <https://doi.org/10.1534/genetics.113.150227>.
- Rincent, R., A. Charcosset, and L. Moreau. 2017. "Predicting Genomic Selection Efficiency to Optimize Calibration Set and to Assess Prediction Accuracy in Highly Structured Populations." *Theoretical and Applied Genetics* 130: 2231–47. <https://doi.org/10.1007/s00122-017-2956-7>.
- Rincent, Renaud, Jean-Paul Charpentier, Patricia Faivre-Rampant, et al. 2018. "Phenomic Selection Is a Low-Cost and High-Throughput Method Based on Indirect Predictions: Proof of Concept on Wheat and Poplar." *G3: Genes, Genomes, Genetics* 8 (12): 3961–72. <https://doi.org/10.1534/g3.118.200760>.
- Rincent, R., D. Laloë, S. Nicolas, et al. 2012. "Maximizing the Reliability of Genomic Selection by Optimizing the Calibration Set of Reference Individuals: Comparison of Methods in Two Diverse Groups of Maize Inbreds (*Zea mays* L.)." *Genetics* 192 (2): 715–28. <https://doi.org/10.1534/genetics.112.141473>.
- Rutkoski, Jessica, Jesse Poland, Suchismita Mondal, et al. 2016. "Canopy Temperature and Vegetation Indices from High-Throughput Phenotyping Improve Accuracy of Pedigree and Genomic Selection for Grain Yield in Wheat." *G3: Genes, Genomes, Genetics* 6 (9): 2799–808. <https://doi.org/10.1534/g3.116.032888>.
- Schrag, Tobias A., Matthias Westhues, Wolfgang Schipprack, et al. 2018. "Beyond Genomic Prediction: Combining Different Types of Omics Data Can Improve Prediction of Hybrid Performance in Maize." *Genetics* 208 (4): 1373–85. <https://doi.org/10.1534/genetics.117.300374>.
- Selby, Peter, Rafael Abbeloos, Jan Erik Backlund, et al. 2019. "BrAPI—an Application Programming Interface for Plant Breeding Applications." *Bioinformatics* 35 (20): 4147–55. <https://doi.org/10.1093/bioinformatics/btz190>.
- Sherman, Jack, and Winifred J. Morrison. 1950. "Adjustment of an Inverse Matrix Corresponding to a Change in One Element of a Given Matrix." *Annals of Mathematical Statistics* 21 (1): 124–27. <https://doi.org/10.1214/aoms/1177729893>.
- VanRaden, P. M. 2008. "Efficient Methods to Compute Genomic Predictions." *Journal of Dairy Science* 91 (11): 4414–23. <https://doi.org/10.3168/jds.2007-0980>.
- Vitezica, Zulma G., Luis Varona, and Andres Legarra. 2013. "On the Additive and Dominant Variance and Covariance of Individuals Within the Genomic Selection Scope." *Genetics* 195 (4): 1223–30. <https://doi.org/10.1534/genetics.113.155176>.
- Williams, Emlyn, Hans-Peter Piepho, and David Whitaker. 2011. "Augmented p-Rep Designs."

Biometrical Journal 53 (1): 19–27. <https://doi.org/10.1002/bimj.201000102>.

Zhang, Zhe, Ulrike Ober, Malena Erbe, et al. 2014. “Improving the Accuracy of Whole Genome Prediction for Complex Traits Using the Results of Genome Wide Association Studies.” *PLoS ONE* 9: e93017. <https://doi.org/10.1371/journal.pone.0093017>.